Hibernate Tutorial:

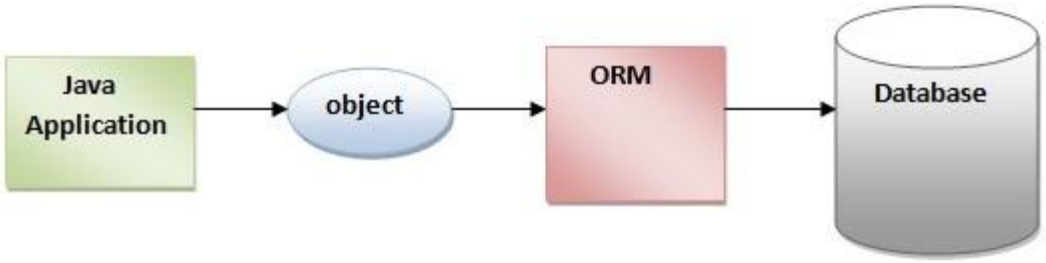


This hibernate tutorial provides in-depth concepts of Hibernate Framework with simplified examples. It was started in 2001 by Gavin King as an alternative to EJB2 style entity bean. The stable release of Hibernate till Aug 28, 2013, is hibernate 2.2.5. It is helpful for beginners and experienced persons.

Hibernate Framework

Hibernate framework simplifies the development of java application to interact with the database. Hibernate is an open source, lightweight, **ORM (Object Relational Mapping)** tool.

An ORM tool simplifies the data creation, data manipulation and data access. It is a programming technique that maps the object to the data stored in the database.



The ORM tool internally uses the JDBC API to interact with the database.

Advantages of Hibernate Framework

There are many advantages of Hibernate Framework. They are as follows:

1) Open source and Lightweight: Hibernate framework is opensource under the LGPL license and lightweight.

2) Fast performance: The performance of hibernate framework is fast because cache is internally used in hibernate framework. There are two types of cache in hibernate framework first level cache and second level cache. First level cache is enabled bydefault.

3) Database Independent query: HQL (Hibernate Query Language) is the object-oriented version of SQL. It generates the database independent queries. So you don't need to write database specific queries. Before Hibernate, If database is changed for the project, we need to change the SQL query as well that leads to the maintenance problem.

4) Automatic table creation: Hibernate framework provides the facility to create the tables of the database automatically. So there is no need to create tables in the database manually.

5) Simplifies complex join: To fetch data form multiple tables is easy in hibernate framework.

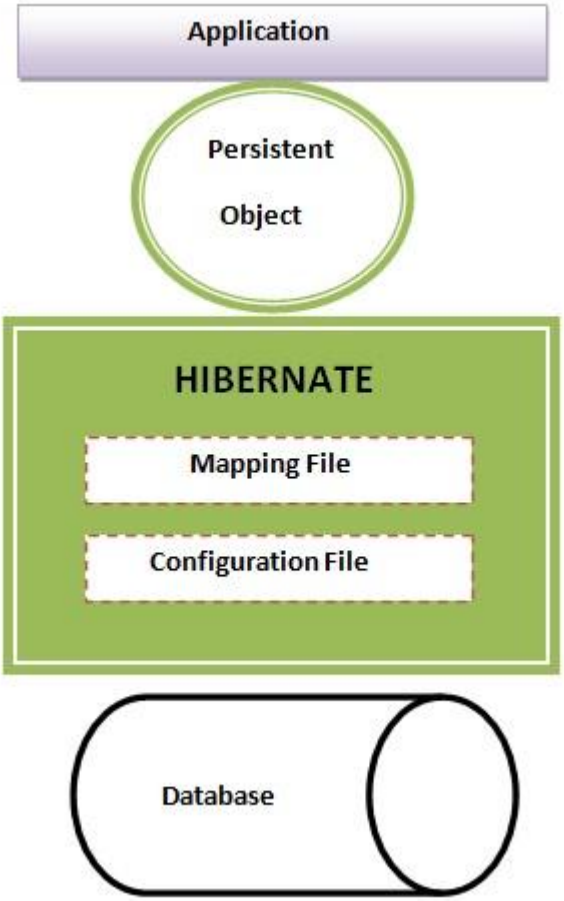
6) Provides query statistics and database status: Hibernate supports Query cache and provide statistics about query and database status.

Hibernate Architecture

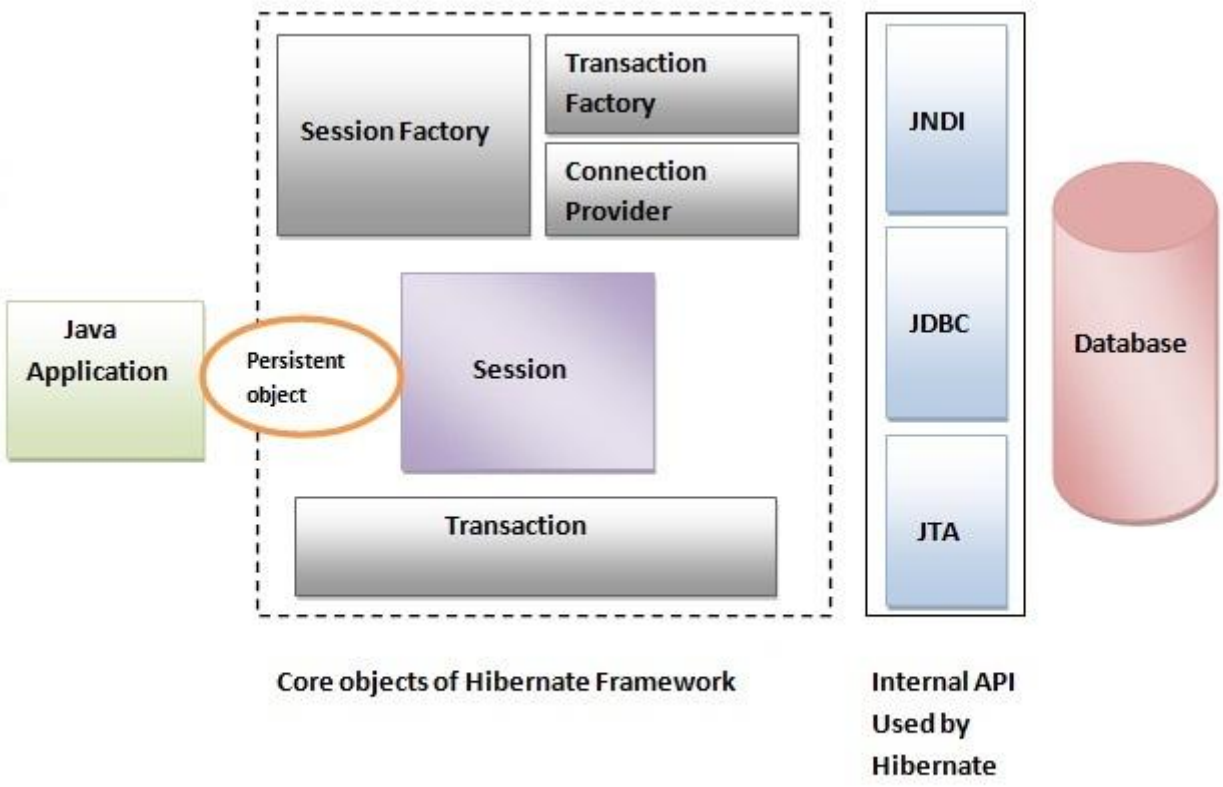
1. **Hibernate Architecture**
2. **Elements of Hibernate Architecture**
    1. **SessionFactory**
    2. **Session**
    3. **Transaction**
    4. **ConnectionProvider**
    5. **TransactionFactory**

The Hibernate architecture includes many objects persistent object, session factory, transaction factory, connection factory, session, transaction etc.

There are 4 layers in hibernate architecutre java application layer, hibernate framework layer, backhand api layer and database layer.Let's see the diagram of hibernate architecture:



This is the high level architecture of Hibernate with mapping file and configuration file.



Hibernate framework uses many objects session factory, session, transaction etc. alongwith existing Java API such as JDBC (Java Database Connectivity), JTA (Java Transaction API) and JNDI (Java Naming Directory Interface).

Elements of Hibernate Architecture

For creating the first hibernate application, we must know the elements of Hibernate architecture. They are as follows:

SessionFactory

The SessionFactory is a factory of session and client of ConnectionProvider. It holds second level cache (optional) of data. The org.hibernate.SessionFactory interface provides factory method to get the object of Session.

Session

The session object provides an interface between the application and data stored in the database. It is a short-lived object and wraps the JDBC connection. It is factory of Transaction, Query and Criteria. It holds a first-level cache (mandatory) of data. The org.hibernate.Session interface provides methods to insert, update

and delete the object. It also provides factory methods for Transaction, Query and Criteria.

Transaction

The transaction object specifies the atomic unit of work. It is optional. The org.hibernate.Transaction interface provides methods for transaction management.

ConnectionProvider

It is a factory of JDBC connections. It abstracts the application from DriverManager or DataSource. It is optional.

TransactionFactory

It is a factory of Transaction. It is optional.

Steps to create first Hibernate Application without IDE

1. **Steps to create first Hibernate Application**
    1. **Create the Persistent class**
    2. **Create the mapping file for Persistent class**
    3. **Create the Configuration file**
    4. **Create the class that retrieves or stores the persistent object**
    5. **Load the jar file**
    6. **Run the first hibernate application without IDE**

Here, we are going to create the first hibernate application without IDE. For creating the first hibernate application, we need to follow following steps:

1. Create the Persistent class
2. Create the mapping file for Persistent class
3. Create the Configuration file
4. Create the class that retrieves or stores the persistent object
5. Load the jar file
6. Run the first hibernate application without IDE

1) Create the Persistent class

A simple Persistent class should follow some rules:

- **A no-arg constructor:** It is recommended that you have a default constructor at least package visibility so that hibernate can create the instance of the Persistent class by newInstance() method.
- **Provide an identifier property (optional):** It is mapped to the primary

key column of the database.

- **Declare getter and setter methods (optional):** The Hibernate recognizes the method by getter and setter method names by default.
- **Prefer non-final class:** Hibernate uses the concept of proxies, that depends on the persistent class. The application programmer will not be able to use proxies for lazy association fetching.

Let's create the simple Persistent class:

Employee.java

copy to clipboard

```
1.    package com.javatpoint.mypackage;
2.
3.    public class Employee {
4.    private int id;
5.    private String firstName,lastName;
6.
7.    public int getId() {
8.       return id;
9.    }
10.   public void setId(int id) {
11.      this.id = id;
12.   }
13.   public String getFirstName() {
14.      return firstName;
15.   }
16.   public void setFirstName(String firstName) {
17.      this.firstName = firstName;
18.   }
19.   public String getLastName() {
20.      return lastName;
21.   }
22.   public void setLastName(String lastName) {
23.      this.lastName = lastName;
24.   }
25.
26.
27.   }
```

2) Create the mapping file for Persistent class

The mapping file name conventially, should be class_name.hbm.xml. There are many elements of the mapping file.

- **hibernate-mapping** is the root element in the mapping file.
- **class** It is the sub-element of the hibernate-mapping element. It specifies the Persistent class.
- **id** It is the subelement of class. It specifies the primary key attribute in the class.
- **generator** It is the subelement of id. It is used to generate the primary key. There are many generator classes such as assigned (It is used if id is

specified by the user), increment, hilo, sequence, native etc. We will learn all the generator classes later.

- **property** It is the subelement of class that specifies the property name of the Persistent class.

Let's see the mapping file for the Employee class:

employee.hbm.xml

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-mapping PUBLIC
3.       "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.       "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.       <hibernate-mapping>
7.        <class name="com.javatpoint.mypackage.Employee" table="emp1000">
8.          <id name="id">
9.           <generator class="assigned"></generator>
10.         </id>
11.
12.         <property name="firstName"></property>
13.         <property name="lastName"></property>
14.
15.       </class>
16.
17.      </hibernate-mapping>
```

---

3) Create the Configuration file

The configuration file contains informations about the database and mapping file. Conventionally, its name should be hibernate.cfg.xml .

hibernate.cfg.xml

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-configuration PUBLIC
3.               "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.               "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6.      <hibernate-configuration>
7.
8.         <session-factory>
9.            <property name="hbm2ddl.auto">update</property>
10.           <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>

11.           <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</p
    roperty>
12.           <property name="connection.username">system</property>
13.           <property name="connection.password">oracle</property>
```

```
14.             <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver
    </property>
15.         <mapping resource="employee.hbm.xml"/>
16.         </session-factory>
17.
18.     </hibernate-configuration>
```

4) Create the class that retrieves or stores the object

In this class, we are simply storing the employee object to the database.

copy to clipboard

```
1.      package com.javatpoint.mypackage;
2.
3.      import org.hibernate.Session;
4.      import org.hibernate.SessionFactory;
5.      import org.hibernate.Transaction;
6.      import org.hibernate.cfg.Configuration;
7.
8.      public class StoreData {
9.      public static void main(String[] args) {
10.
11.         //creating configuration object
12.         Configuration cfg=new Configuration();
13.         cfg.configure("hibernate.cfg.xml");//populates the data of the configuration file

14.
15.         //creating seession factory object
16.         SessionFactory factory=cfg.buildSessionFactory();
17.
18.         //creating session object
19.         Session session=factory.openSession();
20.
21.         //creating transaction object
22.         Transaction t=session.beginTransaction();
23.
24.         Employee e1=new Employee();
25.         e1.setId(115);
26.         e1.setFirstName("sonoo");
27.         e1.setLastName("jaiswal");
28.
29.         session.persist(e1);//persisting the object
30.
31.         t.commit();//transaction is commited
32.         session.close();
33.
34.         System.out.println("successfully saved");
35.
36.     }
37.     }
```

5) Load the jar file

For successfully running the hibernate application, you should have the hibernate4.jar file. **download the latest hibernate jar file**. Some other jar files or packages are required such as

- cglib
- log4j
- commons
- SLF4J
- dom4j
- xalan
- xerces

download the required jar files for hibernate

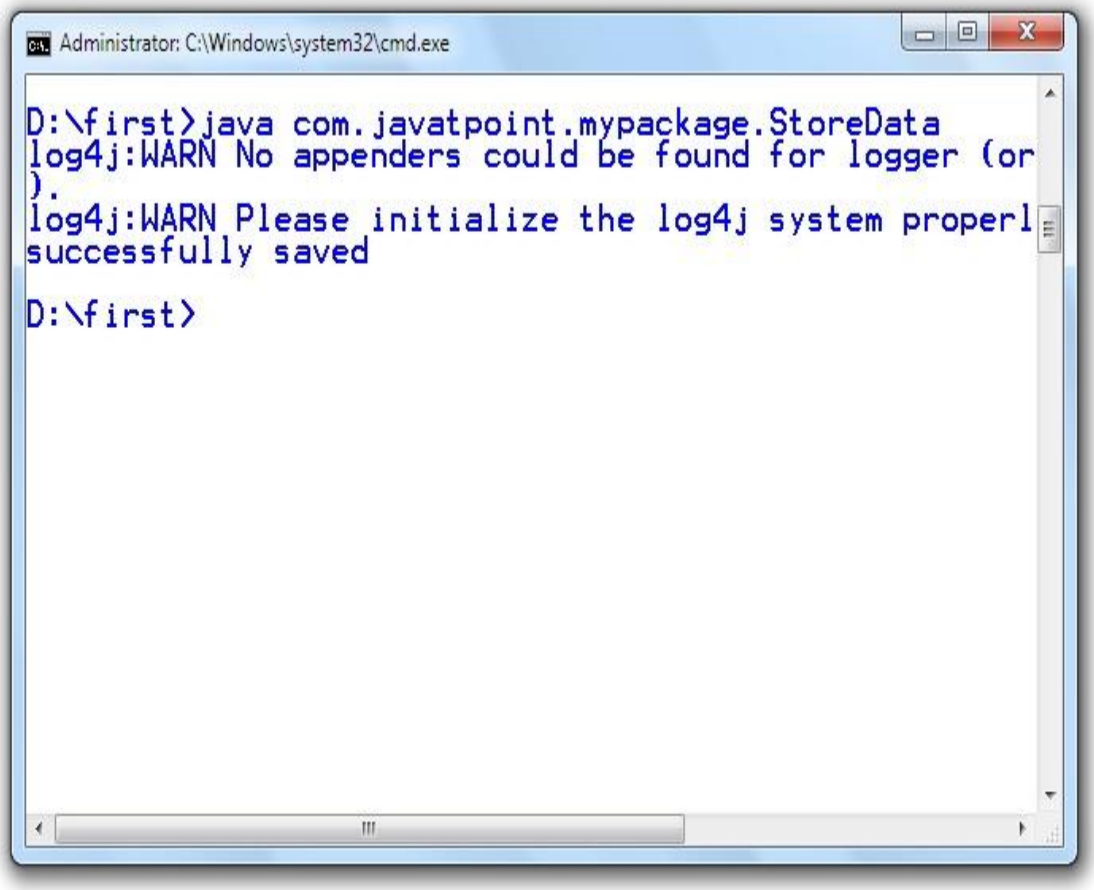
---

6) How to run the first hibernate application without IDE

We may run this hibernate application by IDE (e.g. Eclipse, Myeclipse, Netbeans etc.) or without IDE. We will learn about creating hibernate application in Eclipse IDE in next chapter.

To run the hibernate application without IDE:

- install the oracle10g for this example.
- load the jar files for hibernate. (One of the way to load the jar file is copy all the jar files under the JRE/lib/ext folder). It is better to put these jar files inside the public and private JRE both.
- Now Run the StoreData class by **java com.javatpoint.mypackage.StoreData**



Note: You must need to connect with the internet to run this example.

Example to create the Hibernate Application in Eclipse IDE

1. Example to create the Hibernate Application in Eclipse IDE
    1. Create the java project
    2. Add jar files for hibernate
    3. Create the Persistent class
    4. Create the mapping file for Persistent class
    5. Create the Configuration file
    6. Create the class that retrieves or stores the persistent object
    7. Run the application

Here, we are going to create a simple example of hibernate application using eclipse IDE. For creating the first hibernate application in Eclipse IDE, we need to follow following steps:

1. Create the java project
2. Add jar files for hibernate

3. Create the Persistent class
4. Create the mapping file for Persistent class
5. Create the Configuration file
6. Create the class that retrieves or stores the persistent object
7. Run the application

1) Create the java project

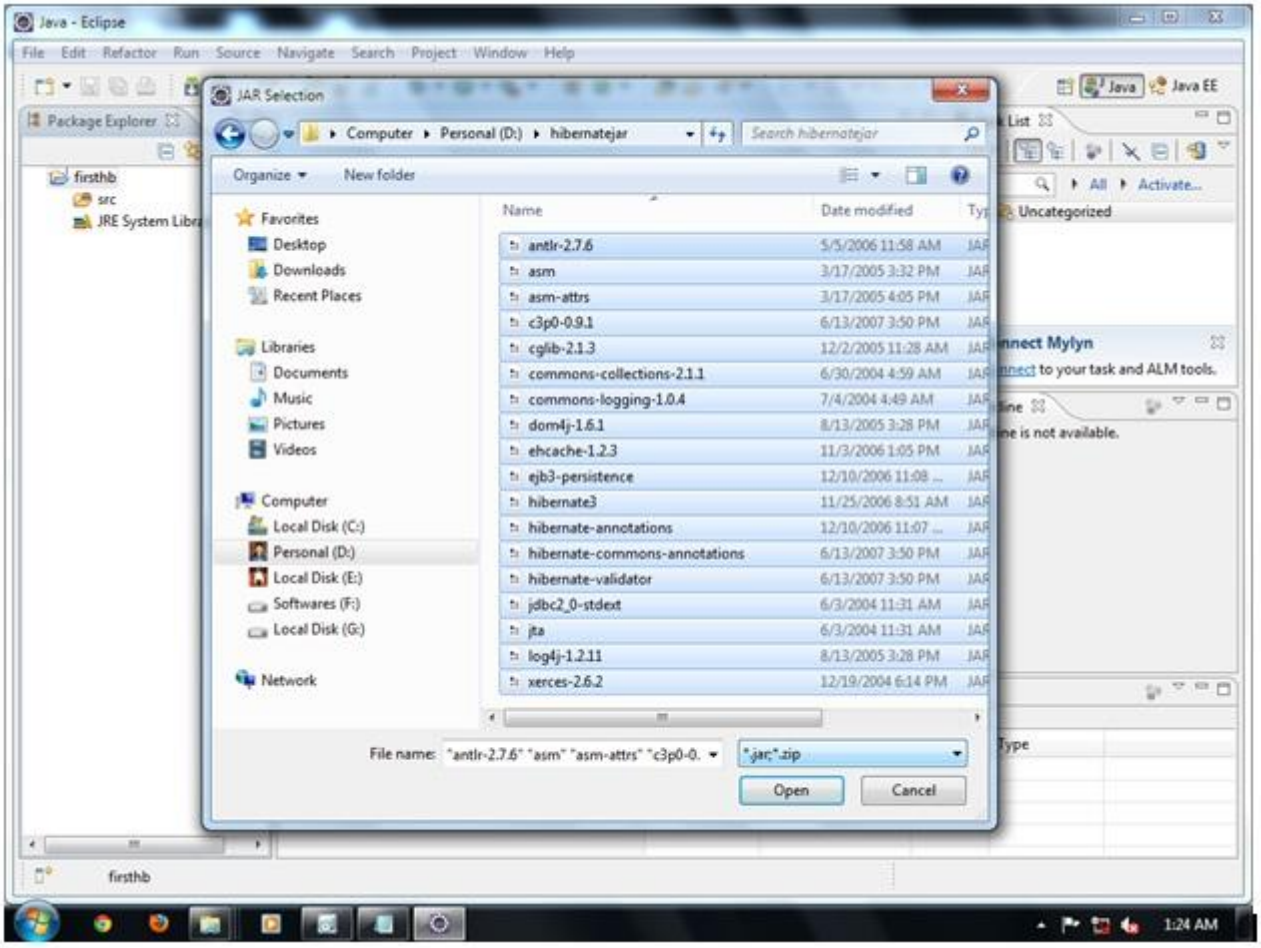
Create the java project by **File Menu** - **New** - **project** - **java project** . Now specify the project name e.g. firsthb then **next** - **finish** .

---

2) Add jar files for hibernate

To add the jar files **Right click on your project** - **Build path** - **Add external archives**. Now select all the jar files as shown in the image given below then click open.

download the required jar file



In this example, we are connecting the application with oracle database. So you must add the ojdbc14.jar file.

download the ojdbc14.jar file

---

3) Create the Persistent class

Here, we are creating the same persistent class which we have created in the previous topic. To create the persistent class, Right click on src - New - Class - specify the class with package name (e.g. com.javatpoint.mypackage) - finish.

Employee.java

copy to clipboard

```
1.      package com.javatpoint.mypackage;
2.
3.      public class Employee {
4.      private int id;
5.      private String firstName,lastName;
6.
7.      public int getId() {
8.         return id;
9.      }
10.     public void setId(int id) {
11.        this.id = id;
12.     }
13.     public String getFirstName() {
14.        return firstName;
15.     }
16.     public void setFirstName(String firstName) {
17.        this.firstName = firstName;
18.     }
19.     public String getLastName() {
20.        return lastName;
21.     }
22.     public void setLastName(String lastName) {
23.        this.lastName = lastName;
24.     }
25.
26.
27.     }
```

---

4) Create the mapping file for Persistent class

Here, we are creating the same mapping file as created in the previous topic. To create the mapping file, Right click on src - new - file - specify the file name (e.g. employee.hbm.xml) - ok. It must be outside the package.

employee.hbm.xml

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-mapping PUBLIC
3.       "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.       "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.       <hibernate-mapping>
7.        <class name="com.javatpoint.mypackage.Employee" table="emp1000">
8.          <id name="id">
9.           <generator class="assigned"></generator>
10.         </id>
11.
12.         <property name="firstName"></property>
13.         <property name="lastName"></property>
14.
15.       </class>
16.
17.      </hibernate-mapping>
```

---

5) Create the Configuration file

The configuration file contains all the informations for the datbase such as connection_url, driver_class, username, password etc. The hbm2ddl.auto property is used to create the table in the database automatically. We will have in-depth learning about Dialect class in next topics. To create the configuration file, right click on src - new - file. Now specify the configuration file name e.g. hibernate.cfg.xml.

hibernate.cfg.xml

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-configuration PUBLIC
3.              "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.              "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6.      <hibernate-configuration>
7.
8.         <session-factory>
9.            <property name="hbm2ddl.auto">update</property>
10.           <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>

11.           <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</p
    roperty>
12.           <property name="connection.username">system</property>
13.           <property name="connection.password">oracle</property>
14.           <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver
    </property>
15.        <mapping resource="employee.hbm.xml"/>
16.        </session-factory>
```

```
17.
18.     </hibernate-configuration>
```

6) Create the class that retrieves or stores the persistent object

In this class, we are simply storing the employee object to the database.

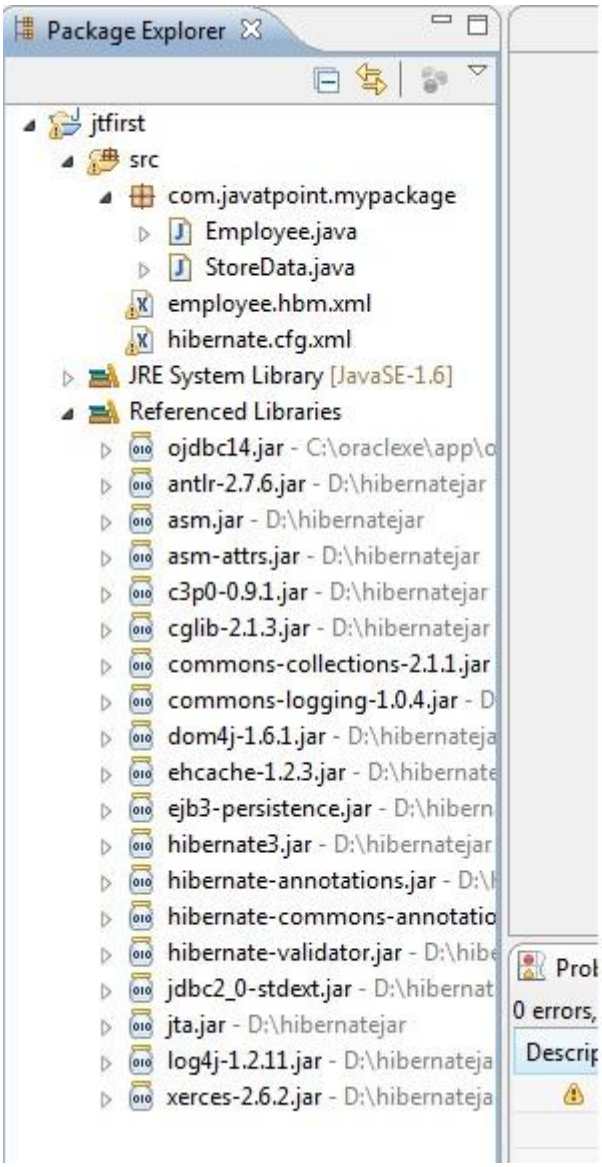
copy to clipboard

```
1.      package com.javatpoint.mypackage;
2.
3.      import org.hibernate.Session;
4.      import org.hibernate.SessionFactory;
5.      import org.hibernate.Transaction;
6.      import org.hibernate.cfg.Configuration;
7.
8.      public class StoreData {
9.      public static void main(String[] args) {
10.
11.         //creating configuration object
12.         Configuration cfg=new Configuration();
13.         cfg.configure("hibernate.cfg.xml");//populates the data of the configuration file

14.
15.         //creating seession factory object
16.         SessionFactory factory=cfg.buildSessionFactory();
17.
18.         //creating session object
19.         Session session=factory.openSession();
20.
21.         //creating transaction object
22.         Transaction t=session.beginTransaction();
23.
24.         Employee e1=new Employee();
25.         e1.setId(115);
26.         e1.setFirstName("sonoo");
27.         e1.setLastName("jaiswal");
28.
29.         session.persist(e1);//persisting the object
30.
31.         t.commit();//transaction is commited
32.         session.close();
33.
34.         System.out.println("successfully saved");
35.
36.     }
37.     }
```

7) Run the application

Before running the application, determine that directory structure is like this.



To run the hibernate application, right click on the StoreData class - Run As - Java Application.

Note: You must need to connect with the internet to run this example.

Example to create the Hibernate Application in MyEclipse

1. **Example to create the Hibernate Application in MyEclipse**
2. **Create the java project**
3. **Add hibernate capabilities**

4. **Create the Persistent class**
5. **Create the mapping file for Persistent class**
6. **Add mapping of hbm file in configuration file**
7. **Create the class that retrieves or stores the persistent object**
8. **Add jar file for oracle**
9. **Run the application**

Here, we are going to create a simple example of hibernate application using myeclipse IDE. For creating the first hibernate application in MyEclipse IDE, we need to follow following steps:

1. Create the java project
2. Add hibernate capabilities
3. Create the Persistent class
4. Create the mapping file for Persistent class
5. Add mapping of hbm file in configuration file
6. Create the class that retrieves or stores the persistent object
7. Add jar file for oracle
8. Run the application

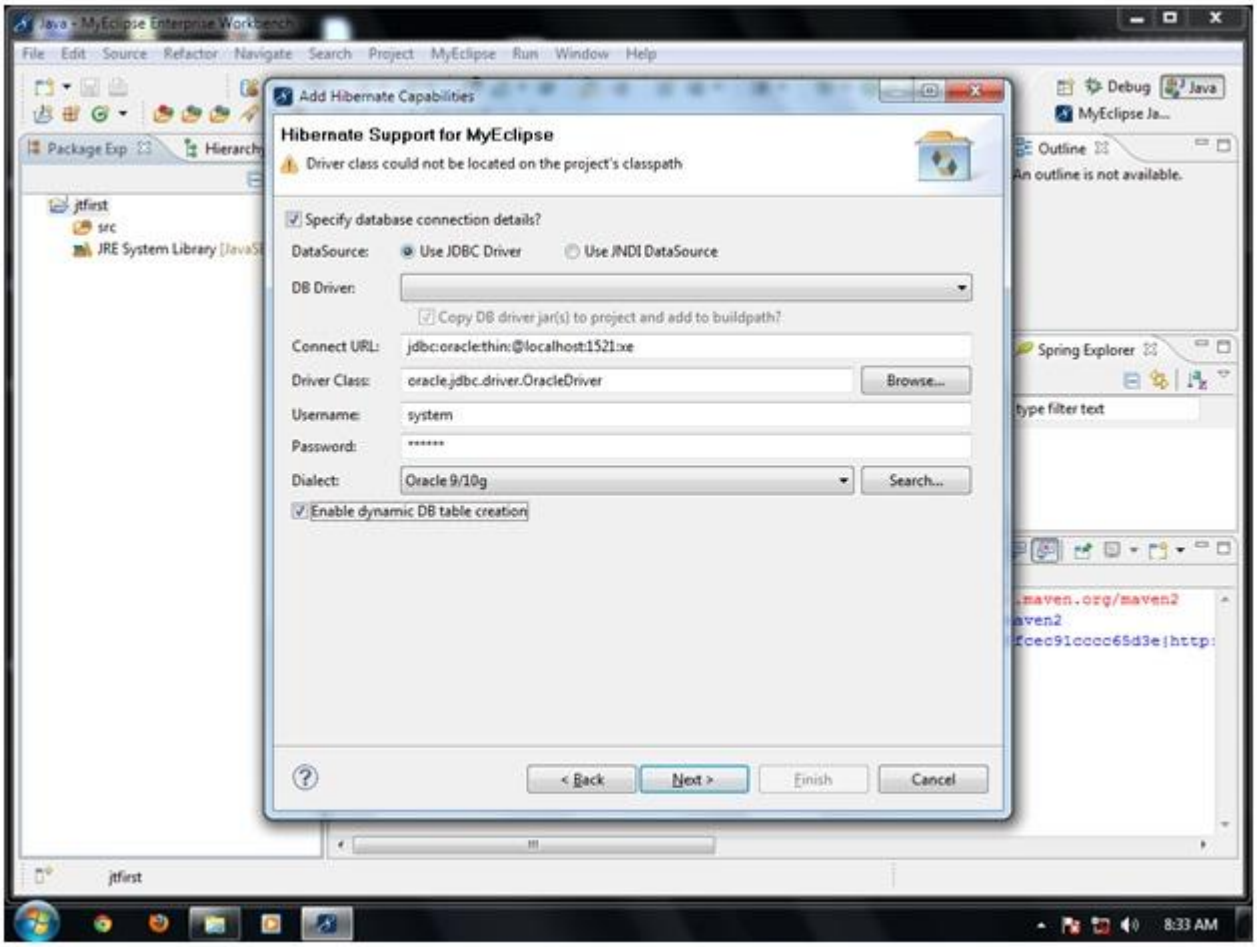
1) Create the java project

Create the java project by **File Menu** - **New** - **project** - **java project** . Now specify the project name e.g. firsthb then **next** - **finish** .

---

2) Add hibernate capabilities

To add the jar files **select your project** - **click on MyEclipse menu** - **Project Capabilities** - **add Hibernate capabilities**- **next**- **next**. Now specify the database connection details as displayed in the figure below.



Here, **Check the Enable dynamic table creation check box** to create automatic table - **next** - **Uncheck the create SessionFactory class** because we are going to write the code to get session object self for better understanding - **finish**.

Now configuration file will be created automatically.

---

3) Create the Persistent class

Here, we are creating the same persistent class which we have created in the previous topic. To create the persistent class, Right click on src - New - Class - specify the class with package name (e.g. com.javatpoint.mypackage) - finish.

Employee.java

copy to clipboard

```
1.      package com.javatpoint.mypackage;
2.
3.      public class Employee {
4.      private int id;
5.      private String firstName,lastName;
```

```
6.
7.      public int getId() {
8.         return id;
9.      }
10.     public void setId(int id) {
11.        this.id = id;
12.     }
13.     public String getFirstName() {
14.        return firstName;
15.     }
16.     public void setFirstName(String firstName) {
17.        this.firstName = firstName;
18.     }
19.     public String getLastName() {
20.        return lastName;
21.     }
22.     public void setLastName(String lastName) {
23.        this.lastName = lastName;
24.     }
25.
26.
27.     }
```

4) Create the mapping file for Persistent class

Here, we are creating the same mapping file as created in the previous topic. To create the mapping file, Right click on src - new - file - specify the file name (e.g. employee.hbm.xml) - ok. It must be outside the package. Copy the dtd for this mapping file from this example after downloading it.

employee.hbm.xml

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-mapping PUBLIC
3.       "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.       "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.       <hibernate-mapping>
7.        <class name="com.javatpoint.mypackage.Employee" table="emp1000">
8.          <id name="id">
9.           <generator class="assigned"></generator>
10.         </id>
11.
12.         <property name="firstName"></property>
13.         <property name="lastName"></property>
14.
15.       </class>
16.
17.      </hibernate-mapping>
```

5) Add mapping of hbm file in configuration file

open the hibernate.cgf.xml file, and add an entry of mapping resource like this:

copy to clipboard

```
1.      <mapping resource="employee.hbm.xml"/>
```

Now the configuration file will look like this:

hibernate.cfg.xml

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-configuration PUBLIC
3.              "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.              "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6.      <hibernate-configuration>
7.
8.         <session-factory>
9.            <property name="hbm2ddl.auto">update</property>
10.           <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>

11.           <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</p
    roperty>
12.           <property name="connection.username">system</property>
13.           <property name="connection.password">oracle</property>
14.           <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver
    </property>
15.        <mapping resource="employee.hbm.xml"/>
16.        </session-factory>
17.
18.     </hibernate-configuration>
```

---

6) Create the class that retrieves or stores the persistent object

In this class, we are simply storing the employee object to the database.

copy to clipboard

```
1.      package com.javatpoint.mypackage;
2.
3.      import org.hibernate.Session;
4.      import org.hibernate.SessionFactory;
5.      import org.hibernate.Transaction;
6.      import org.hibernate.cfg.Configuration;
7.
8.      public class StoreData {
9.      public static void main(String[] args) {
10.
```

```
11.         //creating configuration object
12.         Configuration cfg=new Configuration();
13.         cfg.configure("hibernate.cfg.xml");//populates the data of the configuration file

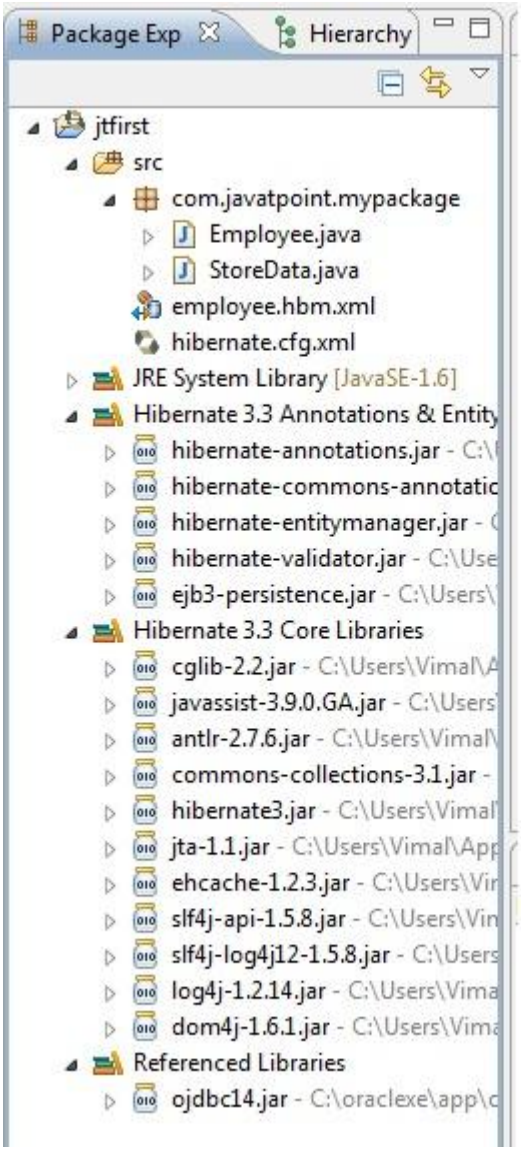
14.
15.         //creating seession factory object
16.         SessionFactory factory=cfg.buildSessionFactory();
17.
18.         //creating session object
19.         Session session=factory.openSession();
20.
21.         //creating transaction object
22.         Transaction t=session.beginTransaction();
23.
24.         Employee e1=new Employee();
25.         e1.setId(115);
26.         e1.setFirstName("sonoo");
27.         e1.setLastName("jaiswal");
28.
29.         session.persist(e1);//persisting the object
30.
31.         t.commit();//transaction is commited
32.         session.close();
33.
34.         System.out.println("successfully saved");
35.
36.     }
37.     }
```

7) Add the jar file for oracle (ojdbc14.jar)

To add the ojdbc14.jar file, right click on your project - build path - add external archives - select the ojdbc14.jar file - open.

8) Run the application

Before running the application, determine that directory structure is like this.



To run the hibernate application, right click on the StoreData class - Run As - Java Application.

Hibernate with Annotation

1. **Hibernate with Annotation**
2. **Example to create the hibernate application with Annotation**
    1. **Add the jar file for annotation**
    2. **Create the Persistent class**
    3. **Add mapping of Persistent class in configuration file**
    4. **Create the class that retrieves or stores the persistent object**

The hibernate application can be created with annotation. There are many annotations that can be used to create hibernate application such as @Entity, @Id,

@Table etc.

Hibernate Annotations are based on the JPA 2 specification and supports all the features.

All the JPA annotations are defined in the javax.persistence.* package. Hibernate **EntityManager** implements the interfaces and life cycle defined by the JPA specification.

The core advantage of using hibernate annotation is that you don't need to create mapping (hbm) file. Here, hibernate annotations are used to provide the meta data.

Example to create the hibernate application with Annotation

There are 4 steps to create the hibernate application with annotation.

1. **Add the jar file for oracle (if your database is oracle) and annotation**
2. **Create the Persistent class**
3. **Add mapping of Persistent class in configuration file**
4. **Create the class that retrieves or stores the persistent object**

---

1) Add the jar file for oracle and annotation

For oracle you need to add **ojdbc14.jar** file. For using annotation, you need to add:

- **hibernate-commons-annotations.jar**
- **ejb3-persistence.jar**
- **hibernate-annotations.jar**

---

2) Create the Persistent class

Here, we are creating the same persistent class which we have created in the previous topic. But here, we are using annotation.

@Entity annotation marks this class as an entity.

@Table annotation specifies the table name where data of this entity is to be persisted. If you don't use @Table annotation, hibernate will use the class name as the table name bydefault.

@Id annotation marks the identifier for this entity.

@Column annotation specifies the details of the column for this property or field. If @Column annotation is not specified, property name will be used as the column name bydefault.

Employee.java

copy to clipboard

```
1.      package com.javatpoint;
2.
3.      import javax.persistence.Entity;
4.      import javax.persistence.Id;
5.      import javax.persistence.Table;
6.
7.      @Entity
8.      @Table(name= "emp500")
9.      public class Employee {
10.     @Id
11.     private int id;
12.     private String firstName,lastName;
13.
14.     public int getId() {
15.        return id;
16.     }
17.     public void setId(int id) {
18.        this.id = id;
19.     }
20.     public String getFirstName() {
21.        return firstName;
22.     }
23.     public void setFirstName(String firstName) {
24.        this.firstName = firstName;
25.     }
26.     public String getLastName() {
27.        return lastName;
28.     }
29.     public void setLastName(String lastName) {
30.        this.lastName = lastName;
31.     }
32.
33.
34.     }
```

---

3) Add mapping of Persistent class in configuration file

open the hibernate.cgf.xml file, and add an entry of mapping resource like this:

copy to clipboard

```
1.      <mapping class="com.javatpoint.Employee"/>
```

Now the configuration file will look like this:

hibernate.cfg.xml

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-configuration PUBLIC
3.              "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.              "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6.      <hibernate-configuration>
7.
8.      <session-factory>
9.       <property name="hbm2ddl.auto">create</property>
10.      <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
11.      <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</proper
    ty>
12.      <property name="connection.username">system</property>
13.      <property name="connection.password">oracle</property>
14.      <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</pro
    perty>
15.
16.      <mapping class="com.javatpoint.Employee"/>
17.      </session-factory>
18.
19.      </hibernate-configuration>
```

---

4) Create the class that retrieves or stores the persistent object

In this class, we are simply storing the employee object to the database. Here, we are using the **AnnotationConfiguration** class to get the information of mapping from the persistent class.

copy to clipboard

```
1.      package com.javatpoint.mypackage;
2.
3.      package com.javatpoint;
4.
5.      import org.hibernate.*;
6.      import org.hibernate.cfg.*;
7.
8.      public class Test {
9.      public static void main(String[] args) {
10.        Session session=new AnnotationConfiguration()
11.            .configure().buildSessionFactory().openSession();
12.
13.        Transaction t=session.beginTransaction();
14.
15.        Employee e1=new Employee();
16.        e1.setId(1001);
17.        e1.setFirstName("sonoo");
18.        e1.setLastName("jaiswal");
19.
```

```
20.         Employee e2=new Employee();
21.         e2.setId(1002);
22.         e2.setFirstName("vimal");
23.         e2.setLastName("jaiswal");
24.
25.         session.persist(e1);
26.         session.persist(e2);
27.
28.         t.commit();
29.         session.close();
30.         System.out.println("successfully saved");
31.     }
32.     }
```

Web Application with Hibernate

1. **Web Application with Hibernate**
2. **Example to create web application using hibernate**

Here, we are going to create a web application with hibernate. For creating the web application, we are using JSP for presentation logic, Bean class for representing data and DAO class for database codes.

As we create the simple application in hibernate, we don't need to perform any extra operations in hibernate for creating web application. In such case, we are getting the value from the user using the JSP file.

Example to create web application using hibernate

In this example, we are going to insert the record of the user in the database. It is simply a registration form.

---

index.jsp

This page gets input from the user and sends it to the register.jsp file using post method.

copy to clipboard

```
1.      <form action="register.jsp" method="post">
2.      Name:<input type="text" name="name"/><br><br/>
3.      Password:<input type="password" name="password"/><br><br/>
4.      Email ID:<input type="text" name="email"/><br><br/>
5.      <input type="submit" value="register"/>"
6.
7.      </form>
```

---

register.jsp

This file gets all request parameters and stores this information into an object of User class. Further, it calls the register method of UserDao class passing the User class object.

copy to clipboard

```
1.      <%@page import="com.javatpoint.mypack.UserDao"%>
2.      <jsp:useBean id="obj" class="com.javatpoint.mypack.User">
3.      </jsp:useBean>
4.      <jsp:setProperty property="*" name="obj"/>
5.
6.      <%
7.      int i=UserDao.register(obj);
8.      if(i>0)
9.      out.print("You are successfully registered");
10.
11.     %>
```

User.java

It is the simple bean class representing the Persistent class in hibernate.

copy to clipboard

```
1.      package com.javatpoint.mypack;
2.
3.      public class User {
4.      private int id;
5.      private String name,password,email;
6.
7.      //getters and setters
8.
9.      }
```

---

user.hbm.xml

It maps the User class with the table of the database.

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-mapping PUBLIC
3.       "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.       "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.       <hibernate-mapping>
7.       <class name="com.javatpoint.mypack.User" table="u400">
8.       <id name="id">
9.       <generator class="increment"></generator>
10.      </id>
11.      <property name="name"></property>
```

```
12.       <property name="password"></property>
13.       <property name="email"></property>
14.       </class>
15.
16.       </hibernate-mapping>
```

UserDao.java

A Dao class, containing method to store the instance of User class.

copy to clipboard

```
1.        package com.javatpoint.mypack;
2.
3.
4.        import org.hibernate.Session;
5.        import org.hibernate.Transaction;
6.        import org.hibernate.cfg.Configuration;
7.
8.        public class UserDao {
9.
10.       public static int register(User u){
11.        int i=0;
12.        Session session=new Configuration().
13.         configure().buildSessionFactory().openSession();
14.
15.         Transaction t=session.beginTransaction();
16.         t.begin();
17.
18.         i=(Integer)session.save(u);
19.
20.         t.commit();
21.         session.close();
22.
23.         return i;
24.        }
25.       }
```

---

hibernate.cfg.xml

It is a configuration file, containing informations about the database and mapping file.

copy to clipboard

```
1.        <?xml version='1.0' encoding='UTF-8'?>
2.        <!DOCTYPE hibernate-configuration PUBLIC
3.                "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.                "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6.        <hibernate-configuration>
7.
8.        <session-factory>
9.         <property name="hbm2ddl.auto">create</property>
10.        <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
```

```
11.       <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</proper
    ty>
12.       <property name="connection.username">system</property>
13.       <property name="connection.password">oracle</property>
14.       <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</pro
    perty>
15.
16.       <mapping resource="user.hbm.xml"/>
17.       </session-factory>
18.
19.      </hibernate-configuration>
```

Generator classes in Hibernate

1. **Hibernate Architecture**
2. **Hibernate Framework**
3. **Advantages of Hibernate Framework**

The <generator> subelement of id used to generate the unique identifier for the objects of persistent class. There are many generator classes defined in the Hibernate Framework.

All the generator classes implements the **org.hibernate.id.IdentifierGenerator interface**. The application programmer may create one's own generator classes by implementing the IdentifierGenerator interface. Hibernate framework provides many built-in generator classes:

1. assigned
2. increment
3. sequence
4. hilo
5. native
6. identity
7. seqhilo
8. uuid
9. guid
10. select
11. foreign
12. sequence-identity

1) assigned

It is the default generator strategy if there is no <generator> element . In this case, application assigns the id. For example:

copy to clipboard

```
1.      ....
2.       <hibernate-mapping>
3.        <class ...>
4.          <id ...>
5.           <generator class="assigned"></generator>
```

```
6.        </id>
7.
8.        .....
9.
10.     </class>
11.    </hibernate-mapping>
```

---

2) increment

It generates the unique id only if no other process is inserting data into this table. It generates **short**, **int** or **long** type identifier. The first generated identifier is 1 normally and incremented as 1. Syntax:

copy to clipboard

```
1.     ....
2.      <hibernate-mapping>
3.       <class ...>
4.         <id ...>
5.          <generator class="increment"></generator>
6.         </id>
7.
8.         .....
9.
10.      </class>
11.     </hibernate-mapping>
```

---

3) sequence

It uses the sequence of the database. if there is no sequence defined, it creates a sequence automatically e.g. in case of Oracle database, it creates a sequence named HIBERNATE_SEQUENCE. In case of Oracle, DB2, SAP DB, Postgre SQL or McKoi, it uses sequence but it uses generator in interbase. Syntax:

copy to clipboard

```
1.     .....
2.      <id ...>
3.       <generator class="sequence"></generator>
4.      </id>
5.      .....
```

For defining your own sequence, use the param subelement of generator.

copy to clipboard

```
1.     .....
2.      <id ...>
3.       <generator class="sequence">
4.          <param name="sequence">your_sequence_name</param>
5.       </generator>
6.      </id>
7.      .....
```

---

4) hilo

It uses high and low algorithm to generate the id of type short, int and long. Syntax:

```
copy to clipboard
1.  .....
2.   <id ...>
3.    <generator class="hilo"></generator>
4.   </id>
5.   .....
```

---

5) native

It uses identity, sequence or hilo depending on the database vendor. Syntax:

```
copy to clipboard
1.  .....
2.   <id ...>
3.    <generator class="native"></generator>
4.   </id>
5.   .....
```

---

6) identity

It is used in Sybase, My SQL, MS SQL Server, DB2 and HypersonicSQL to support the id column. The returned id is of type short, int or long.

---

7) seqhilo

It uses high and low algorithm on the specified sequence name. The returned id is of type short, int or long.

---

8) uuid

It uses 128-bit UUID algorithm to generate the id. The returned id is of type String, unique within a network (because IP is used). The UUID is represented in hexadecimal digits, 32 in length.

---

9) guid

It uses GUID generated by database of type string. It works on MS SQL Server and MySQL.

---

10) select

It uses the primary key returned by the database trigger.

---

11) foreign

It uses the id of another associated object, mostly used with <one-to-one> association.

---

12) sequence-identity

It uses a special sequence generation strategy. It is supported in Oracle 10g drivers only.

SQL Dialects in Hibernate

For connecting any hibernate application with the database, you must specify the SQL dialects. There are many Dialects classes defined for RDBMS in the org.hibernate.dialect package. They are as follows:

RDBMS	Dialect
Oracle (any version)	org.hibernate.dialect.OracleDialect
Oracle9i	org.hibernate.dialect.Oracle9iDialect
Oracle10g	org.hibernate.dialect.Oracle10gDialect
MySQL	org.hibernate.dialect.MySQLDialect
MySQL with InnoDB	org.hibernate.dialect.MySQLInnoDBDialect
MySQL with MyISAM	org.hibernate.dialect.MySQLMyISAMDialect
DB2	org.hibernate.dialect.DB2Dialect
DB2 AS/400	org.hibernate.dialect.DB2400Dialect
DB2 OS390	org.hibernate.dialect.DB2390Dialect
Microsoft SQL Server	org.hibernate.dialect.SQLServerDialect
Sybase	org.hibernate.dialect.SybaseDialect
Sybase Anywhere	org.hibernate.dialect.SybaseAnywhereDialect
PostgreSQL	org.hibernate.dialect.PostgreSQLDialect
SAP DB	org.hibernate.dialect.SAPDBDialect
Informix	org.hibernate.dialect.InformixDialect

HypersonicSQL	org.hibernate.dialect.HSQLDialect
Ingres	org.hibernate.dialect.IngresDialect
Progress	org.hibernate.dialect.ProgressDialect
Mckoi SQL	org.hibernate.dialect.MckoiDialect
Interbase	org.hibernate.dialect.InterbaseDialect
Pointbase	org.hibernate.dialect.PointbaseDialect
FrontBase	org.hibernate.dialect.FrontbaseDialect
Firebird	org.hibernate.dialect.FirebirdDialect

Hibernate Logging by Log4j using xml file

1. **Hibernate Logging**
2. **Example of Hibernate Logging**

Logging enables the programmer to write the log details into a file permanently. Log4j and Logback frameworks can be used in hibernate framework to support logging.

There are two ways to perform logging using log4j:

1. By log4j.xml file (or)
2. By log4j.properties file

Steps to perform Hibernate Logging by Log4j using xml file

There are two ways to perform logging using log4j using xml file:

1. Load the log4j jar files with hibernate
2. Create the log4j.xml file inside the src folder (parallel with hibernate.cfg.xml file)

Example of Hibernate Logging by Log4j using xml file

You can enable logging in hibernate by following only two steps in any hibernate example. This is the first example of hibernate application with logging support using log4j.

Load the required jar files

You need to load the slf4j.jar and log4j.jar files with hibernate jar files.

download all the required jar files

Create log4j.xml file

Now you need to create log4j.xml file. In this example, all the log details will be written in the C:/javatpointlog.log file.

log4j.xml

copy to clipboard

```
1.      <?xml version="1.0" encoding="UTF-8"?>
2.      <!DOCTYPE log4j:configuration SYSTEM "log4j.dtd">
3.      <log4j:configuration xmlns:log4j="http://jakarta.apache.org/log4j/"
4.         debug="false">
5.      <appender name="CONSOLE" class="org.apache.log4j.ConsoleAppender">
6.       <layout class="org.apache.log4j.PatternLayout">
7.        <param name="ConversionPattern" value="[%d{dd/MM/yy hh:mm:ss:sss z}]
    %5p %c{2}: %m%n" />
8.       </layout>
9.      </appender>
10.        <appender name="ASYNC" class="org.apache.log4j.AsyncAppender">
11.           <appender-ref ref="CONSOLE" />
12.           <appender-ref ref="FILE" />
13.     </appender>
14.     <appender name="FILE" class="org.apache.log4j.RollingFileAppender">
15.        <param name="File" value="C:/javatpointlog.log" />
16.        <param name="MaxBackupIndex" value="100" />
17.      <layout class="org.apache.log4j.PatternLayout">
18.       <param name="ConversionPattern" value="[%d{dd/MM/yy hh:mm:ss:sss z}]
    %5p %c{2}: %m%n" />
19.     </layout>
20.     </appender>
21.        <category name="org.hibernate">
22.           <priority value="DEBUG" />
23.        </category>
24.        <category name="java.sql">
25.           <priority value="debug" />
26.        </category>
27.        <root>
28.           <priority value="INFO" />
29.           <appender-ref ref="FILE" />
30.        </root>
31.     </log4j:configuration>
```

Hibernate Logging by Log4j using properties file

1. **Hibernate Logging**
2. **Example of Hibernate Logging**

As we know, Log4j and Logback frameworks are used to support logging in hibernate, there are two ways to perform logging using log4j:

1. By log4j.xml file (or)
2. By log4j.properties file

Here, we are going to enable logging using log4j through properties file.

Steps to perform Hibernate Logging by Log4j using properties file

There are two ways to perform logging using log4j using properties file:

1. Load the log4j jar files with hibernate
2. Create the log4j.properties file inside the src folder (parallel with hibernate.cfg.xml file)

Example of Hibernate Logging by Log4j using properties file

You can enable logging in hibernate by following only two steps in any hibernate example. This is the first example of hibernate application with logging support using log4j.

Load the required jar files

You need to load the slf4j.jar and log4j.jar files with hibernate jar files.

download all the required jar files

Create log4j.properties file

Now you need to create log4j.properties file. In this example, all the log details will be written in the C:\\javatpointhibernate.log file.

log4j.properties

copy to clipboard

```
1.      # Direct log messages to a log file
2.      log4j.appender.file=org.apache.log4j.RollingFileAppender
3.      log4j.appender.file.File=C:\\javatpointhibernate.log
4.      log4j.appender.file.MaxFileSize=1MB
5.      log4j.appender.file.MaxBackupIndex=1
6.      log4j.appender.file.layout=org.apache.log4j.PatternLayout
7.      log4j.appender.file.layout.ConversionPattern=%d{ABSOLUTE} %5p %c{1}:%L -
    %m%n
8.
9.      # Direct log messages to stdout
10.     log4j.appender.stdout=org.apache.log4j.ConsoleAppender
11.     log4j.appender.stdout.Target=System.out
12.     log4j.appender.stdout.layout=org.apache.log4j.PatternLayout
13.     log4j.appender.stdout.layout.ConversionPattern=%d{ABSOLUTE} %5p %c{1}:%
    L - %m%n
14.
15.     # Root logger option
16.     log4j.rootLogger=INFO, file, stdout
```

```
17.
18.     # Log everything. Good for troubleshooting
19.     log4j.logger.org.hibernate=INFO
20.
21.     # Log all JDBC parameters
22.     log4j.logger.org.hibernate.type=ALL
```

Hibernate Inheritance Mapping Tutorial

1. **Table Per Hierarchy**
2. **Table Per Concrete class**
3. **Table Per Subclass**

We can map the inheritance hierarchy classes with the table of the database. There are three inheritance mapping strategies defined in the hibernate:

1. Table Per Hierarchy
2. Table Per Concrete class
3. Table Per Subclass

Table Per Hierarchy

In table per hierarchy mapping, single table is required to map the whole hierarchy, an extra column (known as discriminator column) is added to identify the class. But nullable values are stored in the table .

Table Per Hierarchy using xml file
Table Per Hierarchy using Annotation

Table Per Concrete class

In case of table per concrete class, tables are created as per class. But duplicate column is added in subclass tables.

Table Per Concrete class using xml file
Table Per Concrete class using Annotation

Table Per Subclass

In this strategy, tables are created as per class but related by foreign key. So there are no duplicate columns.

Table Per Subclass using xml file

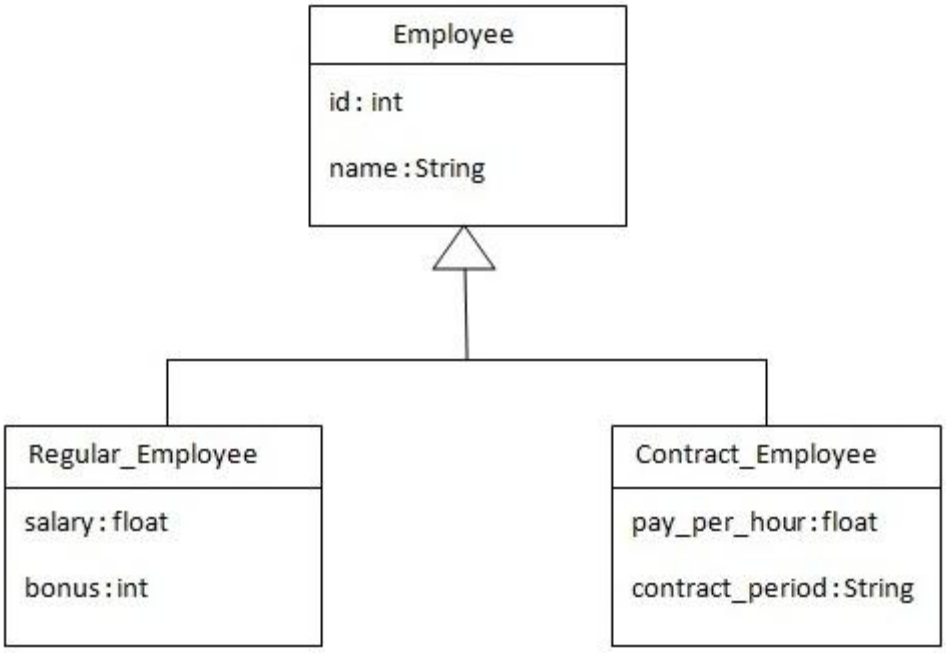
Table Per Subclass using Annotation

Hibernate Table Per Hierarchy using xml file

1. **Table Per Hierarchy**
2. **Example of Table Per Hierarchy**

By this inheritance strategy, we can map the whole hierarchy by single table only. Here, an extra column (also known as **discriminator column**) is created in the table to identify the class.

Let's understand the problem first. I want to map the whole hierarchy given below into one table of the database.



There are three classes in this hierarchy. Employee is the super class for Regular_Employee and Contract_Employee classes. Let's see the mapping file for this hierarchy.

copy to clipboard

```
1.    <?xml version='1.0' encoding='UTF-8'?>
2.    <!DOCTYPE hibernate-mapping PUBLIC
3.            "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.            "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.    <hibernate-mapping>
7.    <class name="com.javatpoint.mypackage.Employee" table="emp121" discrimina
   tor-value="emp">
8.    <id name="id">
9.    <generator class="increment"></generator>
10.   </id>
11.
12.   <discriminator column="type" type="string"></discriminator>
13.   <property name="name"></property>
```

```
14.
15.    <subclass name="com.javatpoint.mypackage.Regular_Employee" discriminator-
    value="reg_emp">
16.    <property name="salary"></property>
17.    <property name="bonus"></property>
18.    </subclass>
19.
20.    <subclass name="com.javatpoint.mypackage.Contract_Employee" discriminator-
    value="con_emp">
21.    <property name="pay_per_hour"></property>
22.    <property name="contract_duration"></property>
23.    </subclass>
24.
25.    </class>
26.
27.    </hibernate-mapping>
```

In case of table per class hierarchy an discriminator column is added by the hibernate framework that specifies the type of the record. It is mainly used to distinguish the record. To specify this, **discriminator** subelement of class must be specified.

The **subclass** subelement of class, specifies the subclass. In this case, Regular_Employee and Contract_Employee are the subclasses of Employee class.

The table structure for this hierarchy is as shown below:

Column Name	Data Type	Nullable	Default	Primary Key
ID	NUMBER(10,0)	No	-	1
TYPE	VARCHAR2(255)	No	-	-
NAME	VARCHAR2(255)	Yes	-	-
SALARY	FLOAT	Yes	-	-
BONUS	NUMBER(10,0)	Yes	-	-
PAY_PER_HOUR	FLOAT	Yes	-	-
CONTRACT_DURATION	VARCHAR2(255)	Yes	-	-
				1 - 7

Example of Table per class hierarchy

In this example we are creating the three classes and provide mapping of these classes in the employee.hbm.xml file.

1) Create the Persistent classes

You need to create the persistent classes representing the inheritance. Let's create the three classes for the above hierarchy:

File: Employee.java

copy to clipboard

```
1.    package com.javatpoint.mypackage;
2.
```

```
3.      public class Employee {
4.      private int id;
5.      private String name;
6.
7.      //getters and setters
8.      }
```

File: Regular_Employee.java

copy to clipboard

```
1.      package com.javatpoint.mypackage;
2.
3.      public class Regular_Employee extends Employee{
4.      private float salary;
5.      private int bonus;
6.
7.      //getters and setters
8.      }
```

File: Contract_Employee.java

copy to clipboard

```
1.      package com.javatpoint.mypackage;
2.
3.      public class Contract_Employee extends Employee{
4.         private float pay_per_hour;
5.         private String contract_duration;
6.
7.      //getters and setters
8.      }
```

---

2) Create the mapping file for Persistent class

The mapping has been discussed above for the hierarchy.

File: employee.hbm.xml

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-mapping PUBLIC
3.              "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.              "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.      <hibernate-mapping>
7.      <class name="com.javatpoint.mypackage.Employee" table="emp121" discrimina
    tor-value="emp">
8.      <id name="id">
9.      <generator class="increment"></generator>
10.     </id>
11.
12.     <discriminator column="type" type="string"></discriminator>
13.     <property name="name"></property>
14.
15.     <subclass name="com.javatpoint.mypackage.Regular_Employee" discriminator-
    value="reg_emp">
16.     <property name="salary"></property>
17.     <property name="bonus"></property>
18.     </subclass>
19.
```

```
20.    <subclass name="com.javatpoint.mypackage.Contract_Employee" discriminator-value="con_emp">
21.    <property name="pay_per_hour"></property>
22.    <property name="contract_duration"></property>
23.    </subclass>
24.
25.    </class>
26.
27.    </hibernate-mapping>
```

---

3) Add mapping of hbm file in configuration file

Open the hibernate.cgf.xml file, and add an entry of mapping resource like this:

copy to clipboard

```
1.    <mapping resource="employee.hbm.xml"/>
```

Now the configuration file will look like this:

File: hibernate.cfg.xml

copy to clipboard

```
1.    <?xml version='1.0' encoding='UTF-8'?>
2.    <!DOCTYPE hibernate-configuration PUBLIC
3.            "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.            "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6.    <hibernate-configuration>
7.
8.       <session-factory>
9.          <property name="hbm2ddl.auto">update</property>
10.         <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>

11.         <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
12.         <property name="connection.username">system</property>
13.         <property name="connection.password">oracle</property>
14.         <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
15.      <mapping resource="employee.hbm.xml"/>
16.      </session-factory>
17.
18.   </hibernate-configuration>
```

The hbm2ddl.auto property is defined for creating automatic table in the database.

---

4) Create the class that stores the persistent object

In this class, we are simply storing the employee objects in the database.

File: StoreData.java

copy to clipboard

```
1.    package com.javatpoint.mypackage;
2.    import org.hibernate.*;
3.    import org.hibernate.cfg.*;
4.
5.    public class StoreData {
```

```
6.     public static void main(String[] args) {
7.        Session session=new Configuration().configure("hibernate.cfg.xml")
8.                        .buildSessionFactory().openSession();
9.
10.       Transaction t=session.beginTransaction();
11.
12.       Employee e1=new Employee();
13.       e1.setName("sonoo");
14.
15.       Regular_Employee e2=new Regular_Employee();
16.       e2.setName("Vivek Kumar");
17.       e2.setSalary(50000);
18.       e2.setBonus(5);
19.
20.       Contract_Employee e3=new Contract_Employee();
21.       e3.setName("Arjun Kumar");
22.       e3.setPay_per_hour(1000);
23.       e3.setContract_duration("15 hours");
24.
25.       session.persist(e1);
26.       session.persist(e2);
27.       session.persist(e3);
28.
29.       t.commit();
30.       session.close();
31.       System.out.println("success");
32.    }
33.    }
```

Output:

EDIT	ID	TYPE	NAME	SALARY	BONUS	PAY_PER_HOUR	CONTRACT_DURATION
	1	emp	sonoo	-	-	-	-
	2	reg_emp	Vivek Kumar	50000	5	-	-
	3	con_emp	Arjun Kumar	-	-	1000	15 hours
							row(s) 1 - 3 of 3

Hibernate Table Per Hierarchy using Annotation

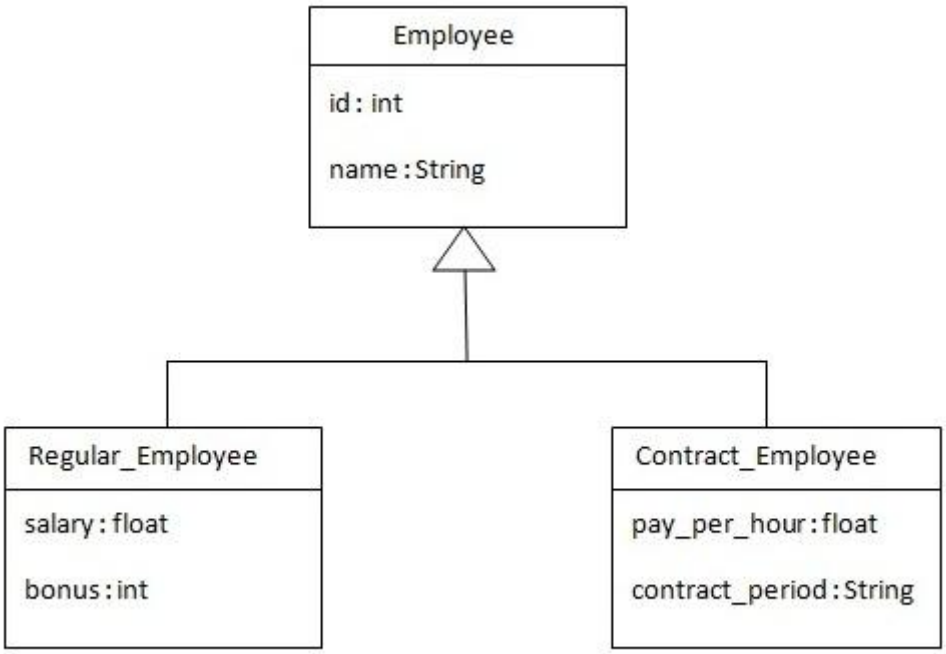
1. **Table Per Hierarchy**
2. **Example of Table Per Hierarchy**

In the previous page, we have mapped the inheritance hierarchy with one table only using xml file. Here, we are going to perform this task using annotation. You need to

use @Inheritance(strategy=InheritanceType.SINGLE_TABLE), @DiscriminatorColumn and @DiscriminatorValue annotations for mapping table per hierarchy strategy.

In case of table per hierarchy, only one table is required to map the inheritance hierarchy. Here, an extra column (also known as **discriminator column**) is created in the table to identify the class.

Let's see the inheritance hierarchy:



There are three classes in this hierarchy. Employee is the super class for Regular_Employee and Contract_Employee classes.

The table structure for this hierarchy is as shown below:

Column Name	Data Type	Nullable	Default	Primary Key
ID	NUMBER(10,0)	No	-	1
TYPE	VARCHAR2(255)	No	-	-
NAME	VARCHAR2(255)	Yes	-	-
SALARY	FLOAT	Yes	-	-
BONUS	NUMBER(10,0)	Yes	-	-
PAY_PER_HOUR	FLOAT	Yes	-	-
CONTRACT_DURATION	VARCHAR2(255)	Yes	-	-
				1 - 7

Example of Hibernate Table Per Hierarchy using Annotation

You need to follow following steps to create simple example:

- Create the persistent classes
- Create the configuration file
- Create the class to store the fetch the data

1) Create the Persistent classes

You need to create the persistent classes representing the inheritance. Let's create the three classes for the above hierarchy:

File: Employee.java

copy to clipboard

```
1.      package com.javatpoint.mypackage;
2.      import javax.persistence.*;
3.
4.      @Entity
5.      @Table(name = "employee101")
6.      @Inheritance(strategy=InheritanceType.SINGLE_TABLE)
7.      @DiscriminatorColumn(name="type",discriminatorType=DiscriminatorType.STRIN
    G)
8.      @DiscriminatorValue(value="employee")
9.
10.     public class Employee {
11.     @Id
12.     @GeneratedValue(strategy=GenerationType.AUTO)
13.
14.     @Column(name = "id")
15.     private int id;
16.
17.     @Column(name = "name")
18.     private String name;
19.
20.     //setters and getters
21.     }
```

File: Regular_Employee.java

copy to clipboard

```
1.      package com.javatpoint.mypackage;
2.
3.      import javax.persistence.*;
4.
5.      @Entity
6.      @DiscriminatorValue("regularemployee")
7.      public class Regular_Employee extends Employee{
8.
9.      @Column(name="salary")
10.     private float salary;
11.
12.     @Column(name="bonus")
13.     private int bonus;
14.
15.     //setters and getters
16.     }
```

File: Contract_Employee.java

copy to clipboard

```
1.      package com.javatpoint.mypackage;
2.
3.      import javax.persistence.Column;
4.      import javax.persistence.DiscriminatorValue;
5.      import javax.persistence.Entity;
6.
7.      @Entity
8.      @DiscriminatorValue("contractemployee")
9.      public class Contract_Employee extends Employee{
10.
11.        @Column(name="pay_per_hour")
12.        private float pay_per_hour;
13.
14.        @Column(name="contract_duration")
15.        private String contract_duration;
16.
17.        //setters and getters
18.     }
```

---

2) Add the persistent classes in configuration file

Open the hibernate.cgf.xml file, and add entries of entity classes like this:

copy to clipboard

```
1.      <mapping class="com.javatpoint.mypackage.Employee"/>
2.       <mapping class="com.javatpoint.mypackage.Contract_Employee"/>
3.       <mapping class="com.javatpoint.mypackage.Regular_Employee"/>
```

Now the configuration file will look like this:

File: hibernate.cfg.xml

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-configuration PUBLIC
3.              "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.              "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6.      <!-- Generated by MyEclipse Hibernate Tools.                   -->
7.      <hibernate-configuration>
8.
9.          <session-factory>
10.             <property name="hbm2ddl.auto">update</property>
11.             <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>

12.             <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</p
    roperty>
13.             <property name="connection.username">system</property>
14.             <property name="connection.password">oracle</property>
```

```
15.           <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver
    </property>
16.
17.           <mapping class="com.javatpoint.mypackage.Employee"/>
18.           <mapping class="com.javatpoint.mypackage.Contract_Employee"/>
19.           <mapping class="com.javatpoint.mypackage.Regular_Employee"/>
20.       </session-factory>
21.
22.   </hibernate-configuration>
```

The hbm2ddl.auto property is defined for creating automatic table in the database.

---

3) Create the class that stores the persistent object

In this class, we are simply storing the employee objects in the database.

File: StoreTest.java

copy to clipboard

```
1.    package com.javatpoint.mypackage;
2.
3.    import org.hibernate.*;
4.    import org.hibernate.cfg.*;
5.
6.    public class StoreData {
7.    public static void main(String[] args) {
8.        AnnotationConfiguration cfg=new AnnotationConfiguration();
9.        Session session=cfg.configure("hibernate.cfg.xml").buildSessionFactory().open
    Session();
10.
11.       Transaction t=session.beginTransaction();
12.
13.       Employee e1=new Employee();
14.       e1.setName("sonoo");
15.
16.       Regular_Employee e2=new Regular_Employee();
17.       e2.setName("Vivek Kumar");
18.       e2.setSalary(50000);
19.       e2.setBonus(5);
20.
21.       Contract_Employee e3=new Contract_Employee();
22.       e3.setName("Arjun Kumar");
23.       e3.setPay_per_hour(1000);
24.       e3.setContract_duration("15 hours");
25.
26.       session.persist(e1);
27.       session.persist(e2);
28.       session.persist(e3);
29.
30.       t.commit();
31.       session.close();
32.       System.out.println("success");
33.   }
```

```
34.     }
```

Output:

EDIT	ID	TYPE	NAME	SALARY	BONUS	PAY_PER_HOUR	CONTRACT_DURATION
	1	emp	sonoo	-	-	-	-
	2	reg_emp	Vivek Kumar	50000	5	-	-
	3	con_emp	Arjun Kumar	-	-	1000	15 hours
							row(s) 1 - 3 of 3

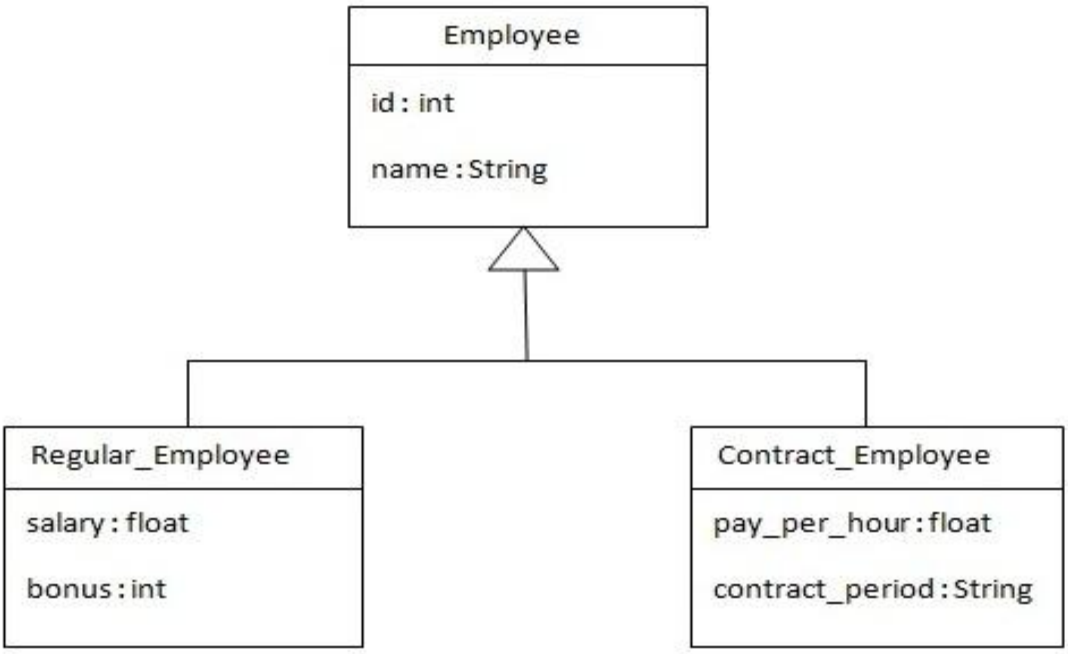
Table Per Concrete class using xml file

1. **Table Per Concrete class**
2. **Example of Table Per Concrete class**

In case of Table Per Concrete class, there will be three tables in the database having no relations to each other. There are two ways to map the table with table per concrete class strategy.

- By union-subclass element
- By Self creating the table for each class

Let's understand what hierarchy we are going to map.



Let's see how can we map this hierarchy by union-subclass element:

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-mapping PUBLIC
3.              "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.              "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.       <hibernate-mapping>
7.       <class name="com.javatpoint.mypackage.Employee" table="emp122">
8.       <id name="id">
9.       <generator class="increment"></generator>
10.      </id>
11.
12.      <property name="name"></property>
13.
14.      <union-
    subclass name="com.javatpoint.mypackage.Regular_Employee" table="regemp122">

15.      <property name="salary"></property>
16.      <property name="bonus"></property>
17.      </union-subclass>
18.
19.      <union-
    subclass name="com.javatpoint.mypackage.Contract_Employee" table="contemp122
    ">
20.      <property name="pay_per_hour"></property>
21.      <property name="contract_duration"></property>
22.      </union-subclass>
23.
24.      </class>
25.
26.      </hibernate-mapping>
```

In case of table per concrete class, there will be three tables in the database, each representing a particular class.

The **union-subclass** subelement of class, specifies the subclass. It adds the columns of parent table into this table. In other words, it is working as a union.

The table structure for each table will be as follows:

Table structure for Employee class

Column Name	Data Type	Nullable	Default	Primary Key
ID	NUMBER(10,0)	No	-	1
NAME	VARCHAR2(255)	Yes	-	-
				1 - 2

Table structure for Regular_Employee class

Column Name	Data Type	Nullable	Default	Primary Key
ID	NUMBER(10,0)	No	-	1
NAME	VARCHAR2(255)	Yes	-	-
SALARY	FLOAT	Yes	-	-
BONUS	NUMBER(10,0)	Yes	-	-
				1 - 4

Table structure for Contract_Employee class

Column Name	Data Type	Nullable	Default	Primary Key
ID	NUMBER(10,0)	No	-	1
NAME	VARCHAR2(255)	Yes	-	-
PAY_PER_HOUR	FLOAT	Yes	-	-
CONTRACT_DURATION	VARCHAR2(255)	Yes	-	-
				1 - 4

Example of Table per concrete class

In this example we are creating the three classes and provide mapping of these classes in the employee.hbm.xml file.

1) Create the Persistent classes

You need to create the persistent classes representing the inheritance. Let's create the three classes for the above hierarchy:

File: Employee.java

copy to clipboard

```
package com.javatpoint.mypackage;

public class Employee {
private int id;
private String name;

//getters and setters
}
```

File: Regular_Employee.java

copy to clipboard

```
package com.javatpoint.mypackage;

public class Regular_Employee extends Employee{
private float salary;
private int bonus;

//getters and setters
}
```

File: Contract_Employee.java

copy to clipboard

```
package com.javatpoint.mypackage;

public class Contract_Employee extends Employee{
   private float pay_per_hour;
   private String contract_duration;

//getters and setters
}
```

---

2) Create the mapping file for Persistent class

The mapping has been discussed above for the hierarchy.

File: employee.hbm.xml

copy to clipboard

```
<?xml version='1.0' encoding='UTF-8'?>
<!DOCTYPE hibernate-mapping PUBLIC
        "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
        "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
```

```
5.
6.
7.        <hibernate-mapping>
8.        <class name="com.javatpoint.mypackage.Employee" table="emp122">
9.        <id name="id">
10.       <generator class="increment"></generator>
11.       </id>
12.
13.       <property name="name"></property>
14.
15.       <union-
    subclass name="com.javatpoint.mypackage.Regular_Employee" table="regemp122">

16.       <property name="salary"></property>
17.       <property name="bonus"></property>
18.       </union-subclass>
19.
20.       <union-
    subclass name="com.javatpoint.mypackage.Contract_Employee" table="contemp122
    ">
21.       <property name="pay_per_hour"></property>
22.       <property name="contract_duration"></property>
23.       </union-subclass>
24.
25.       </class>
26.
27.       </hibernate-mapping>
```

---

3) Add mapping of hbm file in configuration file

Open the hibernate.cgf.xml file, and add an entry of mapping resource like this:

copy to clipboard

```
1.      <mapping resource="employee.hbm.xml"/>
```

Now the configuration file will look like this:

File: hibernate.cfg.xml

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-configuration PUBLIC
3.              "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.              "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6.      <hibernate-configuration>
7.
8.         <session-factory>
9.            <property name="hbm2ddl.auto">update</property>
10.           <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>

11.           <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</p
    roperty>
```

```
12.           <property name="connection.username">system</property>
13.           <property name="connection.password">oracle</property>
14.           <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver
    </property>
15.        <mapping resource="employee.hbm.xml"/>
16.        </session-factory>
17.
18.     </hibernate-configuration>
```

The hbm2ddl.auto property is defined for creating automatic table in the database.

---

4) Create the class that stores the persistent object

In this class, we are simply storing the employee objects in the database.

File: StoreData.java

copy to clipboard

```
1.      package com.javatpoint.mypackage;
2.
3.      import org.hibernate.*;
4.      import org.hibernate.cfg.*;
5.
6.      public class StoreData {
7.      public static void main(String[] args) {
8.         Session session=new Configuration().configure("hibernate.cfg.xml")
9.                         .buildSessionFactory().openSession();
10.
11.        Transaction t=session.beginTransaction();
12.
13.        Employee e1=new Employee();
14.        e1.setName("sonoo");
15.
16.        Regular_Employee e2=new Regular_Employee();
17.        e2.setName("Vivek Kumar");
18.        e2.setSalary(50000);
19.        e2.setBonus(5);
20.
21.        Contract_Employee e3=new Contract_Employee();
22.        e3.setName("Arjun Kumar");
23.        e3.setPay_per_hour(1000);
24.        e3.setContract_duration("15 hours");
25.
26.        session.persist(e1);
27.        session.persist(e2);
28.        session.persist(e3);
29.
30.        t.commit();
31.        session.close();
32.        System.out.println("success");
33.     }
34.     }
```

Table Per Concrete class using Annotation

1. **Table Per Concrete class using annotation**

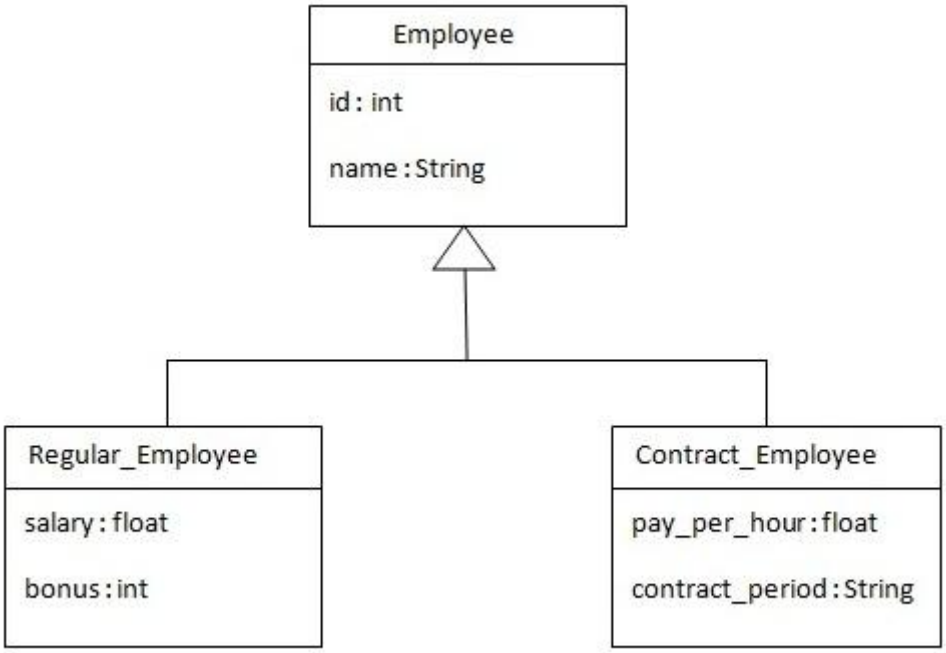
In case of Table Per Concrete class, tables are created per class. So there are no nullable values in the table. Disadvantage of this approach is that duplicate columns are created in the subclass tables.

Here, we need to use @Inheritance(strategy = InheritanceType.TABLE_PER_CLASS) annotation in the parent class and @AttributeOverrides annotation in the subclasses.

@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS) specifies that we are using table per concrete class strategy. It should be specified in the parent class only.

@AttributeOverrides defines that parent class attributes will be overriden in this class. In table structure, parent class table columns will be added in the subclass table.

The class hierarchy is given below:



The table structure for each table will be as follows:

Table structure for Employee class

Column Name	Data Type	Nullable	Default	Primary Key
ID	NUMBER(10,0)	No	-	1
NAME	VARCHAR2(255)	Yes	-	-
				1 - 2

Table structure for Regular_Employee class

Column Name	Data Type	Nullable	Default	Primary Key
ID	NUMBER(10,0)	No	-	1
NAME	VARCHAR2(255)	Yes	-	-
SALARY	FLOAT	Yes	-	-
BONUS	NUMBER(10,0)	Yes	-	-
				1 - 4

Table structure for Contract_Employee class

Column Name	Data Type	Nullable	Default	Primary Key
ID	NUMBER(10,0)	No	-	1
NAME	VARCHAR2(255)	Yes	-	-
PAY_PER_HOUR	FLOAT	Yes	-	-
CONTRACT_DURATION	VARCHAR2(255)	Yes	-	-
				1 - 4

Example of Table per concrete class

In this example we are creating the three classes and provide mapping of these classes in the employee.hbm.xml file.

1) Create the Persistent classes

You need to create the persistent classes representing the inheritance. Let's create the three classes for the above hierarchy:

File: Employee.java

copy to clipboard

```
1.      package com.javatpoint.mypackage;
2.      import javax.persistence.*;
3.
4.      @Entity
5.      @Table(name = "employee102")
6.      @Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
7.
8.      public class Employee {
9.      @Id
10.     @GeneratedValue(strategy=GenerationType.AUTO)
11.
12.     @Column(name = "id")
13.     private int id;
14.
```

```
15.     @Column(name = "name")
16.     private String name;
17.
18.     //setters and getters
19.     }
```

File: Regular_Employee.java

copy to clipboard

```
1.      package com.javatpoint.mypackage;
2.      import javax.persistence.*;
3.
4.      @Entity
5.      @Table(name="regularemployee102")
6.      @AttributeOverrides({
7.          @AttributeOverride(name="id", column=@Column(name="id")),
8.          @AttributeOverride(name="name", column=@Column(name="name"))
9.      })
10.     public class Regular_Employee extends Employee{
11.
12.     @Column(name="salary")
13.     private float salary;
14.
15.     @Column(name="bonus")
16.     private int bonus;
17.
18.     //setters and getters
19.     }
```

File: Contract_Employee.java

copy to clipboard

```
1.      package com.javatpoint.mypackage;
2.      import javax.persistence.*;
3.      @Entity
4.      @Table(name="contractemployee102")
5.      @AttributeOverrides({
6.          @AttributeOverride(name="id", column=@Column(name="id")),
7.          @AttributeOverride(name="name", column=@Column(name="name"))
8.      })
9.      public class Contract_Employee extends Employee{
10.
11.         @Column(name="pay_per_hour")
12.         private float pay_per_hour;
13.
14.         @Column(name="contract_duration")
15.         private String contract_duration;
16.
17.         public float getPay_per_hour() {
18.             return pay_per_hour;
19.         }
20.         public void setPay_per_hour(float payPerHour) {
21.             pay_per_hour = payPerHour;
22.         }
23.         public String getContract_duration() {
24.             return contract_duration;
25.         }
```

```
26.         public void setContract_duration(String contractDuration) {
27.             contract_duration = contractDuration;
28.         }
29.     }
```

---

2) Add mapping of hbm file in configuration file

File: hibernate.cfg.xml

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-configuration PUBLIC
3.              "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.              "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6.      <!-- Generated by MyEclipse Hibernate Tools.                   -->
7.      <hibernate-configuration>
8.          <session-factory>
9.              <property name="hbm2ddl.auto">update</property>
10.             <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>

11.             <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</p
    roperty>
12.             <property name="connection.username">system</property>
13.             <property name="connection.password">oracle</property>
14.     <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</pro
    perty>
15.
16.             <mapping class="com.javatpoint.mypackage.Employee"/>
17.             <mapping class="com.javatpoint.mypackage.Contract_Employee"/>
18.             <mapping class="com.javatpoint.mypackage.Regular_Employee"/>
19.         </session-factory>
20.     </hibernate-configuration>
```

The hbm2ddl.auto property is defined for creating automatic table in the database.

---

3) Create the class that stores the persistent object

In this class, we are simply storing the employee objects in the database.

File: StoreData.java

copy to clipboard

```
1.      package com.javatpoint.mypackage;
2.
3.      import org.hibernate.*;
4.      import org.hibernate.cfg.*;
5.
6.      public class StoreData {
7.      public static void main(String[] args) {
8.      AnnotationConfiguration cfg=new AnnotationConfiguration();
```

```
9.      Session session=cfg.configure("hibernate.cfg.xml").buildSessionFactory().openSe
    ssion();
10.
11.         Transaction t=session.beginTransaction();
12.
13.         Employee e1=new Employee();
14.         e1.setName("sonoo");
15.
16.         Regular_Employee e2=new Regular_Employee();
17.         e2.setName("Vivek Kumar");
18.         e2.setSalary(50000);
19.         e2.setBonus(5);
20.
21.         Contract_Employee e3=new Contract_Employee();
22.         e3.setName("Arjun Kumar");
23.         e3.setPay_per_hour(1000);
24.         e3.setContract_duration("15 hours");
25.
26.         session.persist(e1);
27.         session.persist(e2);
28.         session.persist(e3);
29.
30.         t.commit();
31.         session.close();
32.         System.out.println("success");
33.     }
34.     }
```

Table Per Subclass Example using xml file

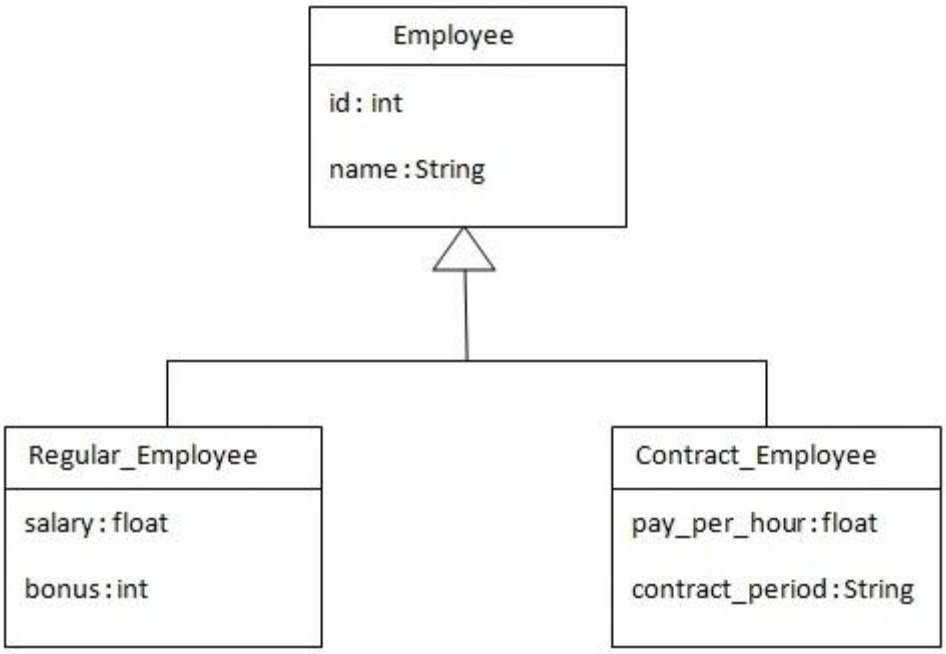
1. **Table Per Subclass class**
2. **Example of Table Per Subclass**

In case of Table Per Subclass, subclass mapped tables are related to parent class mapped table by primary key and foreign key relationship.

The **<joined-subclass>** element of class is used to map the child class with parent using the primary key and foreign key relation.

In this example, we are going to use hb2ddl.auto property to generate the table automatically. So we don't need to be worried about creating tables in the database.

Let's see the hierarchy of classes that we are going to map.



Let's see how can we map this hierarchy by joined-subclass element:

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-mapping PUBLIC
3.
4.        "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
5.        "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
6.
7.
8.        <hibernate-mapping>
9.        <class name="com.javatpoint.mypackage.Employee" table="emp123">
10.       <id name="id">
11.       <generator class="increment"></generator>
12.       </id>
13.
14.       <property name="name"></property>
15.
16.       <joined-
    subclass name="com.javatpoint.mypackage.Regular_Employee" table="regemp123">

17.       <key column="eid"></key>
18.       <property name="salary"></property>
19.       <property name="bonus"></property>
20.       </joined-subclass>
21.
22.       <joined-
    subclass name="com.javatpoint.mypackage.Contract_Employee" table="contemp123
    ">
23.       <key column="eid"></key>
24.       <property name="pay_per_hour"></property>
25.       <property name="contract_duration"></property>
26.       </joined-subclass>
```

```
27.
28.      </class>
29.      </hibernate-mapping>
```

In case of table per subclass class, there will be three tables in the database, each representing a particular class.

The **joined-subclass** subelement of class, specifies the subclass. The **key** subelement of joined-subclass is used to generate the foreign key in the subclass mapped table. This foreign key will be associated with the primary key of parent class mapped table.

The table structure for each table will be as follows:

Table structure for Employee class

Column Name	Data Type	Nullable	Default	Primary Key
ID	NUMBER(10,0)	No	-	1
NAME	VARCHAR2(255)	Yes	-	-
				1 - 2

Table structure for Regular_Employee class

Column Name	Data Type	Nullable	Default	Primary Key
EID	NUMBER(10,0)	No	-	1
SALARY	FLOAT	Yes	-	-
BONUS	NUMBER(10,0)	Yes	-	-
				1 - 3

Table structure for Contract_Employee class

Column Name	Data Type	Nullable	Default	Primary Key
EID	NUMBER(10,0)	No	-	1
PAY_PER_HOUR	FLOAT	Yes	-	-
CONTRACT_DURATION	VARCHAR2(255)	Yes	-	-
				1 - 3

Example of Table per subclass class

In this example we are creating the three classes and provide mapping of these classes in the employee.hbm.xml file.

1) Create the Persistent classes

You need to create the persistent classes representing the inheritance. Let's create the three classes for the above hierarchy:

File: Employee.java

copy to clipboard

```
package com.javatpoint.mypackage;

public class Employee {
private int id;
private String name;

//getters and setters
}
```

File: Regular_Employee.java

copy to clipboard

```
package com.javatpoint.mypackage;

public class Regular_Employee extends Employee{
private float salary;
private int bonus;

//getters and setters
}
```

File: Contract_Employee.java

copy to clipboard

```
package com.javatpoint.mypackage;

public class Contract_Employee extends Employee{
   private float pay_per_hour;
   private String contract_duration;

//getters and setters
}
```

---

2) Create the mapping file for Persistent class

The mapping has been discussed above for the hierarchy.

File: employee.hbm.xml

copy to clipboard

```
<?xml version='1.0' encoding='UTF-8'?>
<!DOCTYPE hibernate-mapping PUBLIC

  "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
```

```
5.        "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
6.
7.
8.        <hibernate-mapping>
9.        <class name="com.javatpoint.mypackage.Employee" table="emp123">
10.       <id name="id">
11.       <generator class="increment"></generator>
12.       </id>
13.
14.       <property name="name"></property>
15.
16.       <joined-
    subclass name="com.javatpoint.mypackage.Regular_Employee" table="regemp123">

17.       <key column="eid"></key>
18.       <property name="salary"></property>
19.       <property name="bonus"></property>
20.       </joined-subclass>
21.
22.       <joined-
    subclass name="com.javatpoint.mypackage.Contract_Employee" table="contemp123
    ">
23.       <key column="eid"></key>
24.       <property name="pay_per_hour"></property>
25.       <property name="contract_duration"></property>
26.       </joined-subclass>
27.
28.       </class>
29.       </hibernate-mapping>
```

---

3) create configuration file

Open the hibernate.cgf.xml file, and add an entry of mapping resource like this:

copy to clipboard

```
1.      <mapping resource="employee.hbm.xml"/>
```

Now the configuration file will look like this:

File: hibernate.cfg.xml

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-configuration PUBLIC
3.              "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.              "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6.      <hibernate-configuration>
7.
8.         <session-factory>
9.            <property name="hbm2ddl.auto">update</property>
10.           <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
```

```
11.           <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</p
    roperty>
12.           <property name="connection.username">system</property>
13.           <property name="connection.password">oracle</property>
14.           <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver
    </property>
15.         <mapping resource="employee.hbm.xml"/>
16.         </session-factory>
17.
18.      </hibernate-configuration>
```

The hbm2ddl.auto property is defined for creating automatic table in the database.

---

4) Create the class that stores the persistent object

In this class, we are simply storing the employee objects in the database.

File: StoreData.java

copy to clipboard

```
1.       package com.javatpoint.mypackage;
2.
3.       import org.hibernate.*;
4.       import org.hibernate.cfg.*;
5.
6.       public class StoreData {
7.       public static void main(String[] args) {
8.          Session session=new Configuration().configure("hibernate.cfg.xml")
9.                          .buildSessionFactory().openSession();
10.
11.         Transaction t=session.beginTransaction();
12.
13.         Employee e1=new Employee();
14.         e1.setName("sonoo");
15.
16.         Regular_Employee e2=new Regular_Employee();
17.         e2.setName("Vivek Kumar");
18.         e2.setSalary(50000);
19.         e2.setBonus(5);
20.
21.         Contract_Employee e3=new Contract_Employee();
22.         e3.setName("Arjun Kumar");
23.         e3.setPay_per_hour(1000);
24.         e3.setContract_duration("15 hours");
25.
26.         session.persist(e1);
27.         session.persist(e2);
28.         session.persist(e3);
29.
30.         t.commit();
31.         session.close();
32.         System.out.println("success");
33.      }
```

```
34.     }
```

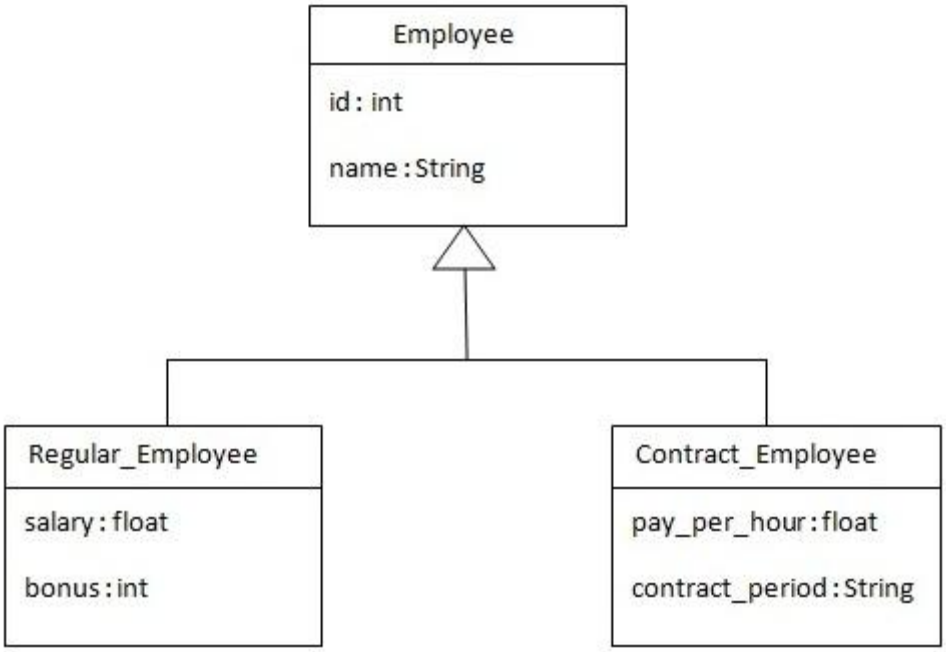
Table Per Subclass using Annotation

1. **Table Per Subclass class**
2. **Example of Table Per Subclass**

As we have specified earlier, in case of table per subclass strategy, tables are created as per persistent classes but they are reated using primary and foreign key. So there will not be duplicate columns in the relation.

We need to specify **@Inheritance(strategy=InheritanceType.JOINED)** in the parent class and **@PrimaryKeyJoinColumn** annotation in the subclasses.

Let's see the hierarchy of classes that we are going to map.



The table structure for each table will be as follows:

Table structure for Employee class

Column Name	Data Type	Nullable	Default	Primary Key
ID	NUMBER(10,0)	No	-	1
NAME	VARCHAR2(255)	Yes	-	-
				1 - 2

Table structure for Regular_Employee class

Column Name	Data Type	Nullable	Default	Primary Key
EID	NUMBER(10,0)	No	-	1
SALARY	FLOAT	Yes	-	-
BONUS	NUMBER(10,0)	Yes	-	-
				1 - 3

Table structure for Contract_Employee class

Column Name	Data Type	Nullable	Default	Primary Key
EID	NUMBER(10,0)	No	-	1
PAY_PER_HOUR	FLOAT	Yes	-	-
CONTRACT_DURATION	VARCHAR2(255)	Yes	-	-
				1 - 3

Example of Table per subclass class using Annotation

In this example we are creating the three classes and provide mapping of these classes in the employee.hbm.xml file.

1) Create the Persistent classes

You need to create the persistent classes representing the inheritance. Let's create the three classes for the above hierarchy:

File: Employee.java

copy to clipboard

```
1.      package com.javatpoint.mypackage;
2.      import javax.persistence.*;
3.
4.      @Entity
5.      @Table(name = "employee103")
6.      @Inheritance(strategy=InheritanceType.JOINED)
7.
8.      public class Employee {
9.      @Id
10.     @GeneratedValue(strategy=GenerationType.AUTO)
11.
12.     @Column(name = "id")
13.     private int id;
14.
15.     @Column(name = "name")
16.     private String name;
17.
```

```
18.     //setters and getters
19.     }
```

File: Regular_Employee.java

copy to clipboard

```
1.      package com.javatpoint.mypackage;
2.
3.      import javax.persistence.*;
4.
5.      @Entity
6.      @Table(name="regularemployee103")
7.      @PrimaryKeyJoinColumn(name="ID")
8.      public class Regular_Employee extends Employee{
9.
10.     @Column(name="salary")
11.     private float salary;
12.
13.     @Column(name="bonus")
14.     private int bonus;
15.
16.     //setters and getters
17.     }
```

File: Contract_Employee.java

copy to clipboard

```
1.      package com.javatpoint.mypackage;
2.
3.      import javax.persistence.*;
4.
5.      @Entity
6.      @Table(name="contractemployee103")
7.      @PrimaryKeyJoinColumn(name="ID")
8.      public class Contract_Employee extends Employee{
9.
10.        @Column(name="pay_per_hour")
11.        private float pay_per_hour;
12.
13.        @Column(name="contract_duration")
14.        private String contract_duration;
15.
16.        //setters and getters
17.     }
```

---

2) create configuration file

Open the hibernate.cgf.xml file, and add an entry of mapping resource like this:

copy to clipboard

```
1.      <mapping class="com.javatpoint.mypackage.Employee"/>
2.      <mapping class="com.javatpoint.mypackage.Contract_Employee"/>
3.      <mapping class="com.javatpoint.mypackage.Regular_Employee"/>
```

Now the configuration file will look like this:

File: hibernate.cfg.xml

copy to clipboard

```
<?xml version='1.0' encoding='UTF-8'?>
<!DOCTYPE hibernate-configuration PUBLIC
        "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
        "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">

<!-- Generated by MyEclipse Hibernate Tools.                   -->
<hibernate-configuration>

   <session-factory>
      <property name="hbm2ddl.auto">update</property>
      <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>

      <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
      <property name="connection.username">system</property>
      <property name="connection.password">oracle</property>
<property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>

      <mapping class="com.javatpoint.mypackage.Employee"/>
      <mapping class="com.javatpoint.mypackage.Contract_Employee"/>
      <mapping class="com.javatpoint.mypackage.Regular_Employee"/>
   </session-factory>

</hibernate-configuration>
```

The hbm2ddl.auto property is defined for creating automatic table in the database.

---

3) Create the class that stores the persistent object

In this class, we are simply storing the employee objects in the database.

File: StoreData.java

copy to clipboard

```
package com.javatpoint.mypackage;
import org.hibernate.*;
import org.hibernate.cfg.*;

public class StoreData {
public static void main(String[] args) {
   AnnotationConfiguration cfg=new AnnotationConfiguration();
   Session session=cfg.configure("hibernate.cfg.xml").buildSessionFactory().openSession();

   Transaction t=session.beginTransaction();

```

```
12.         Employee e1=new Employee();
13.         e1.setName("sonoo");
14.
15.         Regular_Employee e2=new Regular_Employee();
16.         e2.setName("Vivek Kumar");
17.         e2.setSalary(50000);
18.         e2.setBonus(5);
19.
20.         Contract_Employee e3=new Contract_Employee();
21.         e3.setName("Arjun Kumar");
22.         e3.setPay_per_hour(1000);
23.         e3.setContract_duration("15 hours");
24.
25.         session.persist(e1);
26.         session.persist(e2);
27.         session.persist(e3);
28.
29.         t.commit();
30.         session.close();
31.         System.out.println("success");
32.     }
33.     }
```

download this example developed using Myeclipse IDE

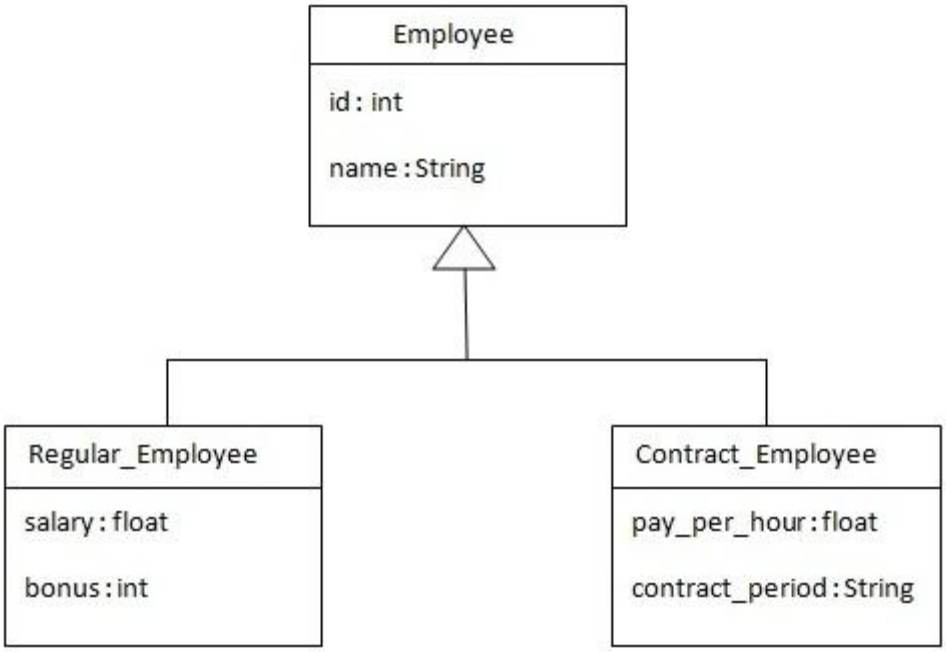
Table Per Subclass using Annotation

1. **Table Per Subclass class**
2. **Example of Table Per Subclass**

As we have specified earlier, in case of table per subclass strategy, tables are created as per persistent classes but they are reated using primary and foreign key. So there will not be duplicate columns in the relation.

We need to specify **@Inheritance(strategy=InheritanceType.JOINED)** in the parent class and **@PrimaryKeyJoinColumn** annotation in the subclasses.

Let's see the hierarchy of classes that we are going to map.



The table structure for each table will be as follows:

Table structure for Employee class

Column Name	Data Type	Nullable	Default	Primary Key
ID	NUMBER(10,0)	No	-	1
NAME	VARCHAR2(255)	Yes	-	-
				1 - 2

Table structure for Regular_Employee class

Column Name	Data Type	Nullable	Default	Primary Key
EID	NUMBER(10,0)	No	-	1
SALARY	FLOAT	Yes	-	-
BONUS	NUMBER(10,0)	Yes	-	-
				1 - 3

Table structure for Contract_Employee class

Column Name	Data Type	Nullable	Default	Primary Key
EID	NUMBER(10,0)	No	-	1
PAY_PER_HOUR	FLOAT	Yes	-	-
CONTRACT_DURATION	VARCHAR2(255)	Yes	-	-
				1 - 3

Example of Table per subclass class using Annotation

In this example we are creating the three classes and provide mapping of these classes in the employee.hbm.xml file.

1) Create the Persistent classes

You need to create the persistent classes representing the inheritance. Let's create the three classes for the above hierarchy:

File: Employee.java

copy to clipboard

```
1.      package com.javatpoint.mypackage;
2.      import javax.persistence.*;
3.
4.      @Entity
5.      @Table(name = "employee103")
6.      @Inheritance(strategy=InheritanceType.JOINED)
7.
8.      public class Employee {
9.      @Id
10.     @GeneratedValue(strategy=GenerationType.AUTO)
11.
12.     @Column(name = "id")
13.     private int id;
14.
15.     @Column(name = "name")
16.     private String name;
17.
18.     //setters and getters
19.     }
```

File: Regular_Employee.java

copy to clipboard

```
1.      package com.javatpoint.mypackage;
2.
3.      import javax.persistence.*;
4.
5.      @Entity
6.      @Table(name="regularemployee103")
7.      @PrimaryKeyJoinColumn(name="ID")
8.      public class Regular_Employee extends Employee{
9.
10.     @Column(name="salary")
11.     private float salary;
12.
13.     @Column(name="bonus")
14.     private int bonus;
15.
```

```
16.     //setters and getters
17.     }
```

File: Contract_Employee.java

copy to clipboard

```
1.      package com.javatpoint.mypackage;
2.
3.      import javax.persistence.*;
4.
5.      @Entity
6.      @Table(name="contractemployee103")
7.      @PrimaryKeyJoinColumn(name="ID")
8.      public class Contract_Employee extends Employee{
9.
10.        @Column(name="pay_per_hour")
11.        private float pay_per_hour;
12.
13.        @Column(name="contract_duration")
14.        private String contract_duration;
15.
16.        //setters and getters
17.     }
```

---

2) create configuration file

Open the hibernate.cgf.xml file, and add an entry of mapping resource like this:

copy to clipboard

```
1.      <mapping class="com.javatpoint.mypackage.Employee"/>
2.      <mapping class="com.javatpoint.mypackage.Contract_Employee"/>
3.      <mapping class="com.javatpoint.mypackage.Regular_Employee"/>
```

Now the configuration file will look like this:

File: hibernate.cfg.xml

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-configuration PUBLIC
3.              "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.              "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6.      <!-- Generated by MyEclipse Hibernate Tools.                   -->
7.      <hibernate-configuration>
8.
9.          <session-factory>
10.             <property name="hbm2ddl.auto">update</property>
11.             <property name="dialect">org.hibernate.dialect.Oracle9Dialect</propert
    y>
12.             <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe
    </property>
```

```
            <property name="connection.username">system</property>
            <property name="connection.password">oracle</property>
     <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>

            <mapping class="com.javatpoint.mypackage.Employee"/>
            <mapping class="com.javatpoint.mypackage.Contract_Employee"/>
            <mapping class="com.javatpoint.mypackage.Regular_Employee"/>
        </session-factory>

    </hibernate-configuration>
```

The hbm2ddl.auto property is defined for creating automatic table in the database.

---

3) Create the class that stores the persistent object

In this class, we are simply storing the employee objects in the database.

File: StoreData.java

copy to clipboard

```
package com.javatpoint.mypackage;
import org.hibernate.*;
import org.hibernate.cfg.*;

public class StoreData {
public static void main(String[] args) {
    AnnotationConfiguration cfg=new AnnotationConfiguration();
    Session session=cfg.configure("hibernate.cfg.xml").buildSessionFactory().openSession();

    Transaction t=session.beginTransaction();

    Employee e1=new Employee();
    e1.setName("sonoo");

    Regular_Employee e2=new Regular_Employee();
    e2.setName("Vivek Kumar");
    e2.setSalary(50000);
    e2.setBonus(5);

    Contract_Employee e3=new Contract_Employee();
    e3.setName("Arjun Kumar");
    e3.setPay_per_hour(1000);
    e3.setContract_duration("15 hours");

    session.persist(e1);
    session.persist(e2);
    session.persist(e3);

    t.commit();
    session.close();
```

```
31.         System.out.println("success");
32.     }
33.     }
```

Collection Mapping in Hibernate

1. **Collection Mapping in Hibernate**
2. **Mapping collection in mapping file**
3. **Understanding key element**
4. **Indexed collections**
5. **Collection Elements**

We can map collection elements of Persistent class in Hibernate. You need to declare the type of collection in Persistent class from one of the following types:

- java.util.List
- java.util.Set
- java.util.SortedSet
- java.util.Map
- java.util.SortedMap
- java.util.Collection
- or write the implementation of org.hibernate.usertype.UserCollectionType

The persistent class should be defined like this for collection element.

copy to clipboard

```
1.    package com.javatpoint;
2.
3.    import java.util.List;
4.
5.    public class Question {
6.    private int id;
7.    private String qname;
8.    private List<String> answers;//List can be of any type
9.
10.   //getters and setters
11.
12.   }
```

Mapping collection in mapping file

There are many subelements of **<class>** elements to map the collection. They are **<list>**, **<bag>**, **<set>** and **<map>**. Let's see how we implement the list for the above class:

copy to clipboard

```
1.    <class name="com.javatpoint.Question" table="q100">
2.            <id name="id">
3.            <generator class="increment"></generator>
4.            </id>
```

```
        <property name="qname"></property>

        <list name="answers" table="ans100">
        <key column="qid"></key>
        <index column="type"></index>
        <element column="answer" type="string"></element>
        </list>

        </class>
```

There are three subelements used in the list:

- **<key>** element is used to define the foreign key in this table based on the Question class identifier.
- **<index>** element is used to identify the type. List and Map are indexed collection.
- **<element>** is used to define the element of the collection.

This is the mapping of collection if collection stores string objects. But if collection stores entity reference (another class objects), we need to define **<one-to-many>** or **<many-to-many>** element. Now the Persistent class will look like:

copy to clipboard

```
package com.javatpoint;

import java.util.List;

public class Question {
private int id;
private String qname;
private List<Answer> answers;//Here, List stores the objects of Answer class

//getters and setters

}
```

copy to clipboard

```
package com.javatpoint;

import java.util.List;

public class Answer {
private int id;
private String answer;
private String posterName;

//getters and setters

}
```

Now the mapping file will be:

copy to clipboard

```
<class name="com.javatpoint.Question" table="q100">
        <id name="id">
        <generator class="increment"></generator>
```

```
4.          </id>
5.          <property name="qname"></property>
6.
7.          <list name="answers" >
8.          <key column="qid"></key>
9.          <index column="type"></index>
10.         <one-to-many class="com.javatpoint.Answer" />
11.         </list>
12.
13.         </class>
```

Here, List is mapped by one-to-many relation. In this scenario, there can be many answers for one question.

Understanding key element

The key element is used to define the foreign key in the joined table based on the original identity. The foreign key element is nullable by default. So for non-nullable foreign key, we need to specify not-null attribute such as:

copy to clipboard

```
1.      <key column="qid" not-null="true" ></key>
```

The attributes of the key element are column, on-delete, property-ref, not-null, update and unique.

copy to clipboard

```
1.      <key
2.      column="columnname"
3.      on-delete="noaction|cascade"
4.      not-null="true|false"
5.      property-ref="propertyName"
6.      update="true|false"
7.      unique="true|false"
8.      />
```

Indexed collections

The collection elements can be categorized in two forms:

- **indexed** ,and
- **non-indexed**

The List and Map collection are indexed whereas set and bag collections are non-indexed. Here, indexed collection means List and Map requires an additional element **<index>**.

Collection Elements

The collection elements can have value or entity reference (another class object). We can use one of the 4 elements

- **element**
- **component-element**
- **one-to-many**, or
- **many-to-many**

The element and component-element are used for normal value such as string, int etc. whereas one-to-many and many-to-many are used to map entity reference.

Upcoming topics in Collection Mapping

Mapping List

In this example, we are going to map the List element.

Mapping Bag

In this example, we are going to use the bag collection of Hibernate framework.

Mapping Set

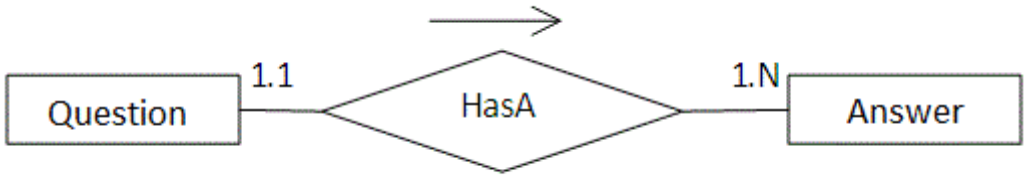
Here, we will map the set element of collection.

Mapping List in Collection Mapping (using xml file)

1. **Mapping List in Collection Mapping**
2. **Example of mapping list in collection mapping**

If our persistent class has List object, we can map the List easily either by <list> element of class in mapping file or by annotation.

Here, we are using the scenario of Forum where one question has multiple answers.



Let's see how we can implement the list in the mapping file:

copy to clipboard

```
<class name="com.javatpoint.Question" table="q100">
    ...
      <list name="answers" table="ans100">
      <key column="qid"></key>
      <index column="type"></index>
      <element column="answer" type="string"></element>
      </list>

    ...
</class>
```

List and Map are index based collection, so an extra column will be created in the table for indexing.

Example of mapping list in collection mapping

In this example, we are going to see full example of collection mapping by list. This is the example of List that stores string value not entity reference that is why are going to use **element** instead of **one-to-many** within the list element.

1) create the Persistent class

This persistent class defines properties of the class including List.

copy to clipboard

```
package com.javatpoint;

import java.util.List;

public class Question {
private int id;
private String qname;
private List<string> answers;

//getters and setters

}

</string>
```

---

2) create the Mapping file for the persistent class

Here, we have created the question.hbm.xml file for defining the list.

copy to clipboard

```
<?xml version='1.0' encoding='UTF-8'?>
<!DOCTYPE hibernate-mapping PUBLIC
        "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
        "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
```

```
5.
6.      <hibernate-mapping>
7.       <class name="com.javatpoint.Question" table="q100">
8.         <id name="id">
9.           <generator class="increment"></generator>
10.        </id>
11.        <property name="qname"></property>
12.
13.        <list name="answers" table="ans100">
14.          <key column="qid"></key>
15.          <index column="type"></index>
16.          <element column="answer" type="string"></element>
17.        </list>
18.
19.      </class>
20.
21.     </hibernate-mapping>
```

---

3) create the configuration file

This file contains information about the database and mapping file.

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-configuration PUBLIC
3.              "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.              "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6.      <!-- Generated by MyEclipse Hibernate Tools.                   -->
7.      <hibernate-configuration>
8.
9.          <session-factory>
10.            <property name="hbm2ddl.auto">update</property>
11.            <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>

12.            <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</p
    roperty>
13.            <property name="connection.username">system</property>
14.            <property name="connection.password">oracle</property>
15.            <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver
    </property>
16.         <mapping resource="question.hbm.xml"/>
17.         </session-factory>
18.
19.     </hibernate-configuration>
```

---

4) Create the class to store the data

In this class we are storing the data of the question class.

copy to clipboard

```
1.      package com.javatpoint;
2.
3.      import java.util.ArrayList;
4.
5.      import org.hibernate.*;
6.      import org.hibernate.cfg.*;
7.
8.      public class StoreData {
9.       public static void main(String[] args) {
10.        Session session=new Configuration().configure("hibernate.cfg.xml")
11.                        .buildSessionFactory().openSession();
12.        Transaction t=session.beginTransaction();
13.
14.        ArrayList<string> list1=new ArrayList<string>();
15.        list1.add("java is a programming language");
16.        list1.add("java is a platform");
17.
18.        ArrayList<string> list2=new ArrayList<string>();
19.        list2.add("Servlet is an Interface");
20.        list2.add("Servlet is an API");
21.
22.        Question question1=new Question();
23.        question1.setQname("What is Java?");
24.        question1.setAnswers(list1);
25.
26.        Question question2=new Question();
27.        question2.setQname("What is Servlet?");
28.        question2.setAnswers(list2);
29.
30.        session.persist(question1);
31.        session.persist(question2);
32.
33.        t.commit();
34.        session.close();
35.        System.out.println("success");
36.     }
37.     }
38.
39.     </string></string></string></string>
```

How to fetch the data of List

Here, we have used HQL to fetch all the records of Question class including answers. In such case, it fetches the data from two tables that are functional dependent.

copy to clipboard

```
1.      package com.javatpoint;
2.
3.      import java.util.*;
4.
5.      import org.hibernate.*;
6.      import org.hibernate.cfg.*;
7.
8.      public class FetchData {
```

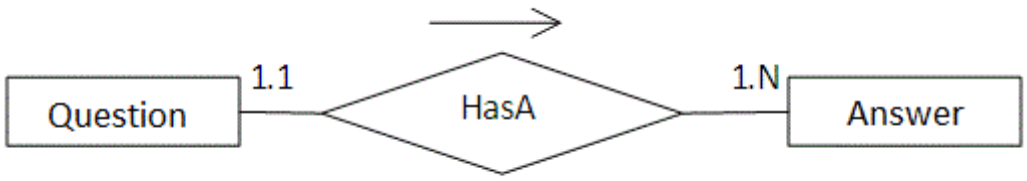
```
9.      public static void main(String[] args) {
10.
11.         Session session=new Configuration().configure("hibernate.cfg.xml")
12.                         .buildSessionFactory().openSession();
13.
14.         Query query=session.createQuery("from Question");
15.         List<question> list=query.list();
16.
17.         Iterator<question> itr=list.iterator();
18.         while(itr.hasNext()){
19.            Question q=itr.next();
20.            System.out.println("Question Name: "+q.getQname());
21.
22.            //printing answers
23.            List<string> list2=q.getAnswers();
24.            Iterator<string> itr2=list2.iterator();
25.            while(itr2.hasNext()){
26.               System.out.println(itr2.next());
27.            }
28.
29.         }
30.         session.close();
31.         System.out.println("success");
32.
33.     }
34.     }
35.
36.     </string></string></question></question>
```

One to Many mapping in Hibernate by List Example (using xml file)

1. **List One to many**
2. **Example of mapping list in collection mapping by one to many association**

If the persistent class has list object that contains the entity reference, we need to use one-to-many association to map the list element.

Here, we are using the scenario of Forum where one question has multiple answers.



In such case, there can be many answers for a question and each answer may have its own informations that is why we have used **list** in the persistent class (containing the reference of Answer class) to represent a collection of answers.

Let's see the persistent class that has list objects (containing Answer class objects).

copy to clipboard

```
1.      package com.javatpoint;
2.
3.      import java.util.List;
4.
5.      public class Question {
6.      private int id;
7.      private String qname;
8.      private List<Answer> answers;
9.      //getters and setters
10.
11.     }
```

The Answer class has its own informations such as id, answername, postedBy etc.

copy to clipboard

```
1.      package com.javatpoint;
2.
3.      public class Answer {
4.      private int id;
5.      private String answername;
6.      private String postedBy;
7.      //getters and setters
8.
9.      }
10.     }
```

The Question class has list object that have entity reference (i.e. Answer class object). In such case, we need to use **one-to-many** of list to map this object. Let's see how we can map it.

copy to clipboard

```
1.      <list name="answers" cascade="all">
2.              <key column="qid"></key>
3.              <index column="type"></index>
4.              <one-to-many class="com.javatpoint.Answer"/>
5.      </list>
```

Full example of One to Many mapping in Hibernate by List

In this example, we are going to see full example of mapping list that contains entity reference.

1) create the Persistent class

This persistent class defines properties of the class including List.

Question.java

copy to clipboard

```
1.      package com.javatpoint;
2.
3.      import java.util.List;
4.
5.      public class Question {
6.      private int id;
7.      private String qname;
8.      private List<Answer> answers;
9.
10.     //getters and setters
11.
12.     }
```

Answer.java

copy to clipboard

```
1.      package com.javatpoint;
2.
3.      public class Answer {
4.      private int id;
5.      private String answername;
6.      private String postedBy;
7.      //getters and setters
8.
9.      }
10.     }
```

---

2) create the Mapping file for the persistent class

Here, we have created the question.hbm.xml file for defining the list.

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-mapping PUBLIC
3.              "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.              "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.              <hibernate-mapping>
7.              <class name="com.javatpoint.Question" table="q501">
8.              <id name="id">
9.              <generator class="increment"></generator>
10.             </id>
11.             <property name="qname"></property>
12.
13.             <list name="answers" cascade="all">
14.             <key column="qid"></key>
15.             <index column="type"></index>
16.             <one-to-many class="com.javatpoint.Answer"/>
17.             </list>
18.
```

```
19.             </class>
20.
21.             <class name="com.javatpoint.Answer" table="ans501">
22.             <id name="id">
23.             <generator class="increment"></generator>
24.             </id>
25.             <property name="answername"></property>
26.             <property name="postedBy"></property>
27.             </class>
28.
29.             </hibernate-mapping>
```

---

3) create the configuration file

This file contains information about the database and mapping file.

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-configuration PUBLIC
3.              "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.              "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6.      <!-- Generated by MyEclipse Hibernate Tools.                   -->
7.      <hibernate-configuration>
8.
9.          <session-factory>
10.             <property name="hbm2ddl.auto">update</property>
11.             <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>

12.             <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</p
    roperty>
13.             <property name="connection.username">system</property>
14.             <property name="connection.password">oracle</property>
15.             <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver
    </property>
16.         <mapping resource="question.hbm.xml"/>
17.         </session-factory>
18.
19.     </hibernate-configuration>
```

---

4) Create the class to store the data

In this class we are storing the data of the question class.

copy to clipboard

```
1.      package com.javatpoint;
2.      import java.util.ArrayList;
3.      import org.hibernate.*;
4.      import org.hibernate.cfg.*;
5.
6.      public class StoreData {
```

```
7.      public static void main(String[] args) {
8.         Session session=new Configuration().configure("hibernate.cfg.xml")
9.                               .buildSessionFactory().openSession();
10.        Transaction t=session.beginTransaction();
11.
12.        Answer ans1=new Answer();
13.        ans1.setAnswername("java is a programming language");
14.        ans1.setPostedBy("Ravi Malik");
15.
16.        Answer ans2=new Answer();
17.        ans2.setAnswername("java is a platform");
18.        ans2.setPostedBy("Sudhir Kumar");
19.
20.        Answer ans3=new Answer();
21.        ans3.setAnswername("Servlet is an Interface");
22.        ans3.setPostedBy("Jai Kumar");
23.
24.        Answer ans4=new Answer();
25.        ans4.setAnswername("Servlet is an API");
26.        ans4.setPostedBy("Arun");
27.
28.        ArrayList<Answer> list1=new ArrayList<Answer>();
29.        list1.add(ans1);
30.        list1.add(ans2);
31.
32.        ArrayList<Answer> list2=new ArrayList<Answer>();
33.        list2.add(ans3);
34.        list2.add(ans4);
35.
36.        Question question1=new Question();
37.        question1.setQname("What is Java?");
38.        question1.setAnswers(list1);
39.
40.        Question question2=new Question();
41.        question2.setQname("What is Servlet?");
42.        question2.setAnswers(list2);
43.
44.        session.persist(question1);
45.        session.persist(question2);
46.
47.        t.commit();
48.        session.close();
49.        System.out.println("success");
50.     }
51.     }
```

OUTPUT

ID	QNAME
1	What is Java?
2	What is Servlet?

ID	ANSWERNAME	POSTEDBY	QID	TYPE
1	java is a programming language	Ravi Malik	1	0
2	java is a platform	Sudhir Kumar	1	1
3	Servlet is an Interface	Jai Kumar	2	0
4	Servlet is an API	Arun	2	1

How to fetch the data of List

Here, we have used HQL to fetch all the records of Question class including answers. In such case, it fetches the data from two tables that are functional dependent. Here, we are direct printing the object of answer class, but we have overridden the **toString()** method in the Answer class returning answername and poster name. So it prints the answer name and postername rather than reference id.

FetchData.java

copy to clipboard

```
1.      package com.javatpoint;
2.      import java.util.*;
3.      import org.hibernate.*;
4.      import org.hibernate.cfg.*;
5.
6.      public class FetchData {
7.      public static void main(String[] args) {
8.
9.          Session session=new Configuration().configure("hibernate.cfg.xml")
10.                         .buildSessionFactory().openSession();
11.
12.         Query query=session.createQuery("from Question");
13.         List<Question> list=query.list();
14.
15.         Iterator<Question> itr=list.iterator();
16.         while(itr.hasNext()){
17.            Question q=itr.next();
18.            System.out.println("Question Name: "+q.getQname());
19.
20.            //printing answers
21.            List<Answer> list2=q.getAnswers();
22.            Iterator<Answer> itr2=list2.iterator();
23.            while(itr2.hasNext()){
24.               System.out.println(itr2.next());
25.            }
26.
27.         }
28.         session.close();
29.         System.out.println("success");
30.     }
31.     }
```

OUTPUT

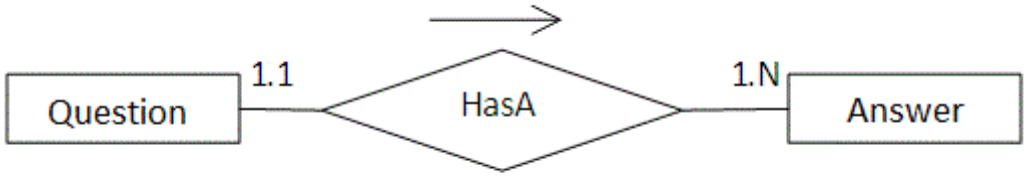
```
Question Name: What is Java?
java is a programming language by: Ravi Malik
java is a platform by: Sudhir Kumar
Question Name: What is Servlet?
Servlet is an Interface by: Jai Kumar
Servlet is an API by: Arun
success
```

Mapping Bag in Collection Mapping (using xml file)

1. **Mapping Bag in Collection Mapping**
2. **Example of mapping Bag in Collection Mapping**

If our persistent class has List object, we can map the List by list or bag element in the mapping file. The bag is just like List but it doesn't require index element.

Here, we are using the scenario of Forum where one question has multiple answers.



Let's see how we can implement the bag in the mapping file:

copy to clipboard

```
<class name="com.javatpoint.Question" table="q100">
    ...
      <bag name="answers" table="ans100">
      <key column="qid"></key>
      <element column="answer" type="string"></element>
      </bag>

    ...
</class>
```

Example of mapping bag in collection mapping

In this example, we are going to see full example of collection mapping by bag. This is the example of bag if it stores value not entity reference that is why are going to

use **element** instead of **one-to-many**. If you have seen the example of mapping list, it is same in all cases instead mapping file where we are using bag instead of list.

1) create the Persistent class

This persistent class defines properties of the class including List.

copy to clipboard

```
1.  package com.javatpoint;
2.
3.  import java.util.List;
4.
5.  public class Question {
6.  private int id;
7.  private String qname;
8.  private List<string> answers;
9.
10. //getters and setters
11.
12. }
13.
14. </string>
```

---

2) create the Mapping file for the persistent class

Here, we have created the question.hbm.xml file for defining the list.

copy to clipboard

```
1.  <?xml version='1.0' encoding='UTF-8'?>
2.  <!DOCTYPE hibernate-mapping PUBLIC
3.          "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.          "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.  <hibernate-mapping>
7.   <class name="com.javatpoint.Question" table="q101">
8.    <id name="id">
9.     <generator class="increment"></generator>
10.   </id>
11.   <property name="qname"></property>
12.
13.   <bag name="answers" table="ans101">
14.    <key column="qid"></key>
15.    <element column="answer" type="string"></element>
16.   </bag>
17.
18.  </class>
19.
20. </hibernate-mapping>
```

---

3) create the configuration file

This file contains information about the database and mapping file.

copy to clipboard

```
<?xml version='1.0' encoding='UTF-8'?>
<!DOCTYPE hibernate-configuration PUBLIC
        "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
        "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">

<!-- Generated by MyEclipse Hibernate Tools.                   -->
<hibernate-configuration>

    <session-factory>
        <property name="hbm2ddl.auto">update</property>
        <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>

        <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
        <property name="connection.username">system</property>
        <property name="connection.password">oracle</property>
        <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
    <mapping resource="question.hbm.xml"/>
    </session-factory>

</hibernate-configuration>
```

---

4) Create the class to store the data

In this class we are storing the data of the question class.

copy to clipboard

```
package com.javatpoint;

import java.util.ArrayList;

import org.hibernate.*;
import org.hibernate.cfg.*;

public class StoreData {
 public static void main(String[] args) {
    Session session=new Configuration().configure("hibernate.cfg.xml")
                    .buildSessionFactory().openSession();
    Transaction t=session.beginTransaction();

    ArrayList<String> list1=new ArrayList<String>();
    list1.add("java is a programming language");
    list1.add("java is a platform");

    ArrayList<String> list2=new ArrayList<String>();
    list2.add("Servlet is an Interface");
    list2.add("Servlet is an API");

    Question question1=new Question();
    question1.setQname("What is Java?");
```

```
24.       question1.setAnswers(list1);
25.
26.       Question question2=new Question();
27.       question2.setQname("What is Servlet?");
28.       question2.setAnswers(list2);
29.
30.       session.persist(question1);
31.       session.persist(question2);
32.
33.       t.commit();
34.       session.close();
35.       System.out.println("success");
36.    }
37.   }
```

How to fetch the data

Here, we have used HQL to fetch all the records of Question class including answers. In such case, it fetches the data from two tables that are functional dependent.

copy to clipboard

```
1.    package com.javatpoint;
2.
3.    import java.util.*;
4.
5.    import org.hibernate.*;
6.    import org.hibernate.cfg.*;
7.
8.    public class FetchData {
9.    public static void main(String[] args) {
10.
11.       Session session=new Configuration().configure("hibernate.cfg.xml")
12.                       .buildSessionFactory().openSession();
13.
14.       Query query=session.createQuery("from Question");
15.       List<Question> list=query.list();
16.
17.       Iterator<Question> itr=list.iterator();
18.       while(itr.hasNext()){
19.          Question q=itr.next();
20.          System.out.println("Question Name: "+q.getQname());
21.
22.          //printing answers
23.          List<String> list2=q.getAnswers();
24.          Iterator<String> itr2=list2.iterator();
25.          while(itr2.hasNext()){
26.             System.out.println(itr2.next());
27.          }
28.
29.       }
30.       session.close();
31.       System.out.println("success");
32.
33.   }
```

```
34.    }
```

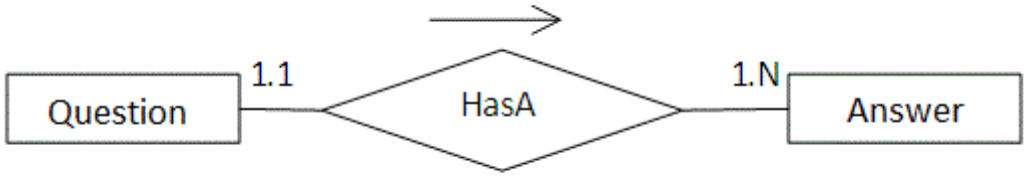
One to Many mapping in Hibernate by List Example (using xml file)

1. **Bag One to many**
2. **Example of mapping bag in collection mapping by one to many association**

If the persistent class has list object that contains the entity reference, we need to use **one-to-many** association to map the list element. We can map this list object by either **list** or **bag**.

Notice that bag is not index-based whereas list is index-based.

Here, we are using the scenario of Forum where one question has multiple answers.



Let's see the persistent class that has list objects. In this case, there can be many answers for a question and each answer may have its own informations that is why we have used list element (containing Answer objects) to represent a collection of answers.

copy to clipboard

```
1.      package com.javatpoint;
2.
3.      import java.util.List;
4.
5.      public class Question {
6.      private int id;
7.      private String qname;
8.      private List<Answer> answers;
9.      //getters and setters
10.
11.     }
```

The Answer class has its own informations such as id, answername, postedBy etc.

copy to clipboard

```
1.      package com.javatpoint;
2.
3.      public class Answer {
4.      private int id;
```

```
5.      private String answername;
6.      private String postedBy;
7.      //getters and setters
8.
9.      }
10.     }
```

The Question class has list object that have entity reference (i.e. Answer class object). In such case, we need to use **one-to-many** of bag to map this object. Let's see how we can map it.

copy to clipboard

```
1.      <bag name="answers" cascade="all">
2.              <key column="qid"></key>
3.              <one-to-many class="com.javatpoint.Answer"/>
4.      </list>
```

Example of mapping bag in collection mapping by one to many association

In this example, we are going to see full example of mapping list that contains entity reference.

1) create the Persistent class

This persistent class defines properties of the class including List.

Question.java

copy to clipboard

```
1.      package com.javatpoint;
2.
3.      import java.util.List;
4.
5.      public class Question {
6.      private int id;
7.      private String qname;
8.      private List<answer> answers;
9.
10.     //getters and setters
11.
12.     }
13.
14.     </answer>
```

Answer.java

copy to clipboard

```
1.      package com.javatpoint;
2.
3.      public class Answer {
```

```
4.      private int id;
5.      private String answername;
6.      private String postedBy;
7.      //getters and setters
8.
9.      }
10.     }
```

2) create the Mapping file for the persistent class

Here, we have created the question.hbm.xml file for defining the list.

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-mapping PUBLIC
3.              "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.              "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.              <hibernate-mapping>
7.              <class name="com.javatpoint.Question" table="q501">
8.              <id name="id">
9.              <generator class="increment"></generator>
10.             </id>
11.             <property name="qname"></property>
12.
13.             <bag name="answers" cascade="all">
14.             <index column="type"></index>
15.             <one-to-many class="com.javatpoint.Answer"/>
16.             </bag>
17.
18.             </class>
19.
20.             <class name="com.javatpoint.Answer" table="ans501">
21.             <id name="id">
22.             <generator class="increment"></generator>
23.             </id>
24.             <property name="answername"></property>
25.             <property name="postedBy"></property>
26.             </class>
27.
28.             </hibernate-mapping>
```

3) create the configuration file

This file contains information about the database and mapping file.

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-configuration PUBLIC
3.              "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.              "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
```

```
5.
6.      <!-- Generated by MyEclipse Hibernate Tools.                 -->
7.      <hibernate-configuration>
8.
9.          <session-factory>
10.             <property name="hbm2ddl.auto">update</property>
11.             <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>

12.             <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</p
    roperty>
13.             <property name="connection.username">system</property>
14.             <property name="connection.password">oracle</property>
15.             <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver
    </property>
16.         <mapping resource="question.hbm.xml"/>
17.         </session-factory>
18.
19.     </hibernate-configuration>
```

---

4) Create the class to store the data

In this class we are storing the data of the question class.

copy to clipboard

```
1.      package com.javatpoint;
2.
3.      import java.util.ArrayList;
4.
5.      import org.hibernate.*;
6.      import org.hibernate.cfg.*;
7.
8.      public class StoreData {
9.      public static void main(String[] args) {
10.         Session session=new Configuration().configure("hibernate.cfg.xml")
11.                         .buildSessionFactory().openSession();
12.         Transaction t=session.beginTransaction();
13.
14.         Answer ans1=new Answer();
15.         ans1.setAnswername("java is a programming language");
16.         ans1.setPostedBy("Ravi Malik");
17.
18.         Answer ans2=new Answer();
19.         ans2.setAnswername("java is a platform");
20.         ans2.setPostedBy("Sudhir Kumar");
21.
22.         Answer ans3=new Answer();
23.         ans3.setAnswername("Servlet is an Interface");
24.         ans3.setPostedBy("Jai Kumar");
25.
26.         Answer ans4=new Answer();
27.         ans4.setAnswername("Servlet is an API");
28.         ans4.setPostedBy("Arun");
29.
30.         ArrayList<answer> list1=new ArrayList<answer>();
```

```
31.         list1.add(ans1);
32.         list1.add(ans2);
33.
34.         ArrayList<answer> list2=new ArrayList<answer>();
35.         list2.add(ans3);
36.         list2.add(ans4);
37.
38.         Question question1=new Question();
39.         question1.setQname("What is Java?");
40.         question1.setAnswers(list1);
41.
42.         Question question2=new Question();
43.         question2.setQname("What is Servlet?");
44.         question2.setAnswers(list2);
45.
46.         session.persist(question1);
47.         session.persist(question2);
48.
49.         t.commit();
50.         session.close();
51.         System.out.println("success");
52.
53.     }
54.     }
55.
56.     </answer></answer></answer></answer>
```

---

download this hibernate example

How to fetch the data of List

Here, we have used HQL to fetch all the records of Question class including answers. In such case, it fetches the data from two tables that are functional dependent. Here, we are direct printing the object of answer class, but we have overridden the **toString()** method in the Answer class returning answername and poster name. So it prints the answer name and postername rather than reference id.

FetchData.java

copy to clipboard

```
1.      package com.javatpoint;
2.
3.      import java.util.*;
4.
5.      import org.hibernate.*;
6.      import org.hibernate.cfg.*;
7.
8.      public class FetchData {
9.      public static void main(String[] args) {
10.
11.         Session session=new Configuration().configure("hibernate.cfg.xml")
12.                         .buildSessionFactory().openSession();
```

```
13.
14.        Query query=session.createQuery("from Question");
15.        List<question> list=query.list();
16.
17.        Iterator<question> itr=list.iterator();
18.        while(itr.hasNext()){
19.           Question q=itr.next();
20.           System.out.println("Question Name: "+q.getQname());
21.
22.           //printing answers
23.           List<answer> list2=q.getAnswers();
24.           Iterator<answer> itr2=list2.iterator();
25.           while(itr2.hasNext()){
26.              System.out.println(itr2.next());
27.           }
28.
29.        }
30.        session.close();
31.        System.out.println("success");
32.
33.     }
34.     }
35.
36.
37.     </answer></answer></question></question>
```

OUTPUT

```
Question Name: What is Java?
java is a programming language by: Ravi Malik
java is a platform by: Sudhir Kumar
Question Name: What is Servlet?
Servlet is an Interface by: Jai Kumar
Servlet is an API by: Arun
success
```

Mapping Set in Collection Mapping

1. **Mapping Set in Collection Mapping**
2. **Example of mapping Set in Collection Mapping**

If our persistent class has Set object, we can map the Set by set element in the mapping file. The set element doesn't require index element. The one difference between List and Set is that, it stores only unique values.

Let's see how we can implement the set in the mapping file:

copy to clipboard

```
1.    <class name="com.javatpoint.Question" table="q102">
2.          ...
```

```
3.            <set name="answers" table="ans102">
4.            <key column="qid"></key>
5.            <element column="answer" type="string"></element>
6.            </set>
7.
8.          ...
9.     </class>
```

Example of mapping set in collection mapping

In this example, we are going to see full example of collection mapping by set. This is the example of set that stores value not entity reference that is why are going to use element instead of one-to-many.

1) create the Persistent class

This persistent class defines properties of the class including Set.

copy to clipboard

```
1.    package com.javatpoint;
2.
3.    import java.util.Set;
4.
5.    public class Question {
6.    private int id;
7.    private String qname;
8.    private Set<String> answers;
9.
10.   //getters and setters
11.
12.   }
```

---

2) create the Mapping file for the persistent class

Here, we have created the question.hbm.xml file for defining the list.

copy to clipboard

```
1.    <?xml version='1.0' encoding='UTF-8'?>
2.    <!DOCTYPE hibernate-mapping PUBLIC
3.            "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.            "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.    <hibernate-mapping>
7.     <class name="com.javatpoint.Question" table="q102">
8.      <id name="id">
9.       <generator class="increment"></generator>
10.     </id>
11.     <property name="qname"></property>
12.
13.     <set name="answers" table="ans102">
14.      <key column="qid"></key>
```

```
15.         <element column="answer" type="string"></element>
16.       </set>
17.
18.     </class>
19.
20.     </hibernate-mapping>
```

---

3) create the configuration file

This file contains information about the database and mapping file.

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-configuration PUBLIC
3.              "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.              "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6.      <!-- Generated by MyEclipse Hibernate Tools.                   -->
7.      <hibernate-configuration>
8.
9.          <session-factory>
10.             <property name="hbm2ddl.auto">update</property>
11.             <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>

12.             <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</p
    roperty>
13.             <property name="connection.username">system</property>
14.             <property name="connection.password">oracle</property>
15.             <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver
    </property>
16.         <mapping resource="question.hbm.xml"/>
17.         </session-factory>
18.
19.     </hibernate-configuration>
```

---

4) Create the class to store the data

In this class we are storing the data of the question class.

copy to clipboard

```
1.      package com.javatpoint;
2.
3.      import java.util.ArrayList;
4.
5.      import org.hibernate.*;
6.      import org.hibernate.cfg.*;
7.
8.      public class StoreData {
9.       public static void main(String[] args) {
10.         Session session=new Configuration().configure("hibernate.cfg.xml")
11.                         .buildSessionFactory().openSession();
12.         Transaction t=session.beginTransaction();
```

```
13.
14.
15.       HashSet<string> set1=new HashSet<string>();
16.       set1.add("java is a programming language");
17.       set1.add("java is a platform");
18.
19.       HashSet<string> set2=new HashSet<string>();
20.       set2.add("Servlet is an Interface");
21.       set2.add("Servlet is an API");
22.
23.       Question question1=new Question();
24.       question1.setQname("What is Java?");
25.       question1.setAnswers(set1);
26.
27.       Question question2=new Question();
28.       question2.setQname("What is Servlet?");
29.       question2.setAnswers(set2);
30.
31.       session.persist(question1);
32.       session.persist(question2);
33.
34.       t.commit();
35.       session.close();
36.       System.out.println("success");
37.    }
38.   }
39.
40.   </string></string></string></string>
```

download this hibernate example

How to fetch the data of Set

Here, we have used HQL to fetch all the records of Question class including answers. In such case, it fetches the data from two tables that are functional dependent.

copy to clipboard

```
1.    package com.javatpoint;
2.
3.    import java.util.*;
4.
5.    import org.hibernate.*;
6.    import org.hibernate.cfg.*;
7.
8.    public class FetchData {
9.    public static void main(String[] args) {
10.
11.       Session session=new Configuration().configure("hibernate.cfg.xml")
12.                       .buildSessionFactory().openSession();
13.
14.       Query query=session.createQuery("from Question");
15.       List<question> list=query.list();
16.
17.       Iterator<question> itr=list.iterator();
18.       while(itr.hasNext()){
```

```
19.          Question q=itr.next();
20.          System.out.println("Question Name: "+q.getQname());
21.
22.          //printing answers
23.          Set<string> set=q.getAnswers();
24.          Iterator<string> itr2=set.iterator();
25.          while(itr2.hasNext()){
26.             System.out.println(itr2.next());
27.          }
28.
29.       }
30.       session.close();
31.       System.out.println("success");
32.
33.    }
34.    }
35.
36.    </string></string></question></question>
```

Set One to many

1. **Set One to many**
2. **Example of mapping set in collection mapping by one to many association**

If the persistent class has set object that contains the entity reference, we need to use one-to-many association to map the set element. We can map this list object by either **set**.

It is non-index based and will not allow duplicate elements.

Let's see the persistent class that has set objects. In this case, there can be many answers for a question and each answer may have its own informations that is why we have used set element to represent a collection of answers.

copy to clipboard

```
1.    package com.javatpoint;
2.
3.    import java.util.List;
4.
5.    public class Question {
6.    private int id;
7.    private String qname;
8.    private Set<Answer> answers;
9.    //getters and setters
10.
11.   }
```

The Answer class has its own informations such as id, answername, postedBy etc.

copy to clipboard

```
1.    package com.javatpoint;
2.
3.    public class Answer {
4.    private int id;
5.    private String answername;
```

```
6.     private String postedBy;
7.     //getters and setters
8.
9.     }
10.    }
```

The Question class has set object that have entity reference (i.e. Answer class object). In such case, we need to use **one-to-many** of set to map this object. Let's see how we can map it.

copy to clipboard

```
1.     <set name="answers" cascade="all">
2.             <key column="qid"></key>
3.             <one-to-many class="com.javatpoint.Answer"/>
4.     </set>
```

Example of mapping set in collection mapping by one to many association

To understand this example, you may see the bag one-to-many relation example. We have changed only bag to set in the hbm file and ArrayList to HashSet in the Store class.

Mapping Map in collection mapping using xml file

1. Mapping Map in collection mapping
2. Example of Mapping Map in collection mapping

Hibernate allows you to map Map elements with the RDBMS. As we know, list and map are index-based collections. In case of map, index column works as the key and element column works as the value.

Example of Mapping Map in collection mapping using xml file

You need to create following pages for mapping map elements.

- Question.java
- question.hbm.xml
- hibernate.cfg.xml
- StoreTest.java
- FetchTest.java

Question.java

copy to clipboard

```
1.     package com.javatpoint;
2.
3.     import java.util.Map;
```

```
4.
5.      public class Question {
6.      private int id;
7.      private String name,username;
8.      private Map<String,String> answers;
9.
10.     public Question() {}
11.     public Question(String name, String username, Map<String, String> answers) {

12.        super();
13.        this.name = name;
14.        this.username = username;
15.        this.answers = answers;
16.     }
17.     public int getId() {
18.        return id;
19.     }
20.     public void setId(int id) {
21.        this.id = id;
22.     }
23.     public String getName() {
24.        return name;
25.     }
26.     public void setName(String name) {
27.        this.name = name;
28.     }
29.     public String getUsername() {
30.        return username;
31.     }
32.     public void setUsername(String username) {
33.        this.username = username;
34.     }
35.     public Map<String, String> getAnswers() {
36.        return answers;
37.     }
38.     public void setAnswers(Map<String, String> answers) {
39.        this.answers = answers;
40.     }
41.     }
```

question.hbm.xml

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-mapping PUBLIC
3.              "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.              "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.      <hibernate-mapping>
7.
8.      <class name="com.javatpoint.Question" table="question736">
9.      <id name="id">
10.     <generator class="native"></generator>
11.     </id>
12.     <property name="name"></property>
13.     <property name="username"></property>
14.
```

```
15.    <map name="answers" table="answer736" cascade="all">
16.    <key column="questionid"></key>
17.    <index column="answer" type="string"></index>
18.    <element column="username" type="string"></element>
19.    </map>
20.    </class>
21.
22.    </hibernate-mapping>
```

hibernate.cfg.xml

copy to clipboard

```
1.     <?xml version='1.0' encoding='UTF-8'?>
2.     <!DOCTYPE hibernate-configuration PUBLIC
3.             "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.             "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6.     <!-- Generated by MyEclipse Hibernate Tools.                   -->
7.     <hibernate-configuration>
8.
9.         <session-factory>
10.            <property name="hbm2ddl.auto">update</property>
11.            <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>

12.            <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</p
    roperty>
13.            <property name="connection.username">system</property>
14.            <property name="connection.password">oracle</property>
15.            <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver
    </property>
16.
17.        <mapping resource="question.hbm.xml"/>
18.        </session-factory>
19.
20.    </hibernate-configuration>
```

StoreTest.java

copy to clipboard

```
1.     package com.javatpoint;
2.
3.     import java.util.HashMap;
4.     import org.hibernate.*;
5.     import org.hibernate.cfg.*;
6.     public class StoreTest {
7.     public static void main(String[] args) {
8.     Session session=new Configuration().configure().buildSessionFactory().openSess
    ion();
9.     Transaction tx=session.beginTransaction();
10.
11.    HashMap<String,String> map1=new HashMap<String,String>();
12.    map1.put("java is a programming language","John Milton");
13.    map1.put("java is a platform","Ashok Kumar");
14.
15.    HashMap<String,String> map2=new HashMap<String,String>();
16.    map2.put("servlet technology is a server side programming","John Milton");
17.    map2.put("Servlet is an Interface","Ashok Kumar");
18.    map2.put("Servlet is a package","Rahul Kumar");
19.
20.    Question question1=new Question("What is java?","Alok",map1);
```

```
21.    Question question2=new Question("What is servlet?","Jai Dixit",map2);
22.
23.    session.persist(question1);
24.    session.persist(question2);
25.
26.    tx.commit();
27.    session.close();
28.    System.out.println("successfully stored");
29.    }
30.    }
```

FetchTest.java

copy to clipboard

```
1.     package com.javatpoint;
2.     import java.util.*;
3.     import org.hibernate.*;
4.     import org.hibernate.cfg.*;
5.     public class FetchTest {
6.     public static void main(String[] args) {
7.      Session session=new Configuration().configure().buildSessionFactory().openSes
    sion();
8.
9.      Query query=session.createQuery("from Question ");
10.    List<Question> list=query.list();
11.
12.    Iterator<Question> iterator=list.iterator();
13.    while(iterator.hasNext()){
14.     Question question=iterator.next();
15.     System.out.println("question id:"+question.getId());
16.     System.out.println("question name:"+question.getName());
17.     System.out.println("question posted by:"+question.getUsername());
18.     System.out.println("answers.....");
19.     Map<String,String> map=question.getAnswers();
20.     Set<Map.Entry<String,String>> set=map.entrySet();
21.
22.     Iterator<Map.Entry<String,String>> iteratoranswer=set.iterator();
23.     while(iteratoranswer.hasNext()){
24.      Map.Entry<String,String> entry=(Map.Entry<String,String>)iteratoranswer.ne
    xt();
25.      System.out.println("answer name:"+entry.getKey());
26.      System.out.println("answer posted by:"+entry.getValue());
27.     }
28.    }
29.    session.close();
30.    }
31.    }
```

Many to Many Mapping in hibernate by map Example (using xml file)

1. **Many to Many Mapping**
2. **Example of Many to Many Mapping**

We can map many to many relation either using set, bag, map etc. Here, we are going to use map for many-to-many mapping. In such case, three tables will be created.

Example of Many to Many Mapping

You need to create following pages for mapping map elements.

- Question.java
- User.java
- question.hbm.xml
- user.hbm.xml
- hibernate.cfg.xml
- StoreTest.java
- FetchTest.java

Question.java

copy to clipboard

```
1.  package com.javatpoint;
2.
3.  import java.util.Map;
4.
5.  public class Question {
6.  private int id;
7.  private String name;
8.  private Map<String,User> answers;
9.
10. public Question() {}
11. public Question(String name, Map<String, User> answers) {
12.    super();
13.    this.name = name;
14.    this.answers = answers;
15. }
16. public int getId() {
17.    return id;
18. }
19. public void setId(int id) {
20.    this.id = id;
21. }
22. public String getName() {
23.    return name;
24. }
25. public void setName(String name) {
26.    this.name = name;
27. }
28. public Map<String, User> getAnswers() {
29.    return answers;
30. }
31. public void setAnswers(Map<String, User> answers) {
32.    this.answers = answers;
33. }
34.
35.
36. }
```

User.java

copy to clipboard

```
1.  package com.javatpoint;
2.
3.  public class User {
4.  private int id;
5.  private String username,email,country;
6.
7.  public User() {}
8.  public User(String username, String email, String country) {
9.     super();
10.    this.username = username;
11.    this.email = email;
12.    this.country = country;
13. }
14. public int getId() {
15.    return id;
16. }
17.
18. public void setId(int id) {
19.    this.id = id;
20. }
21.
22. public String getUsername() {
23.    return username;
24. }
25.
26. public void setUsername(String username) {
27.    this.username = username;
28. }
29.
30. public String getEmail() {
31.    return email;
32. }
33.
34. public void setEmail(String email) {
35.    this.email = email;
36. }
37.
38. public String getCountry() {
39.    return country;
40. }
41.
42. public void setCountry(String country) {
43.    this.country = country;
44. }
45. }
```

question.hbm.xml

copy to clipboard

```
1.  <?xml version='1.0' encoding='UTF-8'?>
2.  <!DOCTYPE hibernate-mapping PUBLIC
3.          "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.          "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
```

```
6.    <hibernate-mapping>
7.    <class name="com.javatpoint.Question" table="question738">
8.    <id name="id">
9.    <generator class="native"></generator>
10.   </id>
11.   <property name="name"></property>
12.
13.   <map name="answers" table="answer738" cascade="all">
14.   <key column="questionid"></key>
15.   <index column="answer" type="string"></index>
16.   <many-to-many class="com.javatpoint.User" column="userid"></many-to-
      many>
17.   </map>
18.   </class>
19.
20.   </hibernate-mapping>
```

user.hbm.xml

copy to clipboard

```
1.    <?xml version='1.0' encoding='UTF-8'?>
2.    <!DOCTYPE hibernate-mapping PUBLIC
3.            "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.            "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.    <hibernate-mapping>
7.    <class name="com.javatpoint.User" table="user738">
8.    <id name="id">
9.    <generator class="native"></generator>
10.   </id>
11.   <property name="username"></property>
12.   <property name="email"></property>
13.   <property name="country"></property>
14.   </class>
15.
16.   </hibernate-mapping>
```

hibernate.cfg.xml

copy to clipboard

```
1.    <?xml version='1.0' encoding='UTF-8'?>
2.    <!DOCTYPE hibernate-configuration PUBLIC
3.            "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.            "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6.    <!-- Generated by MyEclipse Hibernate Tools.                   -->
7.    <hibernate-configuration>
8.
9.      <session-factory>
10.     <property name="hbm2ddl.auto">update</property>
11.     <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
12.     <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</prop
        erty>
13.     <property name="connection.username">system</property>
14.     <property name="connection.password">oracle</property>
15.     <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</pr
        operty>
16.
```

```
17.       <mapping resource="question.hbm.xml"/>
18.       <mapping resource="user.hbm.xml"/>
19.       </session-factory>
20.
21.    </hibernate-configuration>
```

StoreTest.java

copy to clipboard

```
1.    package com.javatpoint;
2.
3.    import java.util.HashMap;
4.    import org.hibernate.*;
5.    import org.hibernate.cfg.*;
6.    public class StoreTest {
7.    public static void main(String[] args) {
8.    Session session=new Configuration().configure().buildSessionFactory().openSess
   ion();
9.    Transaction tx=session.beginTransaction();
10.
11.   HashMap<String,User> map1=new HashMap<String,User>();
12.   map1.put("java is a programming language",
13.   new User("John Milton","john@gmail.com","usa"));
14.
15.   map1.put("java is a platform",
16.   new User("Ashok Kumar","ashok@gmail.com","india"));
17.
18.   HashMap<String,User> map2=new HashMap<String,User>();
19.   map2.put("servlet technology is a server side programming",
20.
21.   new User("John Milton","john@gmail.com","usa"));
22.   map2.put("Servlet is an Interface",
23.   new User("Ashok Kumar","ashok@gmail.com","india"));
24.
25.   Question question1=new Question("What is java?",map1);
26.   Question question2=new Question("What is servlet?",map2);
27.
28.   session.persist(question1);
29.   session.persist(question2);
30.
31.   tx.commit();
32.   session.close();
33.   System.out.println("successfully stored");
34.   }
35.   }
```

FetchTest.java

copy to clipboard

```
1.    package com.javatpoint;
2.    import java.util.*;
3.    import org.hibernate.*;
4.    import org.hibernate.cfg.*;
5.    public class FetchTest {
6.    public static void main(String[] args) {
7.     Session session=new Configuration().configure().buildSessionFactory().openSes
   sion();
8.     Query query=session.createQuery("from Question ");
9.     List<Question> list=query.list();
```

```
10.
11.     Iterator<Question> iterator=list.iterator();
12.     while(iterator.hasNext()){
13.      Question question=iterator.next();
14.      System.out.println("question id:"+question.getId());
15.      System.out.println("question name:"+question.getName());
16.      System.out.println("answers.....");
17.      Map<String,User> map=question.getAnswers();
18.      Set<Map.Entry<String,User>> set=map.entrySet();
19.
20.      Iterator<Map.Entry<String,User>> iteratoranswer=set.iterator();
21.      while(iteratoranswer.hasNext()){
22.       Map.Entry<String,User> entry=(Map.Entry<String,User>)iteratoranswer.next()
    ;
23.       System.out.println("answer name:"+entry.getKey());
24.       System.out.println("answer posted by.....");
25.       User user=entry.getValue();
26.       System.out.println("username:"+user.getUsername());
27.       System.out.println("user emailid:"+user.getEmail());
```

```
System.out.println("user country:"+user.getCountry(
```

One to One Mapping in Hibernate by many-to-one example

1. **Example of One to one mapping in hibernate**
    1. **Persistent classes for one to one mapping**
    2. **Mapping files for the persistent classes**
    3. **Configuration file**
    4. **User classes to store and fetch the data**

We can perform one to one mapping in hibernate by two ways:

- By many-to-one element
- By one-to-one element

Here, we are going to perform one to one mapping by many-to-one element. In such case, a foreign key is created in the primary table.

In this example, one employee can have one address and one address belongs to one employee only. Here, we are using bidirectional association. Let's look at the persistent classes.

One to One Mapping in Hibernate by many-to-one example

1. **Example of One to one mapping in hibernate**
    1. **Persistent classes for one to one mapping**
    2. **Mapping files for the persistent classes**
    3. **Configuration file**
    4. **User classes to store and fetch the data**

We can perform one to one mapping in hibernate by two ways:

- By many-to-one element
- By one-to-one element

Here, we are going to perform one to one mapping by many-to-one element. In such case, a foreign key is created in the primary table.

In this example, one employee can have one address and one address belongs to one employee only. Here, we are using bidirectional association. Let's look at the persistent classes.

1) Persistent classes for one to one mapping

There are two persistent classes Employee.java and Address.java. Employee class contains Address class reference and vice versa.

Employee.java

copy to clipboard

```
1.      package com.javatpoint;
2.
3.      public class Employee {
4.      private int employeeId;
5.      private String name,email;
6.      private Address address;
7.      //setters and getters
8.      }
```

Address.java

copy to clipboard

```
1.      package com.javatpoint;
2.
3.      public class Address {
4.      private int addressId;
5.      private String addressLine1,city,state,country;
6.      private int pincode;
7.      private Employee employee;
8.      //setters and getters
9.      }
```

---

2) Mapping files for the persistent classes

The two mapping files are employee.hbm.xml and address.hbm.xml.

employee.hbm.xml

In this mapping file we are using **many-to-one** element with unique="true" attribute to make the one to one mapping.

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-mapping PUBLIC
3.              "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.              "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.              <hibernate-mapping>
7.              <class name="com.javatpoint.Employee" table="emp211">
8.              <id name="employeeId">
9.              <generator class="increment"></generator>
10.             </id>
11.             <property name="name"></property>
12.             <property name="email"></property>
13.
14.             <many-to-one name="address" unique="true" cascade="all"></many-to-
    one>
15.             </class>
16.
17.             </hibernate-mapping>
```

address.hbm.xml

This is the simple mapping file for the Address class.

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-mapping PUBLIC
3.              "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.              "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.              <hibernate-mapping>
7.              <class name="com.javatpoint.Address" table="address211">
8.              <id name="addressId">
9.              <generator class="increment"></generator>
10.             </id>
11.             <property name="addressLine1"></property>
12.             <property name="city"></property>
13.             <property name="state"></property>
14.             <property name="country"></property>
15.
16.             </class>
17.
18.             </hibernate-mapping>
```

---

3) Configuration file

This file contains information about the database and mapping file.

hibernate.cfg.xml

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-configuration PUBLIC
3.              "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
```

```
4.              "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6.      <!-- Generated by MyEclipse Hibernate Tools.                 -->
7.      <hibernate-configuration>
8.
9.          <session-factory>
10.             <property name="hbm2ddl.auto">update</property>
11.             <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>

12.             <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</p
    roperty>
13.             <property name="connection.username">system</property>
14.             <property name="connection.password">oracle</property>
15.             <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver
    </property>
16.         <mapping resource="employee.hbm.xml"/>
17.         <mapping resource="address.hbm.xml"/>
18.         </session-factory>
19.
20.     </hibernate-configuration>
```

4) User classes to store and fetch the data

Store.java

copy to clipboard

```
1.      package com.javatpoint;
2.      import org.hibernate.cfg.*;
3.      import org.hibernate.*;
4.
5.      public class Store {
6.      public static void main(String[] args) {
7.          Configuration cfg=new Configuration();
8.          cfg.configure("hibernate.cfg.xml");
9.          SessionFactory sf=cfg.buildSessionFactory();
10.         Session session=sf.openSession();
11.         Transaction tx=session.beginTransaction();
12.
13.         Employee e1=new Employee();
14.         e1.setName("Ravi Malik");
15.         e1.setEmail("ravi@gmail.com");
16.
17.         Address address1=new Address();
18.         address1.setAddressLine1("G-21,Lohia nagar");
19.         address1.setCity("Ghaziabad");
20.         address1.setState("UP");
21.         address1.setCountry("India");
22.         address1.setPincode(201301);
23.
24.
25.         e1.setAddress(address1);
26.         address1.setEmployee(e1);
27.
28.         session.persist(e1);
29.         tx.commit();
```

```
30.
31.         session.close();
32.         System.out.println("success");
33.      }
34.      }
```

Fetch.java

copy to clipboard

```
1.       package com.javatpoint;
2.       import java.util.Iterator;
3.       import java.util.List;
4.       import org.hibernate.Query;
5.       import org.hibernate.Session;
6.       import org.hibernate.SessionFactory;
7.       import org.hibernate.cfg.Configuration;
8.
9.       public class Fetch {
10.      public static void main(String[] args) {
11.         Configuration cfg=new Configuration();
12.         cfg.configure("hibernate.cfg.xml");
13.         SessionFactory sf=cfg.buildSessionFactory();
14.         Session session=sf.openSession();
15.
16.         Query query=session.createQuery("from Employee e");
17.         List<employee> list=query.list();
18.
19.         Iterator<employee> itr=list.iterator();
20.         while(itr.hasNext()){
21.          Employee emp=itr.next();
22.          System.out.println(emp.getEmployeeId()+" "+emp.getName()+" "+emp.getE
    mail());
23.          Address address=emp.getAddress();
24.          System.out.println(address.getAddressLine1()+" "+address.getCity()+" "+
25.             address.getState()+" "+address.getCountry());
26.         }
27.
28.         session.close();
29.         System.out.println("success");
30.      }
31.      }
32.
33.
34.      </employee></employee>
```

One to One Mapping in Hibernate by one-to-one example

1. **Example of One to one mapping in hibernate**
    1. **Persistent classes for one to one mapping**
    2. **Mapping files for the persistent classes**
    3. **Configuration file**
    4. **User classes to store and fetch the data**

As We have discussed in the previous example, there are two ways to perform one to one mapping in hibernate:

- By many-to-one element
- By one-to-one element

Here, we are going to perform one to one mapping by one-to-one element. In such case, no foreign key is created in the primary table.

In this example, one employee can have one address and one address belongs to one employee only. Here, we are using bidirectional association. Let's look at the persistent classes.

1) Persistent classes for one to one mapping

There are two persistent classes Employee.java and Address.java. Employee class contains Address class reference and vice versa.

Employee.java

copy to clipboard

```
1.      package com.javatpoint;
2.
3.      public class Employee {
4.      private int employeeId;
5.      private String name,email;
6.      private Address address;
7.      //setters and getters
8.      }
```

Address.java

copy to clipboard

```
1.      package com.javatpoint;
2.
3.      public class Address {
4.      private int addressId;
5.      private String addressLine1,city,state,country;
6.      private int pincode;
7.      private Employee employee;
8.      //setters and getters
9.      }
```

---

2) Mapping files for the persistent classes

The two mapping files are employee.hbm.xml and address.hbm.xml.

employee.hbm.xml

In this mapping file we are using **one-to-one** element in both the mapping files to make the one to one mapping.

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-mapping PUBLIC
3.              "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.              "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.               <hibernate-mapping>
7.              <class name="com.javatpoint.Employee" table="emp212">
8.              <id name="employeeId">
9.              <generator class="increment"></generator>
10.             </id>
11.             <property name="name"></property>
12.             <property name="email"></property>
13.
14.             <one-to-one name="address" cascade="all"></one-to-one>
15.             </class>
16.
17.             </hibernate-mapping>
```

address.hbm.xml

This is the simple mapping file for the Address class. But the important thing is generator class. Here, we are using **foreign** generator class that depends on the Employee class primary key.

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-mapping PUBLIC
3.              "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.              "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.               <hibernate-mapping>
7.              <class name="com.javatpoint.Address" table="address212">
8.              <id name="addressId">
9.              <generator class="foreign">
10.             <param name="property">employee</param>
11.             </generator>
12.             </id>
13.             <property name="addressLine1"></property>
14.             <property name="city"></property>
15.             <property name="state"></property>
16.             <property name="country"></property>
17.
18.             <one-to-one name="employee"></one-to-one>
19.             </class>
20.
21.             </hibernate-mapping>
```

---

3) Configuration file

This file contains information about the database and mapping file.

hibernate.cfg.xml

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-configuration PUBLIC
3.              "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.              "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6.      <!-- Generated by MyEclipse Hibernate Tools.                 -->
7.      <hibernate-configuration>
8.
9.          <session-factory>
10.             <property name="hbm2ddl.auto">update</property>
11.             <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>

12.             <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</p
    roperty>
13.             <property name="connection.username">system</property>
14.             <property name="connection.password">oracle</property>
15.             <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver
    </property>
16.         <mapping resource="employee.hbm.xml"/>
17.         <mapping resource="address.hbm.xml"/>
18.         </session-factory>
19.
20.     </hibernate-configuration>
```

---

4) User classes to store and fetch the data

Store.java

copy to clipboard

```
1.      package com.javatpoint;
2.      import org.hibernate.cfg.*;
3.      import org.hibernate.*;
4.
5.      public class Store {
6.      public static void main(String[] args) {
7.          Configuration cfg=new Configuration();
8.          cfg.configure("hibernate.cfg.xml");
9.          SessionFactory sf=cfg.buildSessionFactory();
10.         Session session=sf.openSession();
11.         Transaction tx=session.beginTransaction();
12.
13.         Employee e1=new Employee();
14.         e1.setName("Ravi Malik");
15.         e1.setEmail("ravi@gmail.com");
16.
17.         Address address1=new Address();
18.         address1.setAddressLine1("G-21,Lohia nagar");
19.         address1.setCity("Ghaziabad");
20.         address1.setState("UP");
21.         address1.setCountry("India");
22.         address1.setPincode(201301);
```

```
23.
24.
25.         e1.setAddress(address1);
26.         address1.setEmployee(e1);
27.
28.         session.persist(e1);
29.         tx.commit();
30.
31.         session.close();
32.         System.out.println("success");
33.      }
34.      }
```

Fetch.java

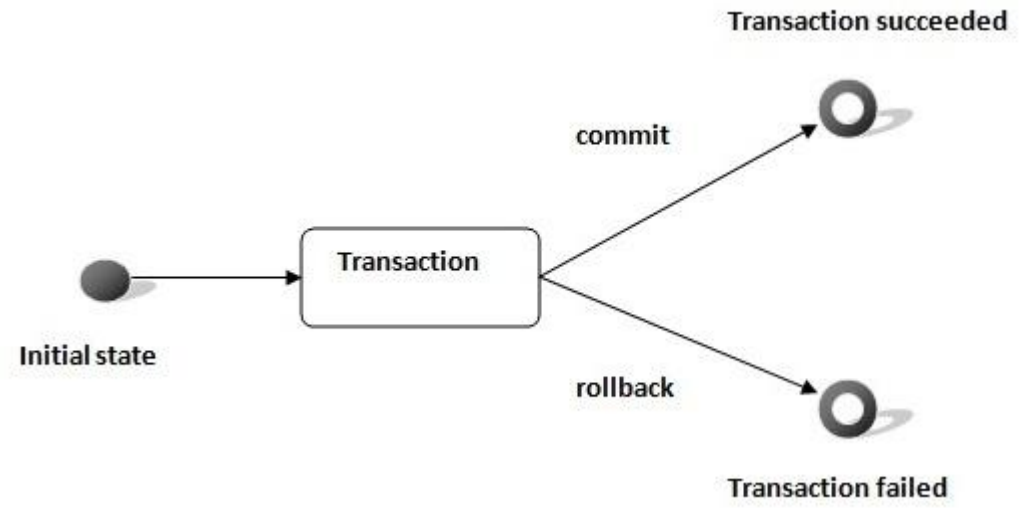
copy to clipboard

```
1.       package com.javatpoint;
2.       import java.util.Iterator;
3.       import java.util.List;
4.       import org.hibernate.Query;
5.       import org.hibernate.Session;
6.       import org.hibernate.SessionFactory;
7.       import org.hibernate.cfg.Configuration;
8.
9.       public class Fetch {
10.      public static void main(String[] args) {
11.         Configuration cfg=new Configuration();
12.         cfg.configure("hibernate.cfg.xml");
13.         SessionFactory sf=cfg.buildSessionFactory();
14.         Session session=sf.openSession();
15.
16.         Query query=session.createQuery("from Employee e");
17.         List<employee> list=query.list();
18.
19.         Iterator<employee> itr=list.iterator();
20.         while(itr.hasNext()){
21.          Employee emp=itr.next();
22.          System.out.println(emp.getEmployeeId()+" "+emp.getName()+" "+emp.getE
    mail());
23.          Address address=emp.getAddress();
24.          System.out.println(address.getAddressLine1()+" "+address.getCity()+" "+
25.             address.getState()+" "+address.getCountry());
26.         }
27.
28.         session.close();
29.         System.out.println("success");
30.      }
31.      }
32.
33.
34.      </employee></employee>
```

Hibernate Transaction Management Example

1. **Understanding Transaction**
2. **ACID Property**
3. **Hibernate Transaction Management**

A **transaction** simply represents a unit of work. In such case, if one step fails, the whole transaction fails (which is termed as atomicity). A transaction can be described by ACID properties (Atomicity, Consistency, Isolation and Durability).



Transaction Interface in Hibernate

In hibernate framework, we have **Transaction** interface that defines the unit of work. It maintains abstraction from the transaction implementation (JTA,JDBC).

A transaction is associated with Session and instantiated by calling **session.beginTransaction()**.

The methods of Transaction interface are as follows:

1. **void begin()** starts a new transaction.
2. **void commit()** ends the unit of work unless we are in FlushMode.NEVER.
3. **void rollback()** forces this transaction to rollback.
4. **void setTimeout(int seconds)** it sets a transaction timeout for any transaction started by a subsequent call to begin on this instance.
5. **boolean isAlive()** checks if the transaction is still alive.
6. **void registerSynchronization(Synchronization s)** registers a user synchronization callback for this transaction.
7. **boolean wasCommited()** checks if the transaction is commited successfully.
8. **boolean wasRolledBack()** checks if the transaction is rolledback successfully.

Example of Transaction Management in Hibernate

In hibernate, it is better to rollback the transaction if any exception occurs, so that resources can be free. Let's see the example of transaction management in hibernate.

```
copy to clipboard
1.    Session session = null;
2.    Transaction tx = null;
3.
4.    try {
5.    session = sessionFactory.openSession();
6.    tx = session.beginTransaction();
7.    //some action
8.
9.    tx.commit();
10.
11.   }catch (Exception ex) {
12.   ex.printStackTrace();
13.   tx.rollback();
14.   }
15.   finally {session.close();}
```

Hibernate Query Language (HQL)

1. Hibernate Query Language
2. Advantage of HQL
3. Query Interface

Hibernate Query Language (HQL) is same as SQL (Structured Query Language) but it doesn't depends on the table of the database. Instead of table name, we use class name in HQL. So it is database independent query language.

Advantage of HQL

There are many advantages of HQL. They are as follows:

- database independent
- supports polymorphic queries
- easy to learn for Java Programmer

Query Interface

It is an object oriented representation of Hibernate Query. The object of Query can be obtained by calling the createQuery() method Session interface.

The query interface provides many methods. There is given commonly used methods:

1. **public int executeUpdate()** is used to execute the update or delete query.
2. **public List list()** returns the result of the ralation as a list.

3. **public Query setFirstResult(int rowno)** specifies the row number from where record will be retrieved.
4. **public Query setMaxResult(int rowno)** specifies the no. of records to be retrieved from the relation (table).
5. **public Query setParameter(int position, Object value)** it sets the value to the JDBC style query parameter.
6. **public Query setParameter(String name, Object value)** it sets the value to a named query parameter.

Example of HQL to get all the records

copy to clipboard

```
1.    Query query=session.createQuery("from Emp");//here persistent class name is E
      mp
2.    List list=query.list();
```

Example of HQL to get records with pagination

copy to clipboard

```
1.    Query query=session.createQuery("from Emp");
2.    query.setFirstResult(5);
3.    query.setMaxResult(10);
4.    List list=query.list();//will return the records from 5 to 10th number
```

Example of HQL update query

copy to clipboard

```
1.    Transaction tx=session.beginTransaction();
2.    Query q=session.createQuery("update User set name=:n where id=:i");
3.    q.setParameter("n","Udit Kumar");
4.    q.setParameter("i",111);
5.
6.    int status=q.executeUpdate();
7.    System.out.println(status);
8.    tx.commit();
```

Example of HQL delete query

copy to clipboard

```
1.    Query query=session.createQuery("delete from Emp where id=100");
2.    //specifying class name (Emp) not tablename
3.    query.executeUpdate();
```

HQL with Aggregate functions

You may call avg(), min(), max() etc. aggregate functions by HQL. Let's see some common examples:

Example to get total salary of all the employees

```
copy to clipboard
1.    Query q=session.createQuery("select sum(salary) from Emp");
2.    List<Emp> list=q.list();
3.       Iterator<Emp> itr=list.iterator();
4.       while(itr.hasNext()){
5.          System.out.println(itr.next());
6.    }
```

Example to get maximum salary of employee

```
copy to clipboard
1.    Query q=session.createQuery("select max(salary) from Emp");
```

Example to get minimum salary of employee

```
copy to clipboard
1.    Query q=session.createQuery("select min(salary) from Emp");
```

Example to count total number of employee ID

```
copy to clipboard
1.    Query q=session.createQuery("select count(id) from Emp");
```

Example to get average salary of each employees

```
copy to clipboard
1.    Query q=session.createQuery("select avg(salary) from Emp");
```

HCQL (Hibernate Criteria Query Language)

1. **Hibernate Criteria Query Language**
2. **Criteria Interface**
3. **Restrictions class**
4. **Examples of HCQL**

The Hibernate Criteria Query Language (HCQL) is used to fetch the records based on the specific criteria. The Criteria interface provides methods to apply criteria such as retreiving all the records of table whose salary is greater than 50000 etc.

Advantage of HCQL

The HCQL provides methods to add criteria, so it is **easy** for the java programmer to add criteria. The java programmer is able to add many criteria on a query.

Criteria Interface

The Criteria interface provides many methods to specify criteria. The object of Criteria can be obtained by calling the **createCriteria()** method of Session interface.

Syntax of createCriteria() method of Session interface

copy to clipboard

```
1.      public Criteria createCriteria(Class c)
```

The commonly used methods of Criteria interface are as follows:

1. **public Criteria add(Criterion c)** is used to add restrictions.
2. **public Criteria addOrder(Order o)** specifies ordering.
3. **public Criteria setFirstResult(int firstResult)** specifies the first number of record to be retreived.
4. **public Criteria setMaxResult(int totalResult)** specifies the total number of records to be retreived.
5. **public List list()** returns list containing object.
6. **public Criteria setProjection(Projection projection)** specifies the projection.

Restrictions class

Restrictions class provides methods that can be used as Criterion. The commonly used methods of Restrictions class are as follows:

1. **public static SimpleExpression lt(String propertyName,Object value)** sets the **less than** constraint to the given property.
2. **public static SimpleExpression le(String propertyName,Object value)** sets the **less than or equal** constraint to the given property.
3. **public static SimpleExpression gt(String propertyName,Object value)** sets the **greater than** constraint to the given property.
4. **public static SimpleExpression ge(String propertyName,Object value)** sets the **greater than or equal** than constraint to the given property.
5. **public static SimpleExpression ne(String propertyName,Object value)** sets the **not equal** constraint to the given property.
6. **public static SimpleExpression eq(String propertyName,Object value)** sets the **equal** constraint to the given property.
7. **public static Criterion between(String propertyName, Object low, Object high)** sets the **between** constraint.
8. **public static SimpleExpression like(String propertyName, Object value)** sets the **like** constraint to the given property.

Order class

The Order class represents an order. The commonly used methods of Restrictions class are as follows:

1. **public static Order asc(String propertyName)** applies the ascending order on the basis of given property.
2. **public static Order desc(String propertyName)** applies the descending order on the basis of given property.

Examples of Hibernate Criteria Query Language

There are given a lot of examples of HCQL.

Example of HCQL to get all the records

copy to clipboard

```
1.      Crietria c=session.createCriteria(Emp.class);//passing Class class argument
2.      List list=c.list();
```

Example of HCQL to get the 10th to 20th record

copy to clipboard

```
1.      Crietria c=session.createCriteria(Emp.class);
2.      c.setFirstResult(10);
3.      c.setMaxResult(20);
4.      List list=c.list();
```

Example of HCQL to get the records whose salary is greater than 10000

copy to clipboard

```
1.      Crietria c=session.createCriteria(Emp.class);
2.      c.add(Restrictions.gt("salary",10000));//salary is the propertyname
3.      List list=c.list();
```

Example of HCQL to get the records in ascending order on the basis of salary

copy to clipboard

```
Crietria c=session.createCriteria(Emp.class);
c.addOrder(Order.asc("salary"));
List list=c.list();
```

HCQL with Projection

We can fetch data of a particular column by projection such as name etc. Let's see the simple example of projection that prints data of NAME column of the table only.

copy to clipboard

```
Criteria c=session.createCriteria(Emp.class);
c.setProjection(Projections.property("name"));
List list=c.list();
```

Hibernate Named Query

1. Hibernate Named Query
2. Hibernate Named Query by annotation
3. Example of Hibernate Named Query by annotation
4. Hibernate Named Query by mapping file

The hibernate named query is way to use any query by some meaningful name. It is like using alias names. The Hibernate framework provides the concept of named queries so that application programmer need not to scatter queries to all the java code.

There are two ways to define the named query in hibernate

- by annotation
- by mapping file.

Hibernate Named Query by annotation

If you want to use named query in hibernate, you need to have knowledge of @NamedQueries and @NamedQuery annotations.

@NameQueries annotation is used to define the multiple named queries.

@NameQuery annotation is used to define the single named query.

Let's see the example of using the named queries:

copy to clipboard

```
@NamedQueries(
   {
      @NamedQuery(
```

```
4.            name = "findEmployeeByName",
5.            query = "from Employee e where e.name = :name"
6.            )
7.         }
8.      )
```

Example of Hibernate Named Query by annotation

In this example, we are using annotations to defined the named query in the persistent class. There are three files only:

- Employee.java
- hibernate.cfg.xml
- FetchDemo

In this example, we are assuming that there is em table in the database containing 4 columns id, name, job and salary and there are some records in this table.

Employee.java

It is a persistent class that uses annotations to define named query and marks this class as entity.

copy to clipboard

```
1.    package com.javatpoint;
2.
3.    import javax.persistence.*;
4.    import javax.persistence.Entity;
5.    import javax.persistence.GeneratedValue;
6.    import javax.persistence.Id;
7.
8.    @NamedQueries(
9.       {
10.         @NamedQuery(
11.         name = "findEmployeeByName",
12.         query = "from Employee e where e.name = :name"
13.         )
14.      }
15.   )
16.
17.   @Entity
18.   @Table(name="em")
19.   public class Employee {
20.
21.      public String toString(){return id+" "+name+" "+salary+" "+job;}
22.
23.      int id;
24.      String name;
25.      int salary;
26.      String job;
27.      @Id
28.      @GeneratedValue(strategy=GenerationType.AUTO)
```

```
29.
30.        //getters and setters
31.     }
```

---

hibernate.cfg.xml

It is a configuration file that stores the informations about database such as driver class, url, username, password and mapping class etc.

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-configuration PUBLIC
3.              "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.              "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6.      <hibernate-configuration>
7.
8.         <session-factory>
9.            <property name="hbm2ddl.auto">update</property>
10.           <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>

11.           <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</p
    roperty>
12.           <property name="connection.username">system</property>
13.           <property name="connection.password">oracle</property>
14.           <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver
    </property>
15.        <mapping class="com.javatpoint.Employee"/>
16.        </session-factory>
17.
18.     </hibernate-configuration>
```

FetchData.java

It is a java class that uses the named query and prints the informations based on the query. The **getNamedQuery** method uses the named query and returns the instance of Query.

copy to clipboard

```
1.      package com.javatpoint;
2.
3.      import java.util.Iterator;
4.      import java.util.List;
5.
6.      import org.hibernate.cfg.AnnotationConfiguration;
7.      import org.hibernate.*;
8.
9.      public class FetchData {
10.      public static void main(String[] args) {
11.
12.       AnnotationConfiguration configuration=new AnnotationConfiguration();
13.        configuration.configure("hibernate.cfg.xml");
```

```
14.         SessionFactory sFactory=configuration.buildSessionFactory();
15.          Session session=sFactory.openSession();
16.
17.          //Hibernate Named Query
18.          Query query = session.getNamedQuery("findEmployeeByName");
19.          query.setString("name", "amit");
20.
21.          List<employee> employees=query.list();
22.
23.          Iterator<employee> itr=employees.iterator();
24.           while(itr.hasNext()){
25.          Employee e=itr.next();
26.          System.out.println(e);
27.           }
28.
29.          session.close();
30.
31.        }
32.      }
33.
34.
35.      </employee></employee>
```

download this hibernate example (developed using Myeclipse IDE)

Hibernate Named Query by mapping file

If want to define named query by mapping file, you need to use **query** element of hibernate-mapping to define the named query.

In such case, you need to create hbm file that defines the named query. Other resources are same as given in the above example except Persistent class Employee.java where you don't need to use any annotation and hibernate.cfg.xml file where you need to specify mapping resource of the hbm file.

The hbm file should be like this:

emp.hbm.xml

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-mapping PUBLIC
3.              "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.              "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.      <hibernate-mapping>
7.      <class name="com.javatpoint.Employee" table="em">
8.      <id name="id">
9.      <generator class="native"></generator>
10.     </id>
11.     <property name="name"></property>
12.     <property name="job"></property>
13.     <property name="salary"></property>
14.     </class>
```

```
15.
16.     <query name="findEmployeeByName">
17.     <![CDATA[from Employee e where e.name = :name]]>
18.     </query>
19.
20.     </hibernate-mapping>
```

The persistent class should be like this:

Employee.java

copy to clipboard

```
1.      package com.javatpoint;
2.      public class Employee {
3.         int id;
4.         String name;
5.         int salary;
6.         String job;
7.         //getters and setters
8.      }
```

Now include the mapping resource in the hbm file as:

hibernate.cfg.xml

copy to clipboard

```
1.      <mapping resource="emp.hbm.xml"/>
```

Caching in Hibernate

1. **Caching in Hibernate**
2. **First Level Cache**
3. **Second Level Cache**

Hibernate caching improves the performance of the application by pooling the object in the cache.

There are mainly two types of caching: first level cache and second level cache.

First Level Cache

Session object holds the first level cache data. It is enabled by default. The first level cache data will not be available to entire application. An application can use many session object.

Second Level Cache

SessionFactory object holds the second level cache data. The data stored in the second level cache will be available to entire application. But we need to enable it explicitely.

Second Level Cache implementations are provided by different vendors such as:

- EH (Easy Hibernate) Cache
- Swarm Cache
- OS Cache
- JBoss Cache

Hibernate Second Level Cache

1. **Hibernate Second Level Cache**
2. **Example of Second Level Cache**

SecondFactory holds the second level cache data. It is global for all the session objects and not enabled bydefault.

Different vendors have provided the implementation of Second Level Cache :

- EH Cache
- Swarm Cache
- OS Cache
- JBoss Cache

Example of Second Level Cahce using EH Cache

To understand the second level cache through example, we need to create following pages:

1. Employee.java
2. employee.hbm.xml
3. hibernate.cfg.xml
4. ehcache.xml
5. FetchTest.java

Here, we are assuming, there is emp1012 table in the oracle database containing some records.

File: Employee.java

copy to clipboard

```
1.      package com.javatpoint;
2.
3.      public class Employee {
4.      private int id;
5.      private String name;
6.      private float salary;
7.
8.      public Employee() {}
9.      public Employee(String name, float salary) {
10.        super();
11.        this.name = name;
12.        this.salary = salary;
```

```
13.     }
14.     //setters and getters
15.     }
```

File: employee.hbm.xml

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-mapping PUBLIC
3.              "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.              "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.              <hibernate-mapping>
7.              <class name="com.javatpoint.Employee" table="emp1012">
8.              <cache usage="read-only" />
9.              <id name="id">
10.             <generator class="native"></generator>
11.             </id>
12.             <property name="name"></property>
13.             <property name="salary"></property>
14.             </class>
15.
16.             </hibernate-mapping>
```

Here, we are using **read-only** cache usage for the class. The cache usage can also be used in collection.

File: hibernate.cfg.xml

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-configuration PUBLIC
3.              "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.              "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6.      <!-- Generated by MyEclipse Hibernate Tools.                 -->
7.      <hibernate-configuration>
8.
9.          <session-factory>
10.             <property name="show_sql">true</property>
11.             <property name="hbm2ddl.auto">update</property>
12.             <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>

13.             <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</p
    roperty>
14.             <property name="connection.username">system</property>
15.             <property name="connection.password">oracle</property>
16.             <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver
    </property>
17.
18.             <property name="cache.provider_class">org.hibernate.cache.EhCacheProvi
    der</property>
```

```
19.             <property name="hibernate.cache.use_second_level_cache">true</prope
    rty>
20.
21.         <mapping resource="employee.hbm.xml"/>
22.         </session-factory>
23.
24.     </hibernate-configuration>
```

To implement second level cache, we need to define **cache.provider_class** property in the configuration file.

File: ehcache.xml

copy to clipboard

```
1.      <?xml version="1.0"?>
2.      <ehcache>
3.      <defaultCache
4.      maxElementsInMemory="100"
5.      eternal="false"
6.      timeToIdleSeconds="120"
7.      timeToLiveSeconds="200" />
8.
9.      <cache name="com.javatpoint.Employee"
10.     maxElementsInMemory="100"
11.     eternal="false"
12.     timeToIdleSeconds="5"
13.     timeToLiveSeconds="200" />
14.     </ehcache>
```

You need to create ehcache.xml file to define the cache property.

defaultCache will be used for all the persistent classes. We can also define persistent class explicitely by using the cache element.

eternal If we specify eternal="true", we don't need to define timeToIdleSeconds and timeToLiveSeconds attributes because it will be handled by hibernate internally. Specifying eternal="false" gives control to the programmer, but we need to define timeToIdleSeconds and timeToLiveSeconds attributes.

timeToIdleSeconds It defines that how many seconds object can be idle in the second level cache.

timeToLiveSeconds It defines that how many seconds object can be stored in the second level cache whether it is idle or not.

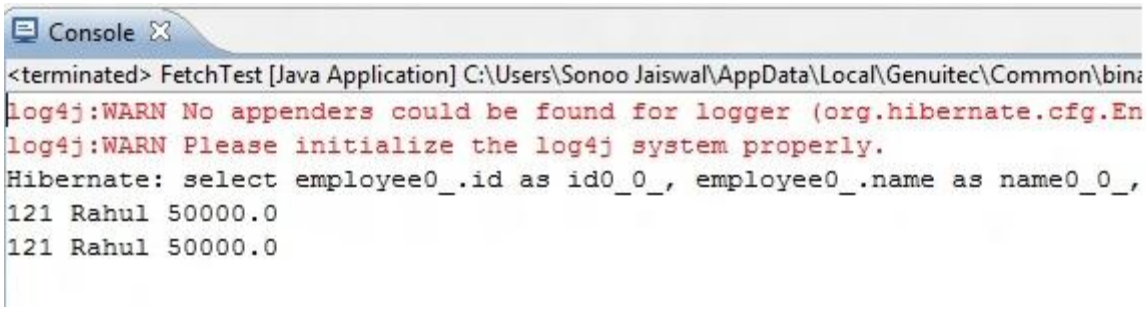
File: FetchTest.java

copy to clipboard

```
1.      package com.javatpoint;
2.
```

```
3.      import org.hibernate.Session;
4.      import org.hibernate.SessionFactory;
5.      import org.hibernate.cfg.Configuration;
6.
7.      public class FetchTest {
8.      public static void main(String[] args) {
9.      Configuration cfg=new Configuration().configure("hibernate.cfg.xml");
10.     SessionFactory factory=cfg.buildSessionFactory();
11.
12.        Session session1=factory.openSession();
13.        Employee emp1=(Employee)session1.load(Employee.class,121);
14.        System.out.println(emp1.getId()+" "+emp1.getName()+" "+emp1.getSalary()
    );
15.        session1.close();
16.
17.        Session session2=factory.openSession();
18.        Employee emp2=(Employee)session2.load(Employee.class,121);
19.        System.out.println(emp2.getId()+" "+emp2.getName()+" "+emp2.getSalary()
    );
20.        session2.close();
21.
22.     }
23.     }
```

Output:



As we can see here, hibernate does not fire query twice. If you don't use second level cache, hibernate will fire query twice because both query uses different session objects.

Hibernate and Struts 2 Integration

1. **Hibernate and Struts2 Integration**
2. **Example of Hibernate and struts2 integration**

We can integrate any **struts application with hibernate**. There is no requirement of extra efforts.

In this example, we going to use struts 2 framework with hibernate. You need to have jar files for struts 2 and hibernate.

Example of Hibernate and struts2 integration

In this example, we are creating the registration form using struts2 and storing this data into the database using Hibernate. Let's see the files that we should create to integrate the struts2 application with hibernate.

- **index.jsp** file to get input from the user.
- **User.java** A action class for handling the request. It uses the dao class to store the data.
- **RegisterDao.java** A java class that uses DAO design pattern to store the data using hibernate.
- **user.hbm.xml** A mapping file that contains information about the persistent class. In this case, action class works as the persistent class.
- **hibernate.cfg.xml** A configuration file that contains informations about the database and mapping file.
- **struts.xml** file contains information about the action class and result page to be invoked.
- **welcome.jsp** A jsp file that displays the welcome information with username.
- **web.xml** A web.xml file that contains information about the Controller of Struts framework.

index.jsp

In this page, we have created a form using the struts tags. The action name for this form is register.

copy to clipboard

```
<%@ taglib uri="/struts-tags" prefix="S" %>

<S:form action="register">
<S:textfield name="name" label="Name"></S:textfield>
<S:submit value="register"></S:submit>

</S:form>
```

User.java

It is a simple POJO class. Here it works as the action class for struts and persistent class for hibernate. It calls the register method of RegisterDao class and returns success as the string.

copy to clipboard

```
package com.javatpoint;

public class User {
private int id;
private String name;
//getters and setters

public String execute(){
```

```
9.          RegisterDao.saveUser(this);
10.         return "success";
11.     }
12.
13.     }
```

RegisterDao.java

It is a java class that saves the object of User class using the Hibernate framework.

copy to clipboard

```
1.      package com.javatpoint;
2.
3.      import org.hibernate.Session;
4.      import org.hibernate.Transaction;
5.      import org.hibernate.cfg.Configuration;
6.
7.      public class RegisterDao {
8.
9.      public static int saveUser(User u){
10.
11.     Session session=new Configuration().
12.     configure("hibernate.cfg.xml").buildSessionFactory().openSession();
13.
14.     Transaction t=session.beginTransaction();
15.     int i=(Integer)session.save(u);
16.     t.commit();
17.     session.close();
18.
19.     return i;
20.
21.     }
22.
23.     }
```

---

user.hbm.xml

This mapping file contains all the information of the persitent class.

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-mapping PUBLIC
3.      "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.      "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.      <hibernate-mapping>
7.      <class name="com.javatpoint.User" table="user451">
8.      <id name="id">
9.      <generator class="increment"></generator>
10.     </id>
11.     <property name="name"></property>
12.     </class>
```

```
13.
14.     </hibernate-mapping>
```

hibernate.cfg.xml

This configuration file contains informations about the database and mapping file. Here, we are using the hb2ddl.auto property, so you don't need to create the table in the database.

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-configuration PUBLIC
3.              "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4.              "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
5.
6.      <!-- Generated by MyEclipse Hibernate Tools.                   -->
7.      <hibernate-configuration>
8.
9.      <session-factory>
10.     <property name="hbm2ddl.auto">update</property>
11.     <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
12.     <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</propert
    y>
13.     <property name="connection.username">system</property>
14.     <property name="connection.password">oracle</property>
15.     <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</pro
    perty>
16.     <mapping resource="user.hbm.xml"/>
17.
18.     </session-factory>
19.
20.     </hibernate-configuration>
```

struts.xml

This files contains information about the action class to be invoked. Here the action class is User.

copy to clipboard

```
1.      <?xml version="1.0" encoding="UTF-8" ?>
2.      <!DOCTYPE struts PUBLIC "-//Apache Software Foundation
3.      //DTD Struts Configuration 2.1//EN"
4.      "http://struts.apache.org/dtds/struts-2.1.dtd">
5.      <struts>
6.
7.      <package name="abc" extends="struts-default">
8.      <action name="register" class="com.javatpoint.User">
9.      <result name="success">welcome.jsp</result>
10.     </action>
11.     </package>
12.     </struts>
```

welcome.jsp

It is the welcome file, that displays the welcome message with username.

copy to clipboard

```
1.      <%@ taglib uri="/struts-tags" prefix="S" %>
2.
3.      Welcome: <S:property value="name"/>
```

---

web.xml

It is web.xml file that contains the information about the controller. In case of Struts2, StrutsPrepareAndExecuteFilter class works as the controller.

copy to clipboard

```
1.      <?xml version="1.0" encoding="UTF-8"?>
2.      <web-app version="2.5"
3.         xmlns="http://java.sun.com/xml/ns/javaee"
4.         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5.         xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
6.         http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd">
7.        <welcome-file-list>
8.          <welcome-file>index.jsp</welcome-file>
9.        </welcome-file-list>
10.       <filter>
11.         <filter-name>struts2</filter-name>
12.         <filter-class>
13.       org.apache.struts2.dispatcher.ng.filter.StrutsPrepareAndExecuteFilter
14.         </filter-class>
15.       </filter>
16.       <filter-mapping>
17.         <filter-name>struts2</filter-name>
18.         <url-pattern>/*</url-pattern>
19.       </filter-mapping>
20.     </web-app>
```

Hibernate and Spring Integration

1. **Hibernate and Spring Integration**
2. **Example of Hibernate and Spring Integration**

We can simply integrate **hibernate application with spring application**.

In hibernate framework, we provide all the database information hibernate.cfg.xml file.

But if we are going to integrate the hibernate application with spring, we don't need to create the hibernate.cfg.xml file. We can provide all the information in the applicationContext.xml file.

Advantage of Spring framework with hibernate

The Spring framework provides **HibernateTemplate** class, so you don't need to follow so many steps like create Configuration, BuildSessionFactory, Session, beginning and committing transaction etc.

So **it saves a lot of code**.

Understanding problem without using spring:

Let's understand it by the code of hibernate given below:

copy to clipboard

```
1.      //creating configuration
2.      Configuration cfg=new Configuration();
3.      cfg.configure("hibernate.cfg.xml");
4.
5.      //creating seession factory object
6.      SessionFactory factory=cfg.buildSessionFactory();
7.
8.      //creating session object
9.      Session session=factory.openSession();
10.
11.     //creating transaction object
12.     Transaction t=session.beginTransaction();
13.
14.     Employee e1=new Employee(111,"arun",40000);
15.     session.persist(e1);//persisting the object
16.
17.     t.commit();//transaction is commited
18.     session.close();
```

As you can see in the code of sole hibernate, you have to follow so many steps.

Solution by using HibernateTemplate class of Spring Framework:

Now, you don't need to follow so many steps. You can simply write this:

copy to clipboard

```
1.      Employee e1=new Employee(111,"arun",40000);
2.      hibernateTemplate.save(e1);
```

Steps

Let's see what are the simple steps for hibernate and spring integration:

1. **create table in the database** It is optional.
2. **create applicationContext.xml file** It contains information of DataSource, SessionFactory etc.
3. **create Employee.java file** It is the persistent class
4. **create employee.hbm.xml file** It is the mapping file.

5. **create EmployeeDao.java file** It is the dao class that uses HibernateTemplate.
6. **create InsertTest.java file** It calls methods of EmployeeDao class.

Example of Hibernate and spring integration

In this example, we are going to integrate the hibernate application with spring.

1) create the table in the database

In this example, we are using the Oracle as the database, but you may use any database. Let's create the table in the oracle database

copy to clipboard

```
1.      CREATE TABLE  "EMP558"
2.        (    "ID" NUMBER(10,0) NOT NULL ENABLE,
3.         "NAME" VARCHAR2(255 CHAR),
4.         "SALARY" FLOAT(126),
5.          PRIMARY KEY ("ID") ENABLE
6.        )
7.      /
```

---

2) applicationContext.xml

In this file, we are providing all the informations of the database in the **BasicDataSource** object. This object is used in the **LocalSessionFactoryBean** class object, containing some other informations such as mappingResources and hibernateProperties. The object of **LocalSessionFactoryBean** class is used in the EmployeeDao class. Let's see the code of applicationContext.xml file.

copy to clipboard

```
1.      <?xml version="1.0" encoding="UTF-8"?>
2.      <beans
3.         xmlns="http://www.springframework.org/schema/beans"
4.         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5.         xmlns:p="http://www.springframework.org/schema/p"
6.         xsi:schemaLocation="http://www.springframework.org/schema/beans
7.        http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">
8.
9.
10.     <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
11.        <property name="driverClassName"
12.           value="oracle.jdbc.driver.OracleDriver"></property>
13.        <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"></proper
    ty>
14.        <property name="username" value="system"></property>
15.        <property name="password" value="oracle"></property>
16.     </bean>
17.
18.     <bean id="mysessionFactory"
```

```
19.     class="org.springframework.orm.hibernate3.LocalSessionFactoryBean">
20.       <property name="dataSource"><ref bean="dataSource" /></property>
21.
22.        <property name="mappingResources">
23.        <list>
24.        <value>employee.hbm.xml</value>
25.        </list>
26.        </property>
27.
28.        <property name="hibernateProperties">
29.         <props>
30.          <prop key="hibernate.dialect">org.hibernate.dialect.Oracle9Dialect</prop>
31.
32.         </props>
33.        </property>
34.     </bean>
35.
36.     <bean id="d" class="com.javatpoint.EmployeeDao">
37.        <property name="sessionFactory" ref="mysessionFactory"></property>
38.     </bean>
39.
40.     </beans>
```

---

3) Employee.java

It is a simple POJO class. Here it works as the persistent class for hibernate.

copy to clipboard

```
1.      package com.javatpoint;
2.
3.      public class Employee {
4.      private int id;
5.      private String name;
6.      private float salary;
7.
8.      //getters and setters
9.
10.     }
```

---

4) employee.hbm.xml

This mapping file contains all the information of the persistent class.

copy to clipboard

```
1.      <?xml version='1.0' encoding='UTF-8'?>
2.      <!DOCTYPE hibernate-mapping PUBLIC
3.      "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
4.      "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
5.
6.      <hibernate-mapping>
7.      <class name="com.javatpoint.Employee" table="emp558">
8.              <id name="id">
```

```
            <generator class="assigned"></generator>
            </id>

            <property name="name"></property>
            <property name="salary"></property>
    </class>

    </hibernate-mapping>
```

5) EmployeeDao.java

It is a java class that uses the **HibernateTemplate** class method to persist the object of Employee class. We are creating the instance of HibernateTemplate class by passing the **SessionFactory** object as the parameter in the HibernateTemplate class constructor.

copy to clipboard

```
    package com.javatpoint;

    import org.hibernate.SessionFactory;
    import org.springframework.orm.hibernate3.HibernateTemplate;

    public class EmployeeDao {
    HibernateTemplate template;

    public void setSessionFactory(SessionFactory factory) {
       template=new HibernateTemplate(factory);
    }

    public void saveEmployee(Employee e){
       template.save(e);
    }
    }
```

---

6) InsertTest.java

This class uses the EmployeeDao class object and calls its saveEmployee method by passing the object of Employee class.

copy to clipboard

```
    package com.javatpoint;

    import org.springframework.beans.factory.BeanFactory;
    import org.springframework.beans.factory.xml.XmlBeanFactory;
    import org.springframework.core.io.ClassPathResource;
    import org.springframework.core.io.Resource;

    public class InsertTest {
    public static void main(String[] args) {

       Resource r=new ClassPathResource("applicationContext.xml");
       BeanFactory factory=new XmlBeanFactory(r);

       EmployeeDao dao=(EmployeeDao)factory.getBean("d");
```

```
15.
16.         Employee e=new Employee();
17.         e.setId(114);
18.         e.setName("varun");
19.         e.setSalary(50000);
20.
21.         dao.saveEmployee(e);
22.
23.     }
24.     }
```

Now, if you see the table in the oracle database, record is inserted successfully.

Enabling automatic table creation, showing sql queries etc.

You can enable many hibernate properties like automatic table creation by hbm2ddl.auto etc. in applicationContext.xml file. Let's see the code:

copy to clipboard

```
1.      <property name="hibernateProperties">
2.              <props>
3.                 <prop key="hibernate.dialect">org.hibernate.dialect.Oracle9Dialect</
   prop>
4.                 <prop key="hibernate.hbm2ddl.auto">update</prop>
5.                 <prop key="hibernate.show_sql">true</prop>
6.
7.              </props>
```

If you write this code, you don't need to create table because table will be created automatically.

Spring and JPA Integration

1. **Spring and JPA Integration**
2. **Example of Spring and JPA Integration**

JPA (Java Persistent API) is the sun specification for persisting objects in the enterprise application. It is currently used as the replacement for complex entity beans.

The implementation of JPA specification are provided by many vendors such as:

- Hibernate
- Toplink
- OpenJPA etc.

In this example, we are going to use hibernate for the implementation of JPA.

Example of Spring and JPA Integration

Let's see the simple steps to integration spring application with JPA:

1. **create table in the database**
2. **create persistence.xml file**
3. **create Account.java file**
4. **create Account.xml file**
5. **create AccountDao.java file**
6. **create applicationContext.xml file**
7. **create AccountsClient.java file**

In this example, we are going to integrate the hibernate application with spring.

1) create the table in the database

In this example, we are using the Oracle as the database, but you may use any database. Let's create the table in the oracle database

copy to clipboard

```
1.    CREATE TABLE  "ACCOUNT100"
2.      (    "ACCOUNT_NUMBER" NUMBER,
3.       "OWNER" VARCHAR2(4000),
4.       "BALANCE" NUMBER,
5.        CONSTRAINT "ACCOUNT100_PK" PRIMARY KEY ("ACCOUNT_NUMBER") ENABL
   E
6.      )
7.    /
```

---

2) persistence.xml

copy to clipboard

```
1.    <?xml version="1.0" encoding="UTF-8"?>
2.    <persistence xmlns="http://java.sun.com/xml/ns/persistence"
3.       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4.       xsi:schemaLocation="http://java.sun.com/xml/ns/persistence
5.       http://java.sun.com/xml/ns/persistence/persistence_1_0.xsd" version="1.0">

6.
7.       <persistence-unit name="ForAccountsDB">
8.          <mapping-file>com/javatpoint/Account.xml</mapping-file>
9.          <class>com.javatpoint.Account</class>
10.      </persistence-unit>
11.   </persistence>
```

3) Account.java

It is a simple POJO class. Here it works as the persistent class for hibernate.

copy to clipboard

```
1.    package com.javatpoint;
2.
3.    public class Account
4.    {
5.       private int accountNumber;
6.       private String owner;
7.       private double balance;
8.
```

```
9.      public Account(){}
10.
11.     public Account(int accountNumber, String owner, double balance)
12.     {
13.        this.accountNumber = accountNumber;
14.        this.owner = owner;
15.        this.balance = balance;
16.     }
17.     public int getAccountNumber() {
18.        return accountNumber;
19.     }
20.     public void setAccountNumber(int accountNumber) {
21.        this.accountNumber = accountNumber;
22.     }
23.     public String getOwner() {
24.        return owner;
25.     }
26.     public void setOwner(String owner) {
27.        this.owner = owner;
28.     }
29.     public double getBalance() {
30.        return balance;
31.     }
32.     public void setBalance(double balance) {
33.        this.balance = balance;
34.     }
35.  }
```

4) Account.xml

This mapping file contains all the information of the persistent class.

copy to clipboard

```
1.    <entity-mappings version="1.0"
2.          xmlns="http://java.sun.com/xml/ns/persistence/orm"
3.          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4.          xsi:schemaLocation="http://java.sun.com/xml/ns/persistence/orm
5.          http://java.sun.com/xml/ns/persistence/orm_1_0.xsd ">
6.
7.       <entity class="com.javatpoint.Account">
8.          <table name="account100"></table>
9.          <attributes>
10.            <id name="accountNumber">
11.               <column name="account_number"/>
12.            </id>
13.            <basic name="owner">
14.               <column name="owner"/>
15.            </basic>
16.            <basic name="balance">
17.               <column name="balance"/>
18.            </basic>
19.         </attributes>
20.      </entity>
21.   </entity-mappings>
```

5) AccountDao.java

copy to clipboard

```
1.  package com.javatpoint;
2.
3.  import java.util.List;
4.  import org.springframework.orm.jpa.support.JpaDaoSupport;
5.  import org.springframework.transaction.annotation.Transactional;
6.
7.  @Transactional
8.  public class AccountsDao extends JpaDaoSupport
9.  {
10.    public void createAccount(int accountNumber,String owner,double balance)

11.    {
12.       Account account = new Account(accountNumber,owner,balance);
13.       getJpaTemplate().persist(account);
14.    }
15.    public void updateBalance(int accountNumber,double newBalance)
16.    {
17.       Account account = getJpaTemplate().find(Account.class, accountNumber);

18.       if(account != null)
19.       {
20.          account.setBalance(newBalance);
21.       }
22.       getJpaTemplate().merge(account);
23.    }
24.    public void deleteAccount(int accountNumber)
25.    {
26.       Account account = getJpaTemplate().find(Account.class, accountNumber);

27.       if(account != null)
28.          getJpaTemplate().remove(account);
29.    }
30.    public List<account> getAllAccounts()
31.    {
32.       List<account> accounts = getJpaTemplate().find("select acc from Account a
   cc");
33.       return accounts;
34.    }
35. }
36. </account></account>
```

6) applicationContext.xml

copy to clipboard

```
1.  <?xml version="1.0" encoding="UTF-8"?>
2.  <beans xmlns="http://www.springframework.org/schema/beans"
3.     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4.     xmlns:tx="http://www.springframework.org/schema/tx"
5.     xsi:schemaLocation="http://www.springframework.org/schema/beans
6.     http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
7.     http://www.springframework.org/schema/tx
8.     http://www.springframework.org/schema/tx/spring-tx-3.0.xsd">
9.
10.     <tx:annotation-driven transaction-manager="jpaTxnManagerBean" proxy-
   target-class="true"/>
11.
```

```
12.         <bean class="hbAdapterBean" class="org.springframework.orm.jpa.vendor.
    HibernateJpaVendorAdapter">
13.           <property name="showSql" value="true"></property>
14.           <property name="databasePlatform" value="org.hibernate.dialect.OracleDi
    alect"></property>
15.         </bean>
16.
17.         <bean id="dataSourceBean" class="org.springframework.jdbc.datasource.Dri
    verManagerDataSource">
18.           <property name="driverClassName" value="oracle.jdbc.driver.OracleDriver"
    ></property>
19.           <property name="url" value="jdbc:oracle:thin:@localhost:1521:xe"></pro
    perty>
20.           <property name="username" value="system"></property>
21.           <property name="password" value="oracle"></property>
22.       </bean>
23.       <bean id="emfBean" class="org.springframework.orm.jpa.LocalContainerEntit
    yManagerFactoryBean">
24.           <property name="dataSource" ref="dataSourceBean"></property>
25.           <property name="jpaVendorAdapter" ref="hbAdapterBean"></property>
26.         </bean>
27.
28.         <bean id="jpaTemplateBean" class="org.springframework.orm.jpa.JpaTempl
    ate">
29.           <property name="entityManagerFactory" ref="emfBean"></property>
30.         </bean>
31.         <bean id="jpaTxnManagerBean" class="org.springframework.orm.jpa.JpaTra
    nsactionManager">
32.           <property name="entityManagerFactory" ref="emfBean"></property>
33.       </bean>
34.
35.         <bean id="accountsDaoBean" class="com.javatpoint.AccountsDao">
36.           <property name="jpaTemplate" ref="jpaTemplateBean"></property>
37.         </bean>
38.
39.
40.    </beans>
```

7) Accountsclient.java

copy to clipboard

```
1.     package com.javatpoint;
2.
3.     import java.util.List;
4.     import org.springframework.context.ApplicationContext;
5.     import org.springframework.context.support.ClassPathXmlApplicationContext;
6.     import org.springframework.context.support.FileSystemXmlApplicationContext;
7.
8.     public class AccountsClient{
9.     public static void main(String[] args){
10.     ApplicationContext context = new ClassPathXmlApplicationContext("applicationC
    ontext.xml");
11.     AccountsDao accountsDao = context.getBean("accountsDaoBean",AccountsDao.c
    lass);
12.
13.     accountsDao.createAccount(15, "Jai Kumar", 41000);
14.     accountsDao.createAccount(20, "Rishi ", 35000);
15.     System.out.println("Accounts created");
```

```
16.
17.     accountsDao.updateBalance(20, 50000);
18.     System.out.println("Account balance updated");
19.
20.
21.     List<account> accounts = accountsDao.getAllAccounts();
22.     for (int i = 0; i < accounts.size(); i++) {
23.       Account acc = accounts.get(i);
24.       System.out.println(acc.getAccountNumber() + " : " + acc.getOwner() + " (" +
    acc.getBalance() + ")");
25.     }
26.
27.      //accountsDao.deleteAccount(111);
28.      //System.out.println("Account deleted");
29.
30.     }
31.    }
32.    </account>
```